Consensus Algorithms

Bridging Theory and Practice, Unifying Classical and Modern

Ling Ren
University of Illinois Urbana-Champaign, renling@illinois.edu

Draft – July 2026

Table of Contents

Preface

Why this book. Consensus is one of the most fascinating and extensively studied problems in computer science. The rise of cryptocurrencies has sparked unprecedented interest in the subject. At the same time, I have repeatedly heard the complaint that there is no good resource for beginners to learn the foundations of consensus. I certainly had my share of struggles when I first entered the field.

In principle, consensus should be relatively easy to learn. It requires only elementary mathematics and little prior background. A modest background in algorithms should be sufficient to appreciate most of its central ideas. Unfortunately, the available literature has not always reflected this. Writing a beginner-friendly textbook on consensus has therefore been on my agenda for many years.

There are many reasons why beginner-friendly materials are difficult to find. The seminal papers of the 1980s were written for researchers rather than newcomers. They also predate modern digital typesetting. Different papers often use inconsistent terminology. Classical textbooks, such as those by Lynch and by Attiya and Welch, cover the much broader field of distributed algorithms rather than consensus alone. While these factors all contribute to the difficulty of learning the subject, they are ultimately minor obstacles.

Two deeper factors stand out.

The first is the gap between theory and practice. For much of its history, consensus was primarily a theoretical subject. Most learning materials naturally focused on questions with the greatest theoretical significance. Some of these results, while theoretically elegant, have limited practical relevance. Conversely, some topics that are central to practice were not regarded as fundamental theoretical questions and received little attention. As a result, practitioners may find that existing materials are not the most effective path to acquiring the understanding they seek.

The second is the disconnect between recent developments and the classical foundations. Bitcoin pioneered a novel solution to the decades-old consensus problem. Unfortunately, this connection is often overlooked. Some treat Bitcoin primarily as an engineering achievement or a clever application of consensus, thereby overlooking its profound algorithmic innovations. A more prevalent view lies at the opposite extreme, regarding Bitcoin as an entirely new class of technology under the newly coined term *blockchain*. Once this perspective is adopted, previously known ideas can be mistaken for innovations, making it difficult to identify the genuine breakthroughs.

Addressing these two issues has been my primary goal when teaching consensus algorithms at the University of Illinois. In my lectures, I prioritize theories and algorithms that inform practice, and I present modern blockchain algorithms in a manner consistent with the classical foundations of consensus. The subtitle, *Bridging Theory and Practice, Unifying Classical and Modern*, reflects these two guiding principles. This book grew out of those lecture notes.

Scope. This book focuses on consensus in the message-passing model under crash or Byzantine faults. Byzantine faults are particularly relevant to cryptocurrencies and distributed ledgers and are therefore the primary focus of this book. Crash faults are also covered because of their importance in practical distributed systems and because they provide a natural stepping stone toward Byzantine fault tolerance. These settings encompass many of the most important ideas, influential algorithms, and fundamental impossibility results in consensus and distributed computing.

At the same time, consensus is a broad subject with a vast literature well beyond the scope of this book. In particular, this book does not cover the shared-memory model, wait-free consensus, consensus numbers, or other fault models. Rather than attempting to survey every branch of consensus, I have chosen depth over breadth. Even within this narrower scope, I include only those results that, in my view, are essential to developing a deep and thorough understanding of consensus.

The book is written to be accessible to beginners. No prior knowledge of distributed computing, cryptography, or related fields is assumed, although basic familiarity with algorithms will be helpful. Throughout the book, I have made every effort to simplify the algorithms, proofs, and notation without sacrificing rigor.

Much of the material in this book is drawn from seminal contributions by pioneers of the field. Other parts cover more recent developments. I acknowledge the original contributors whenever I am sufficiently familiar with the history, but I have omitted a bibliography in order to complete this first draft more quickly. The textbook by Attiya and Welch contains excellent historical notes and references for interested readers.

Acknowledgments. I am grateful to many wonderful mentors, colleagues, and collaborators: Dahlia Malkhi, Ittai Abraham, Elaine Shi, Alexander Spiegelman, Kartik Nayak, David Tse, Zhuolun Xiang, Sourav Das, Atsuki Momose, Julian Loss, Dongning Guo, Joachim Neu, Andrew Lewis-Pye, Tim Roughgarden, and Yuval Efron. It is through countless brainstorming sessions, discussions, and debates with them that my understanding of the field has gradually deepened.

I am grateful for the support of the National Science Foundation. Much of this book has been written during the period of Award #2143058.

Consensus has been shaped by decades of remarkable work from researchers, engineers, and one mysterious figure. I hope this book helps make those ideas more accessible to everyone interested in the subject.

CHAPTER 1
Introduction

Consensus is one of the most fundamental problems in distributed computing. At a high level, the goal is to design algorithms that enable a group of parties to reach a common decision, even if some parties fail, malfunction, or deliberately refuse to follow the algorithm.

Consensus is a critical component of reliable distributed systems deployed throughout the computing industry. It is the key technology behind cryptocurrencies and distributed ledgers (also known as blockchains). All these systems can be captured by an abstraction called *state machine replication*.

1.1 A Motivating Application: State Machine Replication

Let us consider a ticketing website that sells concert tickets. A computer, called a server, hosts the website. The server receives customer orders, processes payments, and confirms seat purchases to customers.

Unfortunately, no computer is perfectly reliable. If the server crashes, customers can no longer use the website to purchase tickets. A natural solution is to deploy multiple servers. If one server crashes, another can take over and keep the website running. With this solution, we have now moved to a *distributed system* that consists of multiple computers.

Making this distributed system work correctly is far from simple. Suppose two customers, Alice and Bob, attempt to purchase seat Q26 at roughly the same time, but their requests arrive at different servers. One server may sell the seat to Alice while the other sells it to Bob. This will be problematic when Alice and Bob both show up to the concert with valid (digital) tickets to seat Q26.

Even if we somehow ensure that only one server is taking orders at any given time, it still does not fully solve the problem. What if that server sells seat Q26 to Alice and then immediately crashes before informing the other servers? The remaining servers have no way of knowing that the seat has been sold and may later sell it to Bob. One might suggest that the server first inform the other servers and only then confirm the purchase with the customer. Unfortunately, this merely shifts the problem. What if the server crashes after informing only a subset of the other servers, and before sending the confirmation to Alice? Now Alice has no ticket, and the remaining servers disagree on whether seat Q26 has been sold. Recovering from such situations again requires the servers to agree on what has happened.

Consensus solves this exact problem. If the servers run a consensus algorithm among them, they will all *agree* on which customer, if any, successfully purchased seat Q26.

The above example illustrates a general approach to building reliable distributed systems. Any deterministic (non-distributed) service, program, or computation logic can be thought of as a *state machine*: it takes inputs from outside, updates its internal state, and produces outputs. We can replicate this state machine across multiple servers and run a consensus algorithm to ensure all servers agree on the sequence of inputs to the state machine. Because the state machine is deterministic, if all servers feed the same sequence of inputs to the state machine, they will always maintain the same internal states and produce the same sequence of outputs.

This approach is called *state machine replication* (SMR), and it is the most common and important application of consensus. For example, a decentralized cryptocurrency is a replicated state machine for a payment system. A blockchain supporting smart contracts (once we look past the non-descriptive terminology) is a replicated state machine capable of performing general-purpose computation.

When studying consensus, we always assume that some servers may be *faulty*. Faulty servers may crash, become unresponsive, make mistakes, or even intentionally sabotage the system. Moreover, we do not know which servers may be faulty. It is not hard to see the necessity of modeling faults this way. If any single server could be trusted to never be faulty, then we could run the entire service on that server, and there would be no need for replication or consensus in the first place. Viewed from this perspective, the SMR problem can be phrased succinctly as follows:

> How can we use several potentially faulty servers to provide the illusion of a single non-faulty server?

Remark 1.1.1 We use the terms parties, participants, and servers interchangeably. They have also been referred to as processes, replicas, nodes, acceptors, validators, or players in the literature. We use the terms algorithms and protocols interchangeably. A party that follows the protocol throughout the execution is said to be non-faulty, correct, or honest.

1.2 Defining the Consensus Problems

Although SMR is the most important and widely deployed application of consensus, we will not start our journey with it. Formally defining the SMR problem is surprisingly tricky, partly because it involves agreement on an infinitely growing sequence of values. Instead, we will begin with consensus on a single value.

The seminal paper by Lamport, Shostak, and Pease (1982) popularized the consensus problem through a fictional tale of Byzantine Generals.

> We imagine that several divisions of the Byzantine army are camped outside an enemy city, each division commanded by its own general. The generals can communicate with one another only by messenger. After observing the enemy, they must decide upon a common plan of action. However, some of the

> generals may be traitors, trying to prevent the loyal generals from reaching agreement. The generals must have an algorithm to guarantee that: All loyal generals decide upon the same plan of action.
>
> — Lamport, Shostak, and Pease, *The Byzantine Generals Problem* (1982).

Setting the medieval military metaphor aside, they distilled two foundational variants of the consensus problems, which form the starting point of our study.

1.2.1 The Broadcast Problem

Definition 1.2.1 (Broadcast) There are n parties, one of which is designated as the *sender* and has an input. Some parties, including the sender, may be faulty. An algorithm solves *broadcast* if it guarantees the following:

- ◇ All non-faulty parties output the same value.
- ◇ If the sender is non-faulty, then every non-faulty party outputs the sender's input.

In some settings, a broadcast algorithm can be used to solve SMR through simple composition (see Chapter 8.2.1). Broadcast is also an important primitive in its own right. It provides the abstraction of a broadcast communication channel, much like speaking to an audience in a room. Everyone in the room agrees on what the speaker said. The speaker (designated sender) cannot say different things to different listeners (i.e., cannot equivocate). The speaker can remain silent, in which case everyone agrees that nothing was said.

Although the two properties in the definition are natural, a few remarks are in order. First, what does it mean for a party to *output* a value? To whom does a party send its output? The right interpretation is that a higher-level distributed algorithm invokes the broadcast algorithm as a subroutine, and each non-faulty party locally outputs a value from the broadcast subroutine to the higher-level algorithm. Thus, an output is not always sent over the network. We do not discuss the outputs of faulty parties. A crashed party may fail to produce any output, while a malicious party may output any value as it wishes.

The second property is called *validity*. Besides being a natural property, it is also necessary for the problem to be meaningful. Without it, there is a trivial solution: every party simply ignores the sender and outputs 0. This solution satisfies the first property (all non-faulty parties output the same value), but is clearly uninteresting.

1.2.2 The Agreement Problem

Agreement is perhaps the most extensively studied consensus problem. In the agreement problem, every party has its own input. This is the main difference from the broadcast problem in which only one party, the designated sender, has an input.

Definition 1.2.2 (Agreement) There are n parties, each with an input. Some parties may be faulty. An algorithm solves *agreement* if it guarantees the following:

- ◇ All non-faulty parties output the same value.
- ◇ If all parties have the same input x, then every non-faulty party outputs x.

The first property is identical to that of broadcast. As in the broadcast problem, this property alone is insufficient, because a trivial algorithm in which every party ignores its input and outputs 0 satisfies it. We therefore again need a *validity* property to rule out this trivial solution.

The validity property in Definition 1.2.2 is called *strong unanimity* and is standard in the literature. But observe that it is a rather weak requirement. It merely states that if the parties have already agreed on a value x, then the agreement algorithm must preserve that agreement; otherwise, any common output is acceptable! Imagine a group of friends using an agreement algorithm to decide what to order for lunch. If everyone wants chicken, they will get chicken. But suppose some prefer chicken while others prefer beef. According to Definition 1.2.2, the agreement algorithm is allowed to output any agreed-upon outcome: chicken, beef, fish, vegetarian, and even skipping lunch altogether are all acceptable outcomes. This hardly seems satisfactory or useful.

Why, then, has agreement with strong unanimity become the most widely studied consensus problem? There are several plausible explanations. First, reaching consensus is difficult, so it is better to start with an easy variant. In fact, many natural and more useful validity properties are unattainable, especially in the presence of malicious faults. Second, agreement with strong unanimity may be a more useful building block than it appears to be, as we will see in later chapters (e.g., in Algorithms 6 and 18). Last but not least, agreement with strong unanimity seems to capture the essence of consensus more often than not, in the sense that the ideas and techniques developed for it have usually proved valuable to other more practical variants of consensus.

Remark 1.2.3 The terminology in the literature is *inconsistent*. The broadcast problem in one paper may be called the agreement problem or the consensus problem in another paper. There is a certain irony in the fact that the research community on consensus failed to reach consensus on its definition. At the same time, this may also serve as a reminder of the inherent difficulty of reaching consensus.

1.3 Models

The broadcast and agreement problems may still seem abstract at this point. Indeed, to study them rigorously, we must specify a formal model in detail. The precise formulation of the model is crucial. One intriguing feature of consensus is that its feasibility depends critically, and often surprisingly sensitively, on many aspects of the model. We now examine these aspects one by one.

1.3.1 Faults

As mentioned earlier, we always assume that some participants in a consensus algorithm may be faulty. That is, they may deviate from the algorithm they are asked to run. A *non-faulty* party follows the algorithm faithfully throughout the execution.

We use n to denote the total number of participants in an algorithm and f to denote the maximum number of faulty participants. In many settings, consensus is unattainable if every participant is faulty. We typically assume an upper bound on the number of

faulty participants. For example, we may assume $f < n/2$, that is, strictly fewer than half of the participants may be faulty.

Consensus algorithms prior to Bitcoin (2008) are designed for specific values of n and f, meaning that n and f appear explicitly as parameters of the algorithms. In particular, every non-faulty party knows the values of n and f. Needless to say, non-faulty parties (or the algorithm) cannot know *which* parties are faulty. If they did, consensus would be trivial: ignore the faulty parties and listen to the non-faulty party with the smallest ID.

Throughout this book, we mainly consider two types of faulty parties: crash and Byzantine.

Crash faults. A crash-faulty party, also called a crash fault for short, may stop executing at any time. Once it stops, it takes no further actions and never resumes execution.

We treat sending a message as an atomic action. That is, a message is either sent in its entirety or not sent at all. This is a reasonable assumption because we can assume that every message comes with error detection and correction mechanisms. If a party crashes in the middle of sending a message, the partial message will be detected as invalid and discarded by any non-faulty party receiving it. From the algorithm's perspective, this is equivalent to treating the message as having never been sent.

On the other hand, if a party intends to send messages to multiple recipients, it is possible that the party crashes unexpectedly during the process, resulting in some recipients receiving the message while other recipients do not. In fact, handling such a partial dissemination is a central challenge for algorithms that tolerate crash faults.

Byzantine faults. A Byzantine-faulty party, also called a Byzantine party or Byzantine fault, may behave arbitrarily and maliciously. If there are multiple Byzantine parties, we assume they can coordinate their actions perfectly.

Historically, Byzantine faults were introduced to capture arbitrary software errors and hardware failures, but that is arguably overkill. Byzantine faults are better suited to model an intelligent adversary that deliberately attempts to disrupt the system.

Throughout this book, we do not consider incentives or game-theoretic behaviors. Byzantine parties will do anything within their power to make an algorithm produce wrong outcomes, regardless of whether doing so benefits them. Conversely, honest parties always faithfully follow their prescribed algorithm.

1.3.2 Communication Model

We consider the message-passing model of distributed computing, in which parties communicate by sending messages to one another. Consensus has also been studied extensively in the shared-memory model, but that will not be our focus.

Unless otherwise specified, we assume the communication network is a complete graph, i.e., every party can directly send messages to every other party.

We assume that each pairwise communication link is an *authenticated* communication channel. This means, if party p receives a message from party q, the message must indeed have been sent by party q and could not have been forged or modified by any other party or entity.

We assume that every communication link provides *reliable message delivery*, meaning that every message sent eventually arrives at its destination. This assumption may

surprise readers familiar with computer networks. After all, it is common for a message to get lost in the network. We will revisit this issue.

1.3.3 Timing Model

The timing model deals with uncertainty in message delays. Observe that it takes time for a message to travel through the network and for the recipient to process it. It is convenient and customary to lump the processing time of a message into the transmission time, so that we can model message processing as instantaneous. There are several mainstream timing models. We begin with the simplest one.

Lockstep synchrony. The lockstep synchrony model assumes that all parties execute and communicate in perfectly synchronized rounds and that every message sent in a round is received by its recipient by the end of that round.

We give a more detailed description. All parties start the first round of the protocol simultaneously. At the beginning of the round, each party determines what messages, if any, to send to every other party. (An honest party may send different messages to different recipients if the protocol requires it; a Byzantine party may always send arbitrary messages.) At the end of the round, each party receives all messages sent to it in the round. Each party then processes the received messages, updates its local state, determines what messages (if any) to send in the next round, and decides whether to produce an output and terminate. Then, parties that have not terminated begin the next round together, and the process repeats.

The lockstep synchrony model is clearly idealized, but it is not unreasonable. The perfectly synchronized rounds can be emulated under the standard non-lockstep synchrony model (introduced next), albeit with some loss in efficiency. Thus, lockstep synchrony is widely used to simplify the presentation and analysis of synchronous algorithms.

Synchrony. The standard synchrony model assumes that *every* event takes a predetermined, bounded time to complete. In particular, every message, once sent, is guaranteed to arrive at and be processed by the recipient within a predetermined bounded amount of time.

Asynchrony. In the asynchrony model, *every* event can take an unbounded time to complete. In particular, every message may take an arbitrarily long time to reach its recipient and get processed.

It is important to note that the asynchrony model still assumes reliable message delivery, i.e., every message is guaranteed to arrive at its recipient eventually. Nevertheless, the asynchrony model alleviates the concern that reliable message delivery is an overly strong assumption. In modern networks, a lost message is retransmitted after some time. While individual transmission attempts may fail, it is hard to imagine a realistic network in which a message cannot be delivered after infinitely many retransmission attempts. Therefore, once the asynchrony model accounts for the potentially long delays caused by retransmission, assuming reliable message delivery becomes reasonable.

Partial synchrony. Synchrony and asynchrony represent two extremes. Note that there is a large gap between them. Synchrony requires *every* message to have a bounded delay, whereas asynchrony allows *every* message to have an unbounded delay. If a model assumes bounded delays for some messages and unbounded delays for other messages, it is neither synchrony nor asynchrony.

Most practical systems lie somewhere between these two extremes. The most widely studied intermediate model is *partial synchrony*. One common formulation assumes that the network alternates unpredictably between synchronous and asynchronous periods. (It also relaxes the reliable message delivery assumption; see Remark 5.1.3.) We study partial synchrony in detail in Chapter 5.

Remark 1.3.1 (Clock drift) Historically, timing models also discuss *clock drifts*, i.e., differences in clock speeds. Observe that a bound on message delay is meaningful only in conjunction with a bound on clock drift. If Alice's clock runs arbitrarily faster than Bob's, then even a message delivered shortly in real time may, from Alice's perspective, appear to have taken an arbitrarily long time. For this reason, historically, the synchrony model means bounded message delay plus bounded clock drift, whereas the asynchrony model means either unbounded message delay *or* unbounded clock drift. In modern computer systems, however, we rarely encounter clocks whose speeds differ so much that they warrant modeling with unbounded drift. We thus assume that clock drift is always bounded and focus exclusively on message delays in the timing model.

1.3.4 Cryptography

Cryptography plays an important role in defending against Byzantine adversaries. Although Byzantine consensus can be solved without cryptography, the resulting solutions often are more complex or tolerate fewer faults. The most important cryptographic primitive for consensus is *digital signature*. We will introduce other relevant cryptographic primitives as they become needed in later chapters.

At a high level, a digital signature scheme allows anyone to verify that a message was indeed sent by the purported sender. In more detail, in a digital signature scheme, a signer generates a pair of keys $(\mathsf{sk}, \mathsf{pk})$. The signer keeps the secret key sk to itself and publishes the corresponding public key pk. A digital signature scheme provides the following two algorithms:

- $\sigma \leftarrow \mathsf{Sign}(\mathsf{sk}, m)$. Using its secret key, the signer produces a signature σ on a message m of its choosing.
- $\mathsf{True}/\mathsf{False} \leftarrow \mathsf{Verify}(\mathsf{pk}, m, \sigma)$. Anyone can verify, using the public key, whether σ is a valid signature on m. Verify returns True if the signature is genuinely produced by the signer, and False otherwise.

Remark 1.3.2 Cryptography texts often refer to the pair of keys as the *signing key* and the *verification key* to emphasize their functionality and avoid confusion with asymmetric encryption. In this book, we use the terms *secret key* and *public key*.

To abstract away the details of digital signature schemes, we write $\langle m \rangle_p$ to denote a message m accompanied by a signature produced by party p. When another party q

receives $\langle m \rangle_p$, q first runs Verify using p's public key (we assume $\langle m \rangle_p$ carries metadata that identifies p as the signer). If Verify returns true, party q proceeds to process the message; otherwise, q discards the message.

It is very common for a consensus algorithm to require multiple parties to sign the same message. Let P be a set of parties. We write $\langle m \rangle_P$ to denote a message m accompanied by a signature from each party in the set P. With basic digital signatures, the size of $\langle m \rangle_P$ is proportional to the number of signers $|P|$. A more advanced primitive called *threshold signature* can make the size of $\langle m \rangle_P$ equal to that of a single signature.

In the early days of consensus research, many algorithms avoided digital signatures because they were computationally expensive and their security guarantees were not yet widely accepted. Today, digital signatures are relatively inexpensive and routinely deployed. Most modern consensus algorithms use digital signatures, and we will do the same throughout this book.

1.3.5 Other Aspects

Randomization. An algorithm is *randomized* if parties are allowed to make random choices (for example, by flipping coins) during its execution. Otherwise, the algorithm is *deterministic*. This distinction turns out to be fundamental. Several fundamental impossibility results and efficiency barriers apply only to deterministic algorithms and can be circumvented by introducing randomization.

Static vs. adaptive corruptions. In the *static corruption* model, the set of faulty parties is fixed before the algorithm starts. By contrast, in the *adaptive corruption* model, the adversary may choose which parties are faulty during the execution of the algorithm.

1.4 Efficiency Metrics

Parties in a distributed algorithm incur local computation and storage costs, but these are analyzed using the same techniques as in traditional (non-distributed) algorithms. We therefore focus on efficiency metrics unique to distributed algorithms, namely, the number of communication rounds and the amount of communication.

Round complexity. Round complexity refers to the number of communication rounds required by a distributed algorithm. Round counting is straightforward in lockstep synchrony. In asynchrony and partial synchrony, a formal definition of rounds can be subtle. Fortunately, for all algorithms we will encounter, round counting remains straightforward.

As an example, consider a simple algorithm in which a special leader party sends a message to every party, and every party replies with an acknowledgment. This simple algorithm uses two rounds and therefore has a round complexity of 2.

Communication complexity. Communication complexity measures the amount of information exchanged among parties. If all messages have roughly the same size, then it is natural to measure communication using *message complexity*, i.e., the total number of messages exchanged among parties. On the other hand, if message sizes vary greatly in

an algorithm, it makes sense to measure communication using *bit complexity*, the total number of bits exchanged among parties.

Using the previous example again, if the leader sends a message to n parties and receives an acknowledgment from each, the message complexity is $2n$. Assuming each message and acknowledgment has w bits, the bit complexity is $2nw$.

In the case of Byzantine faults, we only count messages or bits sent by honest parties. This treatment is both natural and necessary because nothing prevents Byzantine parties from sending unlimited messages. In the case of crash faults, messages and bits sent by a party before it crashes usually count toward the total communication cost.

Worst case, good case, and average case. Efficiency analysis of algorithms traditionally focuses on the *worst case*. In our text, the worst case would have f crash faults occurring in a strategic manner or f Byzantine faults perfectly coordinating their actions. In addition, the network selects delays for all messages in an adversarial and calculated manner.

For consensus algorithms intended for practical adoption, worst-case efficiency is usually not the primary concern. Although a consensus algorithm must withstand any meticulously crafted attack, in the vast majority of cases, such attacks do not occur. Many practical consensus algorithms instead optimize for the *good case* when certain favorable conditions are met, and accept poorer performance under elaborate attacks. This design rationale can be summarized as "prepare for the worst, hope for the best". Paxos and PBFT, which we cover in Chapter 5, are two of the earliest and most influential consensus algorithms intended for practical deployment. One of their defining characteristics is their priority on good-case efficiency. The definition of *good case* depends on the specific problem and setting, and we will elaborate on this in later chapters.

One may also consider the *average case*. For randomized algorithms, we often average over the randomness, which gives *expected* efficiency. For long-running algorithms that produce a sequence of consensus decisions, we often average over the sequence, which gives *amortized* efficiency. We will discuss these efficiency notions when applicable.

Do round and communication complexities reflect actual performance? Suppose we have a practical system whose bottleneck is the consensus algorithm. To some extent, round complexity correlates with the system's latency, especially when the system is not operating at full capacity, whereas communication complexity is correlated with the system's peak throughput.

The actual performance of a real-world system, however, will depend on many factors beyond these two clean theoretical efficiency metrics. For example, between two algorithms with roughly the same communication complexity, the one with a balanced workload across parties will likely outperform the one with a heavy bottleneck at a single party. Nevertheless, round and communication complexities are widely adopted efficiency metrics because they are clearly defined, easy to analyze, and reasonably aligned with practical performance.

CHAPTER 2
Consensus in Synchrony

Consensus is a challenging problem. It is instructive to study it first in the simplest setting. We begin with the broadcast problem with *crash faults* and *lockstep synchrony* (i.e., with perfectly synchronized rounds). We turn to Byzantine faults and the agreement problem later in the chapter.

2.1 Synchronous Broadcast with Crash Faults

Recall the broadcast problem introduced in Chapter 1.2. A designated sender holds an input. The goal is for every non-faulty party to output the same value, which must equal the sender's input if the sender is non-faulty. Let us first see why some strawman solutions do not work.

2.1.1 Initial Attempts

First attempt. The first idea that comes to mind is to have the sender send its input to all parties, and let each party simply output what it hears from the sender.

Unfortunately, this simple algorithm fails if the sender crashes after sending its input to some, but not all, parties. Let x denote the sender's input. Those who receive x from the sender output x. But what about those who receive no messages from the sender? They have no way of knowing the sender's input. The best they can do is output a special value $\perp$ indicating that no message was received from the sender. This outcome violates the first requirement of broadcast (Definition 1.2.1) as correct (i.e., non-faulty) parties output two different values: x and $\perp$.

Second attempt. To remedy this problem, one may try to add a forwarding round. In the second round, every party that has received a value from the sender in the first round forwards that value to all parties. This change fixes the previous scenario if the sender is the only party to crash. Parties that missed the sender's original message will receive the forwarded value, and all non-faulty parties output the sender's input.

However, the same problem can now occur in the forwarding round. Suppose the sender crashes after sending its input x to only one party, Alice. Alice should forward x to all other parties, but may instead crash after sending x to some, but not all, parties. This will again lead to some parties outputting x and others outputting $\perp$.

Third attempt. Can we fix the above problem by adding yet another forwarding round? Indeed, the resulting algorithm will be correct if up to two parties crash, but will fail in the case of three crashes. We leave the reasoning as an exercise.

These attempts point us to a correct algorithm. Each additional forwarding round seems to help the algorithm tolerate one more crash. Since we can design an algorithm for a known value of f (the maximum number of faulty parties), a natural idea is to add f forwarding rounds (following the initial sender-to-all round). This yields a correct broadcast algorithm that tolerates up to f crash faults. We next describe and analyze this algorithm.

2.1.2 An Algorithm in $f+1$ Rounds

The algorithm is given in Algorithm 1. In the first round, only the sender speaks, sending its input to all parties. For the next f rounds, parties forward what they receive. At the end of the algorithm, if a party has received a value, it outputs that value; otherwise, it outputs a special value $\perp$. The special value $\perp$ can be interpreted as "I believe the sender is faulty".

Algorithm 1: Synchronous broadcast with crash faults in $f+1$ rounds.

Round 1: Sender sends its input to all parties.
Rounds 2 to $f+1$: If a party receives x for the first time in the previous round, send x to all parties.
End of round $f+1$: If a party receives x in any round, output x; else, output $\perp$.

Correctness proof. Recall the definition of broadcast (Definition 1.2.1). We need to prove the algorithm provides the following guarantees.

- All correct parties output the same value.
- (Validity) If the sender is correct, all correct parties output the sender's input.

The validity property is immediate. If the sender is correct, which means it does not crash throughout the algorithm, then all correct parties receive the sender's input directly from the sender at the end of round 1 and output it at the end of the algorithm.

We focus on the first property. Since we are dealing with crash faults, any value received by any party in any round must be the sender's input. In other words, no value can be "invented". It follows that every correct party outputs either the sender's input x or the special symbol $\perp$. Thus, establishing the following lemma completes the correctness proof for Algorithm 1.

Lemma 2.1.1 If one correct party outputs the sender's input, then no correct party outputs $\perp$.

Proof. Suppose a correct party p outputs the sender's input x. There are two cases.

Case 1: party p receives x before the last round. In this case, p forwards x to all parties in the following round. Party p being correct means that p does not crash in the entire algorithm. Thus, all correct parties receive x and output x.

Case 2: party p receives x for the first time at the end of round $f+1$, the very last round. In this case, party p has no opportunity to forward x because the algorithm has

reached its end. We need a different strategy to prove that all correct parties receive and output x.

Let q_{f+1} be the party from whom p receives x in round $f+1$. Why does q_{f+1} send x so late? It must be that q_{f+1} received x for the first time in round f. Let q_f be the party from whom q_{f+1} receives x in round f. Why does q_f not send x earlier? It must be that q_f received x for the first time in round $f-1$. Continuing this argument, we obtain a sequence of parties $q_1, q_2, q_3, \ldots, q_f, q_{f+1}$ such that q_{j+1} receives x for the first time from q_j in round j.

Among these $f+1$ parties, at least one is correct, because there are at most f faulty parties. This correct party successfully sends x to all correct parties, so all correct parties receive and output x. □

Note that the correctness proof does not require any relation between n and f. In other words, the algorithm tolerates any number of crash faults.

Remark 2.1.2 While not needed for the proof, we note that only one scenario leads to Case 2 in the proof. The sender q_1 sends x only to q_2 and then crashes; q_2 sends x only to q_3 and then crashes; ..... q_f sends x only to q_{f+1} and then crashes. Finally, q_{f+1} remains correct and sends x to all remaining parties.

Efficiency analysis. The algorithm uses $f+1$ rounds. The algorithm uses at most n^2 messages, since every party sends at most one message to every other party.

2.2 Synchronous Broadcast with Byzantine Faults

Next, we describe a broadcast algorithm that tolerates Byzantine faults in lockstep synchrony. The algorithm is known as the Dolev-Strong algorithm, named after its inventors. The Dolev-Strong algorithm can be viewed as the Byzantine fault-tolerant variant of the forwarding-based Algorithm 1. But additional key techniques are needed, as Byzantine faults are significantly more challenging than crash faults.

Challenges posed by Byzantine faults. To appreciate the challenges posed by Byzantine faults, let us ask: what can go wrong with Algorithm 1 if there are Byzantine faults?

The correctness proof of Algorithm 1 relied on a crucial observation: only one value will ever be circulated among the parties. This is because everyone is benign. The sender sends at most one value, and all other parties merely forward.

With Byzantine faults, two problems immediately arise. First, a Byzantine sender can *equivocate*, i.e., send different values to different parties. Second, even if the sender is honest, another Byzantine party can "invent" values. Fortunately, both problems can be resolved using digital signatures (Chapter 1.3.4). With digital signatures, other parties cannot forge values on the sender's behalf, and a Byzantine sender that equivocates will be caught (more on this later).

Unfortunately, there is a third, subtler problem that is harder to address: a Byzantine party can cause disagreement by delaying forwarding. To elaborate, consider a scenario with two Byzantine parties: the sender and another party named Mallory. All the other

parties are honest. In the first round, the sender sends its input x (after signing it) only to Mallory, and remains silent thereafter. If Mallory were honest, it would immediately forward x in the second round. But the malicious Mallory can hold off until the very last round. In the last round, Mallory sends the sender-signed value x to Alice, but sends nothing to Bob. According to the algorithm, Alice outputs x, and Bob outputs $\perp$ since he receives no value throughout the algorithm. Since this is the last round, Alice does not have an opportunity to inform Bob. Notice that it is hard for Alice to catch Mallory's misbehavior. From Alice's perspective, it is possible that Mallory received x only in the last-but-one round and forwarded x honestly.

The Dolev-Strong algorithm. The crux of the Dolev-Strong algorithm is a clever use of digital signatures to force Byzantine parties to forward values promptly.

In the Dolev-Strong algorithm, a forwarded value must carry the sender's signature as proof that the value originated from the sender. In addition, a value received in round k is deemed valid only if it is signed by k distinct parties. If a non-sender party Bob receives in round k a value signed by k parties including the sender, Bob adds his own signature. The value now carries $k+1$ signatures and is valid when Bob forwards it in round $k+1$. However, if Bob tries to forward it in a later round, the value will not be considered valid because it does not carry enough signatures.

The Dolev-Strong algorithm is given in Algorithm 2. Recall that $\langle x \rangle_j$ denotes a message x signed by party j and $\langle x \rangle_P$ denotes a message x signed by every party in set P. The algorithm has one more important trick to keep communication low: each party forwards at most two values. We discuss it further in the efficiency analysis.

Algorithm 2: The Dolev-Strong broadcast algorithm.

A message $\langle x \rangle_P$ is *valid for round* k if $|P| = k$ and P includes the sender.

Round 1: The sender s signs its input x and sends $\langle x \rangle_s$ to all parties.

Rounds k **(**$2 \le k \le f+1$**):** If in round $k-1$, party i receives $\langle x \rangle_P$ valid for round $k-1$, party i *extracts* x. If x is the first or second value extracted by party i in the algorithm, party i signs x and sends $\langle x \rangle_{P \cup \{i\}}$ to all parties.

End of round $f+1$: If a party receives $\langle x \rangle_P$ valid for round $f+1$, it extracts x. If a party has extracted a single value during the entire execution, output that value; else, output $\perp$.

Correctness proof. The proof is similar to that of the crash-tolerant algorithm. If an honest party extracts a value in a middle round, it helps others extract that value in the next round (except for a corner case in Lemma 2.2.2). If an honest party extracts a value in the very last round, then $f+1$ parties claim to have already forwarded the value, and one of them is honest and has done it correctly.

Lemma 2.2.1 If one honest party extracts no value, then no honest party extracts any value.

Proof. Suppose toward a contradiction that an honest party Alice extracts no value, but an honest party Bob extracts at least one value x. There are two cases.

Case 1: Bob extracts x in a round prior to the last round. In this case, Bob would have forwarded x (after adding his own signature) in the next round. Then, Alice would have extracted x as well, a contradiction.

Case 2: Bob extracts x at the end of the last round. In this case, Bob receives x along with $f + 1$ signatures on x. One of these signatures comes from an honest party. An honest party signs x only when forwarding x in a valid message. Then, Alice would have extracted x, a contradiction. □

Lemma 2.2.2 If one honest party extracts a single value, then every honest party extracts a single value.

Proof. By Lemma 2.2.1, there cannot be an honest party that extracts no value. Suppose toward a contradiction that an honest party Alice extracts a single value y, but an honest party Bob extracts at least two values, one of which is $x \neq y$. There are two cases.

Case 1: Bob extracts x in a round prior to the last round. In this case, Bob would have forwarded x (after adding his own signature) in the next round, unless he has already forwarded two distinct values. Then, Alice would have extracted x or those two distinct values, leading to a contradiction.

Case 2: Bob extracts x at the end of the last round. The proof for this case is identical to Case 2 of Lemma 2.2.1. □

Lemma 2.2.3 If one honest party extracts at least two values, then every honest party extracts at least two values.

Proof. If another honest party extracts no value or a single value, it contradicts either Lemma 2.2.1 or Lemma 2.2.2. □

Theorem 2.2.4 The Dolev-Strong algorithm ensures all honest parties output the same value.

Proof. If an honest party outputs $x \neq \perp$, then x is the only value it extracts. By Lemma 2.2.2, all honest parties extract only x and output x. If an honest party outputs $\perp$, then it has extracted either no value or at least two values. In the former case, no honest party extracts any value; in the latter case, every honest party extracts at least two values; in either case, all honest parties output $\perp$. □

Theorem 2.2.5 The Dolev-Strong algorithm satisfies validity: if the sender is honest, then all honest parties output the sender's input.

Proof. Let x be the honest sender's input. At the end of the first round, all honest parties receive signed x from the sender, so they all extract x. An honest sender signs only its input and nothing else, so no honest party extracts any other value. Thus, all honest parties extract only x and output x. □

Note that the correctness proof does not require any relation between n and f. In other words, the Dolev-Strong broadcast algorithm can be parameterized to tolerate any number of Byzantine faults.

Efficiency analysis. The Dolev-Strong algorithm has $f+1$ rounds. Its message complexity is $O(n^2)$ since every party sends at most two messages to every other party.

Forwarding at most two values is crucial for achieving quadratic message complexity. A naïve variant without it, i.e., every party forwards every value it extracts, can have unbounded message complexity. A Byzantine sender can sign arbitrarily many distinct values, causing honest parties to extract and forward all of them.

Although the Dolev-Strong algorithm uses at most $2n^2$ messages, its bit complexity is asymptotically higher because a message can carry up to f signatures. If the sender's input has constant size and each signature has κ bits, then the bit complexity of Dolev-Strong is $O(n^2 f\kappa)$, or $O(n^3\kappa)$ if f is linear in n.

As we will see in later chapters, the Dolev-Strong algorithm is optimal in worst-case round complexity and asymptotically optimal in worst-case message complexity. Its worst-case bit complexity has a gap from known lower bounds and is a long-standing open problem.

2.3 Synchronous Agreement

We now turn to the agreement problem. We directly consider Byzantine faults. Recall the agreement problem (Definition 1.2.2): every party has an input, and honest parties must output a common value; furthermore, if all parties input the same value, then that value must be the common output (strong unanimity).

It is not meaningful to speak of the inputs of Byzantine parties since they can lie about their inputs. Thus, with Byzantine faults, the strong unanimity validity requirement applies as long as all *honest* parties have the same input.

2.3.1 Synchronous Agreement via Broadcast

In synchrony, with $f < n/2$ Byzantine faults, agreement can be solved using broadcast, which we just saw how to construct. The algorithm invokes n instances of the broadcast subroutine, each party serving as the sender of one instance to broadcast its input. Naturally, messages are annotated to make it clear which message belongs to which broadcast instance. After all n broadcast instances complete, each party outputs the most frequent broadcast value. Ties can be broken arbitrarily, provided that all honest parties follow the same deterministic tie-breaking rule.

Algorithm 3: Synchronous agreement via broadcast.

Run n instances of broadcast, with party i being the sender of the i-th instance.
Output the most frequent value among the n broadcast outputs.

Efficiency Analysis. The agreement algorithm in Algorithm 3 has the same number of rounds as the underlying broadcast algorithm. Note that the last step is not a communication round. Algorithm 3 is a factor of n more expensive than the underlying broadcast algorithm in communication complexity (in both messages and bits).

Correctness proof. We first prove the main property that all honest parties output the same value. All honest parties agree on the result of each broadcast instance. In other words, honest parties agree on a common *vector* of n values. Since all honest parties then pick from this common vector in the same way, i.e., the most frequent value (breaking ties in the same way), they all pick the same value as the final output.

We next prove the validity property of strong unanimity: if all honest parties have the same input, then that input is the agreed-upon output. A broadcast instance ensures that, if the sender is honest, the broadcast result is the sender's input. If all $n-f$ honest parties have the same input x, the results of these $n-f$ broadcast instances will be x. Since $f < n/2$, x is the unique most frequent value in the vector. Thus, all honest parties output x.

Note that the above proof relies crucially on $f < n/2$, i.e., fewer than half of the parties can be Byzantine and the majority of parties are honest. In fact, Algorithm 3 fails to satisfy strong unanimity if half of the parties are malicious. Suppose all honest parties have input x and $f = n/2$. All malicious parties can simply claim to have input x' and otherwise follow the algorithm faithfully. This will result in a tie between x and x' in the vector of broadcast results. If the malicious parties pick a value x' that wins the tiebreak against x, all honest parties output x' and strong unanimity is violated.

2.3.2 Fault Tolerance Bounds of Byzantine Agreement

This section shows that $f < n/2$ is the maximum number of Byzantine faults that can be tolerated for agreement (with strong unanimity validity).

Theorem 2.3.1 No algorithm can solve the agreement problem with strong unanimity (Definition 1.2.2) in the presence of $f \geq n/2$ Byzantine faults.

Proof. Divide the n parties into two roughly equal groups P and Q. If n is even, $|P| = |Q| = n/2 \leq f$; if n is odd, $|P| = (n+1)/2$, $|Q| = (n-1)/2$, and $f \geq (n+1)/2$ since f is an integer. In either case, $|Q| \leq |P| \leq f$, so either group may consist entirely of Byzantine parties. For any agreement algorithm, consider the following three scenarios.

Scenario I. All parties in P are honest and have input x. All parties in Q are Byzantine, claim to have input y, and otherwise follow the algorithm faithfully. As all honest parties have the same input x, by strong unanimity, all parties in P output x.

Scenario II. All parties in P are Byzantine, claim to have input x, and otherwise follow the algorithm faithfully. All parties in Q are honest and have input y. This is symmetric to Scenario I and, by the same argument, all parties in Q output y.

Scenario III. All parties are honest. All parties in P have input x, and all parties in Q have input y. Parties in P cannot distinguish Scenario III from Scenario I, and Parties in Q cannot distinguish Scenario III from Scenario II. Thus, parties in P output x and parties in Q output y. The algorithm fails to achieve consensus among honest parties. □

Remark 2.3.2 The theorem holds for deterministic and randomized algorithms alike. Although the proof does not explicitly discuss randomized algorithms, it is straightforward to extend it to do so.

This is the first *negative* result we have encountered. Notice that in the proof, important aspects of the model such as the timing model or the use of signatures did not

come up. Indeed, the impossibility holds even if each party's input is a single bit, the network is lockstep synchronous, and the algorithm uses signatures. Designing an algorithm that works for multi-bit inputs, under less favorable network conditions, or without using signatures is strictly more difficult and, hence, also impossible when there are $f \geq n/2$ Byzantine faults. More generally, when proving a negative result, making the target problem *easier strengthens* the result.

Combining Theorem 2.3.1 and Algorithm 3, we have that Byzantine agreement in synchrony and with signatures can be solved if and only if $f < n/2$.

2.3.3 Synchronous Broadcast via Agreement

Previously, we constructed synchronous agreement algorithms from synchronous broadcast algorithms. We now show that synchronous agreement algorithms can also help solve broadcast in synchrony. This converse direction may be less important, since we already know how to solve synchronous broadcast directly. Nevertheless, it is a good exercise and illustrates the connection between the two problems.

A simple transformation from agreement to broadcast is given in Algorithm 4. The sender sends its input to all parties to feed into an agreement instance. Algorithm 4 relies crucially on synchrony. We leave it as an exercise for the reader to identify why.

Algorithm 4: Synchronous Byzantine broadcast via Byzantine agreement.

In the first round, the designated sender sends its input to all parties.
In the second round, start a single instance of agreement. Each party inputs to the agreement subroutine what it received from the sender in the first round.
Output the agreement instance's output.

Correctness proof and efficiency analysis. Since the output comes directly out of an agreement instance, all honest parties agree on it. If the sender is honest and has input x, all honest parties receive x from the sender in the first round, and input x to the agreement instance. The strong unanimity property of agreement ensures that the output is x.

Algorithm 4 uses one additional round and n additional messages than the underlying agreement algorithm.

CHAPTER 3
Round Complexity Bounds

In Chapter 2, we have seen synchronous broadcast and agreement algorithms that finish in $f + 1$ rounds, where f is the maximum number of crash or Byzantine faults. In this chapter, we show that $f + 1$ rounds are necessary in the *worst case* for *deterministic* broadcast and agreement algorithms.

Modern large-scale decentralized systems often run consensus among thousands of participants and wish to tolerate one-half or one-third of faulty participants. The above result suggests that thousands of communication rounds are needed to reach a consensus decision in the worst case. Fortunately, there is still hope for low-latency consensus, as the $(f + 1)$-round worst-case lower bound does not preclude algorithms that require much fewer rounds in certain good cases or average/expected cases. The second half of this chapter covers the approach of good-case round complexity. We will show in later chapters how randomization and amortization circumvent the lower bound.

3.1 Worst-case Round Lower Bound

We show that the lower bound holds even under lockstep synchrony, crash faults, and one-bit inputs. It follows that $f+1$ rounds are also necessary in the worst case for harsher networks, Byzantine faults, or multi-bit inputs.

Theorem 3.1.1 Any deterministic broadcast or agreement algorithm that tolerates f crash faults has worst-case round complexity at least $f + 1$.

We prove this lower bound in the remainder of this section. The proof presented here follows the approach of Aguilera and Toueg. First, we need to introduce some concepts that are commonly used in impossibility proofs.

3.1.1 Configurations and Valency

Definition 3.1.2 (Configuration) A *configuration* is the union of states across all correct parties and all communication channels at a given step of an algorithm's run.

A configuration captures the entire state of the algorithm at a particular point during its execution. In our proofs, we only analyze algorithms that run in lockstep rounds. For these algorithms, it suffices to examine only the configurations at round boundaries. While a general definition of configuration must include channel states (to account for messages in transit), we can ignore them because no messages are in transit at round boundaries in lockstep synchrony.

An execution of an algorithm can be viewed as a sequence of configurations, each evolving into the next, eventually reaching a configuration in which every correct party has produced an output. (Parties need not output in the same round and can continue to take actions after outputting.) Since we consider only correct algorithms and executions permitted by the model, correct parties never output different values.

The key idea behind many impossibility proofs is to classify configurations according to the outputs they can still produce. This leads to the notion of *valency*.

Definition 3.1.3 (x-valency) A configuration C is x-valent if, in every configuration C' that evolves from C, x is the only value output by correct parties.

Definition 3.1.4 (univalency) A configuration is *univalent* if it is x-valent for some x.

We focus on the setting with binary inputs and binary outputs. Thus, a univalent configuration is either 0-valent or 1-valent.

Definition 3.1.5 (bivalency) A configuration is *bivalent* if it is not univalent.

In plain terms, a univalent configuration has its fate already decided. If a configuration is x-valent, then regardless of how the execution proceeds thereafter, every correct party is guaranteed to output x eventually. By contrast, the fate of a bivalent configuration remains undecided. Depending on how the execution unfolds, for example, which parties crash and when, correct parties may end up outputting 0 or outputting 1.

It is easy to see that every configuration evolving from an x-valent configuration must be x-valent. On the other hand, a bivalent configuration may evolve into a 0-valent configuration, a 1-valent configuration, or another bivalent configuration.

3.1.2 Proof of the Lower Bound

With the notions of configurations and valency in place, the overall proof strategy for Theorem 3.1.1 is now fairly clear. Intuitively, the algorithm cannot end in a bivalent configuration because, by definition, the output has not yet been determined. We first show that the initial configuration of broadcast or agreement can be bivalent. We then show that each additional crash fault can keep the algorithm in bivalency for one more round.

Lemma 3.1.6 Any broadcast algorithm has a bivalent initial configuration.

Here is the intuition behind the lemma. If the sender crashes immediately after the algorithm starts without sending any message, the result of any "sensible" broadcast algorithm should be $\perp$. In the case of binary inputs and outputs, the special output $\perp$ is also a single bit. Without loss of generality, suppose $\perp$ equals 0. Then, if the sender's input is 1, this initial configuration is bivalent, because the output can be either 0 or 1, depending on how the execution unfolds, for example, whether the sender crashes immediately or stays correct.

The above is not a rigorous proof because it assumes the algorithm outputs $\perp$ if the sender crashes right away. To prove a negative result that *no* algorithm can accomplish a task, we cannot make assumptions about the behavior of the algorithm beyond its specification. We give a rigorous proof below.

Proof. For the broadcast problem with binary inputs, there are only two initial configurations: the sender has input 0, or the sender has input 1. Call them C^0 and C^1.

Suppose towards a contradiction that both initial configurations are univalent. C^0 must be 0-valent: if the sender remains correct throughout the algorithm, the output must be 0 to satisfy the validity requirement. For the same reason, C^1 must be 1-valent.

Now, let the sender crash right after the algorithm starts, without sending any message. Starting from either C^0 or C^1, the resulting configuration is the same: the sender crashed without sending any message, so its input is no longer relevant. However, a 0-valent configuration and a 1-valent configuration cannot evolve into the same configuration, since any evolution of an x-valent configuration must remain x-valent. This is a contradiction, so at least one of the two initial configurations must be bivalent. □

Lemma 3.1.7 Any agreement algorithm has a bivalent initial configuration.

Proof. Suppose towards a contradiction that every initial configuration is univalent. For each $0 \leq i \leq n$, let C^i denote the initial configuration in which the first i parties have input 1, and the remaining parties have input 0.

C^0 is the initial configuration in which all parties have input 0. The strong unanimity validity property requires an agreement algorithm to output 0 regardless of what happens during the execution (subject to model constraints). In other words, C^0 is 0-valent. C^n is the initial configuration in which all parties have input 1. For the same reason, C^n is 1-valent.

Now consider the sequence of configurations C^i $(0 \leq i \leq n)$. Since C^0 is 0-valent and C^n is 1-valent, there must exist two adjacent configurations C^{j-1} and C^j such that C^{j-1} is 0-valent and C^j is 1-valent. Observe that C^{j-1} and C^j differ only in the input of party j. Now let party j crash right after the algorithm starts, before it sends any message. This crash leads C^{j-1} and C^j to the same resulting configuration, since party j never has an opportunity to reveal its input. But the 0-valent C^{j-1} and the 1-valent C^j cannot evolve into the same configuration, leading to a contradiction. □

Next, we prove that each additional crash can keep the algorithm in a bivalent configuration for one more round. Here, we add a constraint that in each round, at most one party crashes. We call it the *crash-sparse* constraint. The crash-sparse constraint restricts how faults can occur. It thus makes the problem easier and only strengthens the negative result. Therefore, proving a lower bound under the crash-sparse constraint establishes the same lower bound for the general setting without it.

Lemma 3.1.8 There is a bivalent configuration at the end of round $f - 1$.

Proof. We show by induction that, for every $0 \leq k \leq f - 1$, there exists a bivalent configuration C_k after k rounds. Lemma 3.1.6 and Lemma 3.1.7 establish the base case: an initial bivalent configuration C_0 exists.

For the inductive step, we show that a bivalent configuration C_{k-1} after $k - 1$ rounds can evolve into a bivalent configuration after k rounds $(1 \leq k \leq f - 1)$. Suppose towards a contradiction that all configurations after k rounds are univalent.

First, let C_k^* be the one-round evolution of C_{k-1} in which no further crash occurs. Without loss of generality, suppose C_k^* is 0-valent. Because C_{k-1} is bivalent, it must have a one-round evolution C_k^{**} that is 1-valent and involves one crash. (Recall that we

added the crash-sparse constraint.) Suppose party p crashes and fails to send messages to parties $\{q_1, q_2, \ldots, q_m\}$ in C_k^{**} in round k.

For each $0 \leq i \leq m$, let C_k^i denote the one-round evolution from C_{k-1}, in which party p crashes and fails to send messages to parties $\{q_1, q_2, \ldots, q_i\}$ in round k. Note that $C_k^0 = C_k^*$ and $C_k^m = C_k^{**}$. Because C_k^0 is 0-valent and C_k^m is 1-valent, there must exist two adjacent configurations C_k^{j-1} and C_k^j such that C_k^{j-1} is 0-valent and C_k^j is 1-valent.

Observe that the only difference between C_k^{j-1} and C_k^j is whether or not party p manages to send its round-k message to party q_j. Thus, C_k^{j-1} and C_k^j differ only in the state of party q_j. Now let party q_j crash before the next round starts. This crash leads C_k^{j-1} and C_k^j to the same resulting configuration, since party q_j never has an opportunity to inform others whether or not it heard from party p in round k. But a 0-valent configuration and a 1-valent configuration cannot evolve into the same configuration, leading to a contradiction. Therefore, there exists a bivalent configuration after round k. □

A reader may wonder why we have to stop the inductive proof at round $f-1$. Why not extend the above lemma to round f? This is because the bivalent C_{k-1} after $k-1$ rounds may have involved $k-1$ crashes (one per round), and the inductive step requires two crashes (p and q_j). With up to f crashes allowed, the inductive step from round $f-1$ to round f does not go through. This is not a proof artifact. In fact, it is indeed possible that all one-round evolutions of C_{f-1} lead to univalent configurations. In those configurations, f crashes have already occurred, so there is no more uncertainty left, fixing the valency of the configuration.

Therefore, the f-th round needs some special handling. The intuition is that, even if all configurations after f rounds are univalent, parties still cannot output because they do not know *which* univalent configuration they have reached.

Lemma 3.1.9 Starting from a bivalent configuration C_{f-1} after $f-1$ rounds, there is a one-round evolution in which not all correct parties output.

Proof. Suppose towards a contradiction that all correct parties output in all one-round evolutions of C_{f-1}. Let C_f^* be the one-round evolution of C_{f-1} in which no further crash occurs. Without loss of generality, suppose all correct parties output 0 in C_f^*. Since C_{f-1} is bivalent, it must have a one-round evolution C_f^{**} in which a crash occurs and all correct parties output 1. Suppose party p crashes and fails to send messages to parties $\{q_1, q_2, \ldots, q_m\}$ in C_f^{**} in round f. Parties outside $\{q_1, q_2, \ldots, q_m\}$ cannot distinguish C_f^{**} from C_f^*, as they receive identical messages (including from p) in both cases. Thus, they output 0 in C_f^{**}, which is a contradiction. □

Combining Lemmas 3.1.6, 3.1.8, and 3.1.9 proves Theorem 3.1.1 for broadcast. Combining Lemmas 3.1.7, 3.1.8, and 3.1.9 proves Theorem 3.1.1 for agreement.

3.2 Good-case Round Complexity

Chapter 1.4 motivated the notion of *good-case* efficiency. For broadcast, the most natural and important good case is when the sender is non-faulty. Some practical systems retain a leader (analogous to a sender) as long as it continues to make progress, replacing it only if it crashes or misbehaves. In other systems, leaders may have strong incentives to

behave correctly, such as maintaining reputation or earning protocol rewards. Therefore, it is reasonable to expect that the sender (or leader) is usually non-faulty. Of course, the algorithm does not know whether the sender is faulty. It must guarantee correctness even when the sender is faulty, but may prioritize efficiency for the common case in which the sender is non-faulty.

So far, we have assumed that parties *output* at the end of the last round, and also *terminate* (i.e., quit the algorithm) at that point. But we now distinguish these two notions. For good-case analysis, we measure the number of rounds until non-faulty parties *output*, not until they *terminate*. The goal is to have non-faulty parties output in just a few rounds. After outputting, a party may continue to participate in the algorithm because other parties may still need its help.

Crash broadcast in one round in the good case. Algorithm 1 already achieves optimal good-case round complexity in the presence of crash faults. Although the algorithm, as written, instructs parties to output after the last round, it is not hard to see that a party can safely output as soon as it receives a value. If the sender is correct, all correct parties receive the sender's input and output at the end of the first round.

3.2.1 Graded Broadcast

To construct a synchronous broadcast algorithm with two-round good-case output, we introduce a primitive called *graded broadcast*. In a graded broadcast algorithm, each party outputs a value and a *grade*. For our purposes in this section, we restrict the grade to be either 1 or 0.

Definition 3.2.1 There are n parties, one of which is designated as the *sender* and has an input. By the end of the algorithm, every correct party outputs a pair (x, g) where $g \in \{0, 1\}$. An algorithm solves *graded broadcast* if it guarantees the following:

- ◇ (Grade Consistency) If an honest party outputs $(x, 1)$, every honest party outputs either $(x, 1)$ or $(x, 0)$.
- ◇ (Validity) If the sender is honest and has input x, every honest party outputs $(x, 1)$.

The grade g can be viewed as a *confidence* level. In plain terms, a graded broadcast ensures the following. First, if the sender is honest, all honest parties output the sender's input with high confidence. Second, if one honest party outputs a value with high confidence, then all honest parties output that value, with either high or low confidence. Outside these two scenarios, honest parties are allowed to output conflicting values with low confidence.

Algorithm 5 gives a synchronous graded broadcast algorithm that tolerates $f < n/2$ Byzantine faults. Let s be the sender. Recall that $\langle x \rangle_Q$ denotes the value x signed by every party in the set Q. At a high level, the sender sends its signed input. Then, parties forward and sign what they receive from the sender. The forwarding step is crucial and serves to detect potential sender equivocation, that is, the sender signing and sending different values to different recipients. If a party p receives enough signatures on a value x *and* detects no sender equivocation, it outputs x with high confidence; Such a party p then forwards the signatures on x to other parties to help them output x with low confidence.

Algorithm 5: Synchronous graded broadcast with $f < n/2$ Byzantine faults.

Round 1: The sender s signs its input x and sends $\langle x \rangle_s$ to all parties.
Round 2: If party i receives $\langle x \rangle_s$ in round 1, then party i forwards $\langle x \rangle_s$ and sends $\langle x \rangle_i$ to all parties.
End of Round 2: If party i receives $\langle x \rangle_Q$ where $|Q| \geq n - f$ and no $\langle x' \rangle_s$ where $x' \neq x$, then party i outputs $(x, 1)$.
Round 3: If party i outputs $(x, 1)$, it sends $\langle x \rangle_Q$ where $|Q| \geq n - f$ to all parties.
End of Round 3: (only for parties who did not output): If it has received $\langle x \rangle_Q$ where $|Q| \geq n - f$, output $(x, 0)$ (breaking ties arbitrarily); else, output $(\perp, 0)$.

We prove that Algorithm 5 solves graded broadcast against up to $f < n/2$ Byzantine faults.

Proof. If the sender is honest and has input x, it sends $\langle x \rangle_s$ to all parties. All $n - f$ honest parties sign and send x, so they all receive $\langle x \rangle_Q$ with $|Q| \geq n - f$. No $\langle x' \rangle_s$ ever exists since the honest sender does not sign anything else. Thus, all honest parties output $(x, 1)$ at the end of round 2.

If an honest party p outputs $(x, 1)$ at the end of round 2, then no honest party signed x'. This is because if one ever did, then that honest party would have forwarded $\langle x' \rangle_s$ to party p, allowing p to detect sender equivocation and preventing p from outputting $(x, 1)$. Without signatures from honest parties, x' receives at most f signatures from Byzantine parties, which does not meet the $n - f$ threshold as $f < n/2$. On the other hand, p sends $\langle x \rangle_Q$ where $|Q| \geq n - f$ to all parties in round 3. Thus, all honest parties either output $(x, 1)$ in round 2 or output $(x, 0)$ in round 3, establishing grade consistency. □

3.2.2 Byzantine Broadcast in Two Rounds in the Good Case

Using graded broadcast as a subroutine, Algorithm 6 solves broadcast against up to minority Byzantine faults in synchrony and outputs in two rounds in the good case. The algorithm first runs a graded broadcast subroutine whose sender and input are the same as those of the broadcast. Parties that obtain high-confidence values immediately output it (but continue to participate in the algorithm). Next, all parties execute a Byzantine agreement subroutine, using the graded broadcast value (without the confidence component) as input. Parties that obtained only low-confidence values earlier output the agreement result.

Algorithm 6: A synchronous broadcast algorithm that tolerates minority Byzantine faults and outputs in two rounds in the good case. Code for party i.

Step 1: $(x_i, g_i) \leftarrow \mathsf{GB}()$ `// A graded broadcast subroutine`
If $g_i = 1$, output x_i.
Step 2: $y_i \leftarrow \mathsf{BA}(x_i)$ `// A Byzantine agreement subroutine`
If $g_i = 0$, output y_i.

We prove that Algorithm 6 solves the broadcast problem.

Proof. We first prove the algorithm ensures validity. If the sender is honest and has input x, then all honest parties output $(x, 1)$ from the graded broadcast according to Definition 3.2.1, and output x from the broadcast.

Next, we prove the algorithm ensures consensus among honest parties. If all honest parties have $g_i = 0$, then they all output the result of the agreement subroutine, which, by Definition 1.2.2, is identical across parties. If one honest party has $(x, 1)$ from graded broadcast, then by graded consistency, all honest parties have either $(x, 1)$ or $(x, 0)$. So they all input x to the agreement subroutine. The agreement result is x due to the strong unanimity validity of agreement. Any honest party who has $g_i = 1$ outputs x directly. Any honest party who has $g_i = 0$ outputs $y_i = x$. □

We have presented both subroutines needed by Algorithm 6 in the targeted setting of $f < n/2$ Byzantine faults and lockstep synchrony: graded broadcast (Algorithm 5) and Byzantine agreement (Algorithm 3).

Moreover, observe that Algorithm 5 produces grade-1 outputs at the end of round 2. Thus, in the good case with an honest sender, all honest parties output from the broadcast (Algorithm 6) after two rounds.

The two-round good-case output latency is optimal for Byzantine broadcast. We have the following theorem.

Theorem 3.2.2 No Byzantine broadcast algorithm can output in a single round, even in the best case where all parties are honest.

Proof sketch. Suppose the sender is Byzantine. In the first round, it sends 0 to some honest parties and 1 to the others. All other parties are honest and faithfully send their first-round messages (if any). From the perspective of an individual (non-sender) party, all parties appear honest. If honest parties were required to output after a single round in the best case, they would each output the value they received from the sender for validity, thereby breaking consensus. It is not hard to formalize this argument using the standard technique of indistinguishable executions. □

CHAPTER 4

Consensus in Asynchrony

In previous chapters, we studied synchronous algorithms for broadcast and agreement that tolerate minority (or more) crashes or Byzantine faults. We now turn to the asynchrony model. In an asynchronous network, every message is guaranteed to reach its recipient eventually, but it may take an arbitrarily long time.

Asynchrony turns out to be significantly more challenging. We are immediately confronted with two fundamental impossibility results: broadcast cannot be solved, and agreement cannot be solved deterministically.

Fortunately, these impossibility results point us toward three natural paths forward: (i) study easier problems, (ii) adopt stronger models, and (iii) use randomization (for agreement). We explore the first and third directions in the latter half of this chapter, and leave the second to the next chapter.

4.1 Impossibility of Asynchronous Broadcast

Broadcast, as defined in Definition 1.2.1, is impossible in asynchrony. Intuitively, the problem is that there is no way to distinguish a crashed sender from a correct sender experiencing long network delays. Waiting indefinitely for the sender may cause the algorithm to never output, whereas proceeding without hearing from the sender may violate the validity property if the sender turns out to be correct and just slow.

We now prove this impossibility theorem. The theorem applies to deterministic and randomized algorithms alike. The proof itself is quite simple. We use it as an opportunity to demonstrate how an impossibility proof should explicitly account for randomization. Many other impossibility results in this book also apply to randomized algorithms, but for simplicity we often omit the technical details needed to account for randomization.

To account for randomized algorithms, the theorem statement must explicitly specify a success probability. While a deterministic algorithm must succeed in every execution, a randomized algorithm is allowed to fail with a small probability. It is imprecise to claim that no algorithm ever succeeds, since an algorithm may succeed purely by chance. The theorem below states that every algorithm must fail with probability at least 1/3. Such a constant failure probability is far too large to be useful.

Theorem 4.1.1 No algorithm can solve broadcast in asynchrony with greater than 2/3 probability while tolerating a single crash fault.

Proof. Suppose toward a contradiction that such an algorithm exists. First consider a scenario in which the sender crashes before sending any messages, all other parties are

non-faulty, and all their messages are delivered instantly. Since the algorithm tolerates one crash fault, the non-faulty parties must eventually output a common value with probability greater than 2/3. Without loss of generality, suppose the output is 0 with probability at least 1/3.

Now consider another scenario in which all parties are non-faulty, the sender has input 1, all of the sender's messages are indefinitely delayed, and all other messages are delivered instantly. From the perspectives of non-sender parties, this scenario is indistinguishable from the previous one. Thus, they output 0, violating the validity of broadcast, with at least 1/3 probability. This contradicts the assumption that the algorithm solves broadcast with probability greater than 2/3. □

4.2 Impossibility of Deterministic Asynchronous Agreement

Theorem 4.2.1 (FLP) No deterministic agreement algorithm tolerates a single crash fault in asynchrony.

Theorem 4.2.1, first proved by Fischer, Lynch, and Patterson, is known as the FLP impossibility. We emphasize that the FLP impossibility applies only to deterministic algorithms.

The original FLP proof is ingenious but technically involved. Abraham and Stern (2024) introduced a new framework that significantly simplifies the proof. The proof presented below builds on their framework and adds further important refinements, rigor, and simplifications.

Lockstep synchrony with one mobile pause. It is customary to model an asynchronous network as an adversary that strategically controls the delay of every message in an attempt to break the algorithm. This high degree of uncertainty makes it challenging to reason about asynchronous algorithms.

The new framework instead returns to the simple and elegant model that is almost lockstep synchrony. At first sight, this seems impossible, since deterministic agreement algorithms are known to exist under lockstep synchrony. The crucial twist is that we grant the adversary an additional capability, which we call *one mobile pause*. In each round, the adversary may delay messages from exactly one party. Remarkably, this extra capability makes deterministic agreement impossible in an otherwise lockstep synchronous network. The new proof framework is not only simpler but also highlights that even a small amount of asynchrony, affecting a single party per lockstep round, renders deterministic agreement impossible.

We next describe the lockstep one-mobile-pause model in detail. An algorithm proceeds in lockstep rounds. Messages sent in a round arrive at their respective recipients by the end of that round, except that in each round, the adversary can pick one party as the *pause target* and exert some control over the delivery of its outgoing messages. Suppose the adversary chooses party p as the pause target in round r. The adversary can deliver some of p's outgoing messages normally and delay the rest of p's outgoing messages, but it must deliver p's messages in the *order* in which p sent them. To elaborate on this last point, suppose that in round r, p sends messages to parties $\{q_1, q_2, q_3, \ldots, q_m\}$ in that

order. The adversary can choose an index j, deliver p's messages to $\{q_1, q_2, \ldots, q_j\}$ and delay p's messages to $\{q_{j+1}, q_{j+2}, \ldots, q_m\}$.

In each round, the adversary can either stay with its current pause target or move to another pause target. If the adversary continues to pause p, p's outgoing messages in those rounds can also be delayed at the adversary's discretion, again with the constraint that p's outgoing messages (across all rounds) are delivered in the order they are sent. If the adversary moves to pause another party $q \neq p$ in some round $r' > r$, all of p's delayed messages are delivered by the end of round r'. The adversary can begin delaying q's outgoing messages, again with the in-order delivery constraint.

Reduction to asynchrony with one crash. Note that the lockstep one-mobile-pause model is just a proof technique. Our actual goal is to prove an impossibility result for the asynchrony model. Thus, we need to establish that lockstep synchrony with one mobile pause is strictly easier than asynchrony with one crash, formalized by the lemma below.

Lemma 4.2.2 If an algorithm fails under lockstep synchrony with one mobile pause, then it must also fail under asynchrony with one crash.

Proof. We show that an adversarial asynchronous network, with the help of a single crash, can simulate any strategy from a one-mobile-pause adversary in lockstep synchrony.

If every message is eventually delivered, the asynchronous adversary can clearly simulate the one-mobile-pause adversary's strategy, as it can control the delay of every message.

If at least one message, say from party p, is permanently delayed and never delivered, then at some point the one-mobile-pause adversary must make p its pause target forever. Find the earliest outgoing message of p that is never delivered. All subsequent messages from p are also never delivered due to the in-order delivery constraint. Let p crash right before sending that earliest never-delivered message. All other messages in the algorithm are eventually delivered, and the asynchronous adversary can easily simulate their respective delays. □

Note that the asynchronous adversary by itself is not strictly more powerful than the one-mobile-pause adversary, because the one-mobile-pause adversary can permanently delay some messages, whereas the asynchronous adversary cannot. The single crash takes care of permanently delayed messages. Also observe that a crash cannot cause an earlier message to be lost while delivering a later message sent by the same party. This is why the in-order delivery constraint on the one-mobile-pause adversary is crucial to the proof.

The impossibility proof. It remains to show that agreement cannot be solved deterministically under lockstep synchrony against a one-mobile-pause adversary. As we are again in the setting of lockstep rounds and deterministic algorithms, the proof is very similar to that of Theorem 3.1.1 in Chapter 3.1.

Recall the concepts of configuration, univalency, and bivalency from Chapter 3.1. That chapter has also proved that any agreement algorithm has a bivalent initial configuration (Lemma 3.1.7). Next, we show that the one-mobile-pause adversary can keep the algorithm in bivalency forever. The proof is similar to that of Lemma 3.1.8.

Lemma 4.2.3 For any bivalent configuration C (in any round), there exists a bivalent configuration C' that is a one-round evolution of C.

Proof. Suppose toward a contradiction that all one-round evolutions of C are univalent. Let C^* denote the one-round evolution of C in which all outstanding messages are delivered (including previously delayed ones, if any). Without loss of generality, suppose C^* is 0-valent. Because C is bivalent, it must have a one-round evolution C^{**} that is 1-valent and involves a pause target p. Let $[m_1, m_2, \ldots, m_\ell]$ be the sequence of p's outgoing messages that remain undelivered in C^{**}, listed in the order in which p sent them. Note that some of these messages may be sent in prior rounds.

For every $0 \leq i \leq \ell$, let C^i denote the one-round evolution from C in which the last i messages in the sequence remain undelivered. Note that $C^0 = C^*$ and $C^\ell = C^{**}$. Because C^0 is 0-valent and C^ℓ is 1-valent, there must exist two adjacent configurations C^{j-1} and C^j such that C^{j-1} is 0-valent and C^j is 1-valent.

The only difference between C^{j-1} and C^j is whether or not a particular message in the sequence is delivered. Let q_j be the recipient of that message. C^{j-1} and C^j differ only in the state of party q_j. The adversary now makes q_j its permanent pause target and delays all future messages sent by q_j indefinitely. This leads C^{j-1} and C^j to evolve into configurations that are indistinguishable to parties other than q_j. Eventually, parties output the same value in a configuration evolving from C^{j-1} and a configuration evolving from C^j. This is impossible, since any evolution of C^{j-1} is 0-valent, whereas any evolution of C^j is 1-valent. Therefore, C must have a one-round extension that is bivalent. □

It is also clear that an algorithm cannot output in a bivalent configuration. Combining Lemmas 3.1.7 and 4.2.3 proves that any deterministic agreement algorithm has an infinite run against the one-mobile-pause adversary. Finally, adding Lemma 4.2.2 proves Theorem 4.2.1, namely that any deterministic agreement algorithm that hopes to tolerate even a single crash fault in asynchrony must have an infinite run.

4.3 Deterministic Asynchronous Consensus Variants

Given that broadcast and deterministic agreement are impossible in asynchrony, a natural direction is to study *easier* variants of the consensus problem that can be solved deterministically in asynchrony. We cover two such problems in this section: reliable broadcast and graded agreement. Both frequently serve as important building blocks for more advanced asynchronous consensus algorithms.

We take a small digression to discuss safety vs. liveness. Generally speaking, we design algorithms to achieve certain desired properties, or in other words, to ensure that "some good outcomes happen". We can always rephrase a desired property as a pair of properties: (i) some outcomes happen, and (ii) bad outcomes never happen. The former is called a *liveness* property, and the latter is called a *safety* property. We can then prove an algorithm correct by establishing its liveness and safety separately.

As an example, consider the first property of both broadcast and agreement: all non-faulty parties output the same value. This property can be rephrased as a combination of two properties: "all non-faulty parties output" (liveness) and "non-faulty parties do not output different values" (safety).

For simple algorithms, there is often little benefit in separating safety and liveness, and we have not done so in the preceding chapters. But as we venture into more sophisticated algorithms, analyzing safety and liveness separately becomes both natural and convenient.

4.3.1 Reliable Broadcast

Definition 4.3.1 (Reliable broadcast) There are n parties, one of which is designated as the *sender* and has an input. An algorithm solves *reliable broadcast* if it guarantees the following:

- (Safety) Honest parties do not output different values.
- (Liveness) Either all honest parties output, or no honest party outputs.
- (Validity) If the sender is honest, every honest party outputs the sender's input.

Once we break down the first property of broadcast into safety and liveness, it is clear that reliable broadcast preserves the same safety and validity properties as broadcast while relaxing liveness. Broadcast requires all honest parties to eventually output. Reliable broadcast allows honest parties to *hang* indefinitely without producing outputs, but requires that honest parties either all hang or all output.

Asynchronous reliable broadcast tolerating crash faults is fairly straightforward and is left as an exercise. Algorithm 7 presents a simple reliable broadcast algorithm that tolerates $f < n/3$ Byzantine faults in asynchrony. As we will see in Chapter 6.3, the threshold $f < n/3$ is optimal for most asynchronous consensus problems including reliable broadcast.

The algorithm has the sender send its input to all parties for their signatures. (For convenience, we assume that the sender also sends its input to itself to trigger its own signing step.) A value signed by two-thirds of the parties becomes the output. Recall that $\langle x \rangle_Q$ denotes the value x signed by every party in the set Q.

Algorithm 7: A simple Byzantine reliable broadcast algorithm.

upon starting the algorithm
    The sender sends its input to all parties (including itself)

upon receiving x from the sender `// only for 1st message from sender`
    Party i sends $\langle x \rangle_i$ to all parties

upon receiving $\langle x \rangle_Q$ where $|Q| > 2n/3$
    Party i sends $\langle x \rangle_Q$ to all parties, outputs x, and terminates

Conventions for asynchronous pseudocode. Asynchronous algorithms are almost always *event-driven*. A party takes an action only when triggered by some event, such as receiving a message. This is reflected by the **upon** clauses in the pseudocode.

It is perfectly normal for parties to be in vastly different stages of the algorithm. Some fast parties may have already produced outputs, while a slow (but correct) party has yet to receive or deliver a single message.

It is also customary to allow parties to start the algorithm at different times. But we need to assume that if a message is delivered to a party before that party starts

the algorithm, the message will not be lost; instead, the message will be buffered and delivered to the party once it starts the algorithm.

Unless stated otherwise, each **upon** clause is triggered at most once. For example, in Algorithm 7, if a party receives multiple messages from the sender, only the first message triggers the signing step (the second **upon**). This convention is natural because an honest sender is supposed to send its input only once. Algorithm 7 has an explicit clarification to that effect, but in later chapters we will omit such clarifications.

Correctness proof. We prove that Algorithm 7 solves reliable broadcast against up to $f < n/3$ Byzantine faults in asynchrony.

Proof. Validity. If the sender is honest and has input x, it sends x to all honest parties. All honest parties send their signatures for x. There are at least $n - f$ honest parties. Thus, every honest party eventually receives $\langle x \rangle_Q$ where $|Q| \geq n - f > 2n/3$ and outputs x.

Liveness. If one honest party outputs, it sends $\langle x \rangle_Q$ for some x and $|Q| \geq 2n/3$ to all parties. Every honest party eventually receives this message and outputs.

Safety. Suppose toward a contradiction that one honest party outputs x and another honest party outputs $y \neq x$. A group Q_x of more than $2n/3$ parties signed x. A group Q_y of more than $2n/3$ parties signed y. We have

$$|Q_x \cap Q_y| \geq |Q_x| + |Q_y| - |Q_x \cup Q_y| > \frac{2n}{3} + \frac{2n}{3} - n = \frac{n}{3} > f.$$

This implies more than f parties signed both x and y. But only malicious parties sign two values, and there are at most f of them. We have obtained a contradiction. □

Quorum intersection. The safety proof above uses a *quorum intersection* argument, one of the most common techniques in consensus algorithm design and analysis. An honest party outputs a value only if it receives "votes" (signatures in this case) for that value from a quorum of parties. If every honest party votes for at most one value and the quorum size is large enough that the intersection of two quorums exceeds f parties, then safety is established against up to f Byzantine faults.

Efficiency analysis. Algorithm 7 uses three asynchronous rounds.

The communication bottleneck lies in the latter two rounds, during which every party sends a message to every other party. The message complexity is thus $O(n^2)$. As we will see in Chapter 7.1, this is asymptotically optimal for deterministic reliable broadcast algorithms.

The third-round message, $\langle x \rangle_Q$, has size linear in n (assuming basic signatures). The bit complexity is thus cubic in n. Chapter 7.2 presents a reliable broadcast algorithm with the optimal quadratic bit complexity.

4.3.2 Graded Agreement

Graded agreement is similar to graded broadcast, except that every party has an input and there is no designated sender. The grade consistency property is identical to that of graded broadcast (Definition 3.2.1). The validity property is analogous to that of

agreement (Definition 1.2.2): if all parties (not just correct parties) have the same input, then every correct party outputs that value with high confidence. We again restrict ourselves to only two grades.

Definition 4.3.2 There are n parties, each with an input. By the end of the algorithm, every correct party outputs a pair (x, g) where $g \in \{0, 1\}$. An algorithm solves *graded agreement* if it guarantees the following:

- (Grade Consistency) If a correct party outputs $(x, 1)$, every correct party outputs either $(x, 1)$ or $(x, 0)$.
- (Validity) If all parties have input x, every correct party outputs $(x, 1)$.

Although not always part of the graded agreement definition, another property called *weak agreement* is often useful. Let $(x, *)$ denote either $(x, 0)$ or $(x, 1)$.

- (Weak Agreement) If a correct party outputs $(x, *)$ where $x \neq \perp$, then no correct party outputs $(y, *)$ where $y \neq x$ and $y \neq \perp$.

Algorithm. We present in Algorithm 8 an asynchronous graded agreement algorithm that tolerates $f < n/2$ crash faults. The algorithm was introduced by Ben-Or (1983). Graded agreement can also be achieved against Byzantine faults in asynchrony, but the algorithms are considerably more involved and are omitted here.

Every party starts by sending its input to all parties (including itself). A party waits to receive inputs from $n - f$ parties. If all $n - f$ received inputs are the same value x, the party votes for x. Otherwise, the party votes for a special symbol $\perp$, which can be interpreted as *abstain* vote. The special symbol $\perp$ in this algorithm must be distinct from a regular value. This differs from several algorithms in previous chapters, where $\perp$ could be equal to a regular value. Finally, each party waits for $n - f$ votes and determines its output accordingly.

Remark 4.3.3 The threshold $n - f$ will frequently appear in asynchronous algorithms. This is the maximum number of messages a party can wait for at any step, since up to f faulty parties may never send the corresponding message.

Algorithm 8: An asynchronous graded agreement algorithm for binary inputs with up to $f < n/2$ crash faults.

upon starting the algorithm
Party i sends its input x_i to all parties

upon receiving inputs from $n - f$ parties
If all received inputs are x, then send (Vote, x) to all parties
Else, send $(\text{Vote}, \perp)$ to all parties

upon receiving $n - f$ Vote messages
If all of them vote for $x \neq \perp$, output $(x, 1)$ and terminate;
Else, if at least one of them votes for $x \neq \perp$, output $(x, 0)$ and terminate;
Else, output $(\perp, 0)$ and terminate.

Correctness proofs. Throughout the proofs below, let x and y denote two distinct values not equal to $\perp$, i.e., $x \neq y$, $x \neq \perp$, and $y \neq \perp$.

Lemma 4.3.4 If one party votes for x, then no party votes for y.

Proof. Suppose toward a contradiction that one correct party votes for x and another correct party votes for y. A group Q_x of $n-f$ parties send input x. A group Q_y of $n-f$ parties send input y. We have

$$|Q_x \cap Q_y| \geq |Q_x| + |Q_y| - |Q_x \cup Q_y| = 2(n-f) - n = n - 2f \geq 1.$$

This implies that at least one party sends both input x and input y, which is impossible. This is the quorum intersection argument for crash faults. □

Theorem 4.3.5 Algorithm 8 solves graded agreement in asynchrony against up to $f < n/2$ crash faults.

Proof. Liveness. There are at least $n-f$ correct parties, so every correct party eventually receives $n-f$ inputs and $n-f$ Vote messages, and eventually outputs.

Validity. If all parties have the same input x, then no party votes for $\perp$ or any value other than x. Combined with liveness, every correct party outputs $(x, 1)$.

Weak agreement. For a correct party to output x with either grade, there must be at least one vote for x. The same holds for y. Lemma 4.3.4 rules out this possibility.

Grade consistency. If a correct party outputs $(x, 1)$, a set S of $n-f$ parties must have sent (Vote, x). By Lemma 4.3.4, no party sends (Vote, y). When another party receives $n-f$ Vote messages, it misses at most f Vote messages from S. That party receives at least one (Vote, x) message because $|S| - f = n - 2f \geq 1$. Therefore, every correct party outputs either $(x, 1)$ or $(x, 0)$. □

Efficiency analysis. Algorithm 8 uses two asynchronous rounds. In both rounds, every party sends at most one message (of constant size) to every party, so the message complexity and bit complexity are both $O(n^2)$.

4.4 Randomized Asynchronous Agreement

The FLP impossibility only rules out *deterministic* agreement algorithms in asynchrony. Agreement can still be solved in asynchrony by *randomized* algorithms.

Algorithm 9 is a randomized asynchronous agreement algorithm due to Ben-Or (1983). The algorithm assumes binary inputs and requires asynchronous graded agreement (GA) that additionally ensures weak agreement. Outside the GA subroutine, the algorithm itself imposes no constraints on the number or type of faults. The previous section presented an asynchronous GA algorithm that tolerates up to $f < n/2$ crash faults. There also exist asynchronous GA algorithms that tolerate up to $f < n/3$ Byzantine faults.

The algorithm proceeds in iterations. Each iteration invokes a GA instance. A party's input to the first GA is its input to the agreement algorithm. A party's input to a subsequent GA is either its output from the last GA (if not $\perp$) or a random bit. A

high-confidence (i.e., grade-1) output from GA becomes the output of the agreement algorithm.

Algorithm 9, as presented, does not allow parties to terminate (they only output). There is a simple and standard gadget for achieving termination, but we omit it to keep the presentation focused on the core ideas.

Algorithm 9: A randomized asynchronous agreement algorithm.

```
// Code below is for party i.  Initially, x_i is party i's input.
upon beginning iteration k (iteration 1 begins when the algorithm starts)
    (y_i, g_i) ← GA(x_i)      // Start a new graded agreement instance
    If y_i ≠ ⊥, x_i ← y_i; Else, x_i ← a uniformly random bit.
    If g_i = 1, output y_i
    Begin iteration k + 1      // even after output
```

Correctness proofs. We recall the desired properties of agreement below. Note that we break "all parties output the same value" into liveness and safety.

- (Liveness) Every correct party outputs.
- (Safety) If two correct parties output x and y, respectively, then $x = y$.
- (Validity) If all parties have input x, all correct parties output x.

Lemma 4.4.1 For any iteration k, if every party i that enters iteration k has $x_i = x$, then every correct party outputs x from the agreement algorithm in iteration k.

Proof. Up to f parties may crash and never participate in iteration k. From the perspective of the parties that do participate, this execution is indistinguishable from one in which the other parties input x but crashed before sending any messages. The validity property of GA implies that every correct party outputs (x,1) from the GA in iteration k and outputs x from the agreement algorithm in iteration k. □

Validity directly follows from applying Lemma 4.4.1 to the first iteration. We next prove safety.

Proof of safety. Let k be the smallest iteration in which some correct party outputs. Let that party be p and its output be x. Then, p outputs $(x, 1)$ from the GA in iteration k. By graded consistency, every party that outputs from that GA instance must output either $(x, 0)$ or $(x, 1)$. Thus, every party i that enters iteration $k + 1$ sets $x_i = x$. By Lemma 4.4.1, all correct parties output x in iteration $k + 1$. □

The safety proof alone does not establish the correctness of Algorithm 9, because it assumes some correct party outputs. We still need to prove liveness to rule out the possibility that no correct party ever outputs.

The liveness proof turns out to be quite tricky. Intuitively, the algorithm eventually hits a *lucky* iteration as described in Lemma 4.4.1, from which liveness follows.

Intuition for liveness. Consider an iteration and its GA instance. There are two cases.

Case 1: If no party has a non-$\perp$ output value from the GA instance, then every party i that enters the next iteration sets x_i to a uniformly random bit. There is a (small) probability that all such parties choose the same random bit.

Case 2: If some correct party has a non-$\perp$ output value from the GA instance (with either grade), then no party has a different non-$\perp$ output value from that GA instance due to the weak agreement property. Some parties may output $(\perp, 0)$ from the GA and adopt uniformly random bits. With some (small) probability, the random bits generated by all those parties equal x.

Therefore, every iteration has a positive probability of being *lucky*. With probability 1, a lucky iteration eventually occurs. When that happens, every party that enters the next iteration has the same x_i. Lemma 4.4.1 then ensures that all correct parties output in the next iteration. □

The above intuition is not a rigorous proof because it implicitly assumes that the GA outputs are independent of the random bits generated by the parties. As it turns out, this independence assumption does not hold for the simple GA algorithm presented in the previous section. We refer interested readers to the paper and blog posts by Abraham, Ben-David, and Yandamuri for an in-depth discussion on this topic.

Efficiency analysis. Algorithm 9 has poor efficiency. The probability that all parties generate the same random bit in a given iteration is exponentially small in the number of parties n. Thus, the expected round complexity and expected communication complexity are both exponential in n.

Subsequent randomized asynchronous agreement algorithms overcome this inefficiency by using a primitive called a *common coin*. As the name suggests, a common coin primitive guarantees (possibly with a constant probability) that all parties obtain the same coin flip result. Replacing the local random bits in Algorithm 9 with a common coin reduces the expected round complexity to $O(1)$ and the expected communication complexity to $O(n^2)$.

CHAPTER 5

Consensus in Partial Synchrony

The previous chapter explored two directions toward consensus in asynchrony. One is to study easier variants of consensus, such as reliable broadcast and graded agreement. These weaker primitives typically serve as building blocks for other more demanding consensus problems. The other is to use randomization. As we have seen, randomized agreement algorithms are more complex and less efficient.

This chapter covers the third, perhaps the most widely adopted, approach to circumvent the impossibility of asynchrony: considering a less pessimistic network timing model. The most influential such model is *partial synchrony*, introduced by Dwork, Lynch, and Stockmeyer. We first introduce the partial synchrony model and then study two landmark consensus algorithms designed for it: Paxos and PBFT.

5.1 The Partial Synchrony Model

Recall that the asynchrony model assumes that *all* messages may be arbitrarily delayed. This assumption is often overly pessimistic. In practice, a network may experience occasional congestion or disruption, but it is unlikely to remain in a problematic state indefinitely.

Partial synchrony models a more realistic network that alternates between periods of instability and periods of synchrony. The following is a widely used formulation of partial synchrony.

Definition 5.1.1 (Partial synchrony) In a partially synchronous network, there is an unknown Global Stabilization Time GST and a known delay bound Δ such that every message sent after GST is delivered within Δ time.

A few remarks are in order to understand the above definition.

Unknown GST. What does it mean for the GST to be *unknown* and for the delay bound Δ to be known? To whom are these quantities known and unknown?

The answer is that they are known or unknown to the algorithm. A parameter known to the algorithm (such as Δ) can be, and usually is, hardcoded in the algorithm description and used by the parties running the algorithm. In contrast, a parameter *unknown* to the algorithm cannot appear in the algorithm description and cannot be used by the parties.

Here is another helpful way to understand this concept. Imagine a game between two players: an algorithm designer and an adversary. The adversary moves first by choosing the delay bound Δ and revealing it to the algorithm designer. The algorithm

designer then constructs a consensus algorithm using this information and hands it to the adversary. Finally, the adversary inspects the algorithm, chooses the value of GST, and attempts to find an execution in which the algorithm fails. If the adversary finds such a failing execution, the adversary wins; otherwise, the algorithm designer wins.

This game precisely captures the distinction between known and unknown parameters. Δ is fixed and provided to the algorithm designer beforehand. The algorithm can be tailored to that particular value of Δ. (The number of parties n and the fault bound f are likewise known parameters.) By contrast, GST is chosen only after the algorithm has been designed. The algorithm must work correctly for every possible value of GST.

Remark 5.1.2 Dwork, Lynch, and Stockmeyer (1988) also proposed another formulation of partial synchrony in which the delay bound Δ is unknown (and there is no GST). Only in 2024 did Constantinescu et al. prove the equivalence between the two formulations. This book adopts Definition 5.1.1, as the author finds it both more realistic and more convenient to work with.

Remark 5.1.3 It is worth noting that Definition 5.1.1 actually allows the pre-GST period to be even less well behaved than asynchrony. In particular, messages sent before GST may be lost, which is not allowed in asynchrony. In this sense, Definition 5.1.1 does *not* strictly strengthen the asynchrony model. That said, the literature generally treats partial synchrony as an intermediate between synchrony and asynchrony. Following this convention, we will also slightly abuse the terminology and refer to the pre-GST period as *asynchronous*.

Only one transition for one-shot consensus. Now that we understand the meaning of the unknown GST, the partial synchrony model can be summarized as follows: the network is asynchronous until an unknown time GST, after which it becomes synchronous.

A reader may object that it is unreasonable for a practically oriented model to assume that the network remains permanently synchronous after GST. In practice, a network may undergo many unstable periods.

This observation is correct. The one-time transition in Definition 5.1.1 is a theoretical simplification. It is particularly well suited for studying *single-shot* consensus problems, such as broadcast and agreement, in which only a single decision is made. For single-shot consensus, all that matters is that the network stays synchronous long enough for parties to reach a decision and possibly terminate. (A synchronous period that is too short can be regarded as part of the pre-GST period.) Any subsequent network instability is irrelevant. When we later study long-running consensus algorithms that make many decisions, the partial synchrony model should indeed be interpreted as having *recurrent* synchronous periods that are sufficiently long.

Safety before GST, Liveness after GST. Partially synchronous algorithms have a general structure. During asynchronous periods (or before GST), the algorithm must maintain safety but need not make progress toward a decision. Progress, and hence liveness, is guaranteed only during synchronous periods (or after GST). This structure is reflected in the correctness proofs. Safety and liveness are typically proved separately. The safety proof holds even in asynchrony. The liveness proof relies on the additional guarantees provided by partial synchrony and reasons about the execution after GST.

Efficiency metrics in partial synchrony. The efficiency of a partially synchronous algorithm should be measured only after GST. If rounds or communication are counted from the beginning of the execution, both may be unbounded because GST itself is unbounded, and an arbitrary number of rounds and amount of communication can occur before GST.

It is worth noting that algorithms in partial synchrony need not wait for the worst-case delay bound in every round. In a synchronous algorithm, we must allocate at least Δ time for each round, where Δ must be chosen conservatively so that it bounds the delay of every message. Following the philosophy of partial synchrony, most messages experience much shorter delays than this bound. We therefore distinguish between the *actual* message delay and the conservative delay bound. We use δ to denote the former and Δ to denote the latter.

The broadcast problem under partial synchrony. In this chapter, we study broadcast under partial synchrony, but the definition requires a small modification. Recall that Theorem 4.1.1 established that the standard broadcast problem (Definition 1.2.1) is unsolvable in asynchrony. A quick inspection of the proof shows that the same argument applies to partial synchrony. The crux of the proof is that, if the sender can be arbitrarily delayed, then a correct but slow sender is indistinguishable from a crashed one. As a result, liveness under a crashed sender and validity under a slow sender are fundamentally at odds. The partial synchrony model also faces this difficulty, because the sender can be arbitrarily delayed before GST. We therefore relax the validity property of broadcast as follows.

Definition 5.1.4 (Broadcast under partial synchrony) There are n parties, including a *sender* who has an input. An algorithm solves *broadcast* under partial synchrony if it guarantees the following:

- (Safety) Correct parties do not output different values.
- (Liveness) All correct parties output.
- (Validity) If the sender is correct, GST $= 0$, and parties start the algorithm within Δ time of one another, then all correct parties output the sender's input.

The only difference from the standard broadcast definition is that validity is required only if the network is synchronous from the beginning and parties start the algorithm around the same time. These additional conditions ensure that the sender is not delayed excessively.

We give two justifications for this relaxation. First, this relaxed validity property still rules out the trivial solution that ignores the sender's input and outputs a fixed value. More importantly, Definition 5.1.4 seems to capture the essence of consensus under partial synchrony, in the sense that algorithms solving this problem closely resemble those used in practice. In particular, the algorithms presented in the remainder of this chapter capture the essence of Paxos and PBFT, two landmark practical consensus algorithms.

Chapter 1.4 motivated *good-case* efficiency. For synchronous broadcast, the good case is when the sender is correct. Under partial synchrony, a correct sender alone is not useful if it is excessively delayed by an asynchronous period. It is thus natural to define the good case for partially synchronous broadcast to coincide with the conditions for validity: the sender is correct, GST $= 0$, and parties start the algorithm within Δ time of one another.

The additional conditions are consistent with the philosophy behind partial synchrony, namely that the network tends to be fast *most of the time* and experiences disruptions only occasionally.

5.2 Paxos

Paxos is one of the most influential consensus algorithms. Designed by Lamport, it is deterministic and tolerates up to a minority of crash faults (i.e., $f < n/2$) under partial synchrony. Paxos is a practical algorithm that achieves consensus on a sequence of values. In this chapter, we focus on its single-shot variant, which is sufficient to convey the algorithm's core ideas. The original Paxos paper by Lamport likewise began with the single-shot formulation.

5.2.1 Algorithm

Algorithm 10 presents a basic version of single-shot Paxos. To simplify the presentation further, we assume that all parties have perfectly synchronized clocks or, equivalently, access to a shared global clock. This is a strong assumption. It is not necessary for Paxos, as the algorithm can instead rely on much more realistic assumptions. We adopt this strong assumption because it simplifies several non-essential details and allows us to focus on the central ideas of Paxos.

The Paxos algorithm proceeds in iterations. Each iteration has a designated leader. The sender serves as the leader of the first iteration. Thereafter, the leader role rotates among parties in a round-robin order. In each iteration, the leader proposes a value together with the iteration number. The iteration number is also referred to in the literature as the view number, ballot number, epoch number, rank, and several other names. Every message carries the iteration number of the iteration in which it is sent.

Parties vote for the leader's proposal. For convenience, we assume the leader also sends its proposal to itself and votes for it. If the leader receives votes for its proposal from a majority of parties before the iteration ends, it decides (i.e., outputs) its proposed value and instructs the other parties to do the same.

A new leader cannot simply propose an arbitrary value of its choosing, because a previous leader *may* have already obtained enough votes to decide some value, unbeknownst to the new leader. The most intricate part of Paxos is how the new leader determines what value is *safe* to propose based on information gathered from other parties. To solve this problem, Paxos requires every party to keep track of the *highest* vote it has cast so far, where votes are ranked by the iterations in which they were cast. At the beginning of each iteration (except the first), each party reports its highest vote to the new leader. The new leader waits to hear from a majority of the parties and chooses the value in the *highest* vote among them. If none of those parties has previously cast a vote, the leader is free to propose any value.

Remark 5.2.1 We use more descriptive message names than the traditional Paxos terminology. Propose, Vote, and Decide messages are called Prepare, Promise, and Accept messages, respectively, in the literature.

Algorithm 10: Single-shot Paxos (assuming perfectly synchronized clocks).

upon starting the algorithm (at time 0)
Initialize a local variable highestVote $= (0, \perp)$ and enter iteration 1
L_1, the sender, sends $(\text{Propose}, 1, x)$ to all parties where x is its input

upon receiving $(\text{Propose}, k, x)$ while in iteration k
Set highestVote $= (k, x)$ and send (Vote, k, x) to L_k, the leader of iteration k

upon receiving (Vote, k, x) from $n - f$ parties while in iteration k
L_k sends (Decide, k, x) to all parties

upon receiving (Decide, k, x) `// even outside iteration` k
Output x, send (Decide, k, x) to all parties, and terminate

upon 3Δ time since entering iteration $k - 1$
Enter iteration k (quit iteration $k - 1$)
Send $(\text{Status}, k, \text{highestVote})$ to L_k

upon receiving $(\text{Status}, k, *)$ from $n - f$ parties while in iteration k
L_k sends $(\text{Propose}, k, x)$ to all parties, where x is the value in the highest vote reported in the $n - f$ Status messages, or $x = \perp$ if none reports ever voting

5.2.2 Correctness Proofs and Efficiency Analysis

Safety. As mentioned, the crux of Paxos lies in how a new leader ensures that its proposed value is consistent with any decisions made in previous iterations. Let us understand how Paxos achieves this.

At a high level, a decision requires votes from $n - f$ parties, and a new leader waits to hear from $n - f$ parties on how they voted. Since $n \geq 2f + 1$, these two quorums of $n - f$ parties must intersect (see proofs in Chapter 4.3.2). Therefore, if a value has been decided, the new leader is guaranteed to hear from at least one party whose vote contributed to that decision. The new leader then reproposes the decided value to ensure safety.

More concretely, let k be the first iteration in which a Decide message is sent, and suppose that message is (Decide, k, x). For this to happen, $n - f$ parties must have sent (Vote, k, x). Now consider iteration $k + 1$. Before making a proposal, the new leader L_{k+1} waits to receive Status messages from $n - f$ parties, each reporting the highest iteration in which the party voted (and the voted value). Among the $n - f$ parties who sent (Vote, k, x), at most f fail to send their Status messages to L_{k+1}, either because they crash or because their messages are delayed. Since $n - f - f = n - 2f \geq 1$, at least one Status message received by L_{k+1} reports the vote (Vote, k, x). This vote from iteration k is surely the vote with the highest iteration number at this point. So L_{k+1} reproposes x in iteration $k + 1$, and no party votes for any other value $x' \neq x$ in iteration $k + 1$.

The same argument now applies inductively. In iteration $k + 2$, the new leader L_{k+2} again hears from a majority of parties, one of whom reports a highest vote for x (from either iteration k or iteration $k + 1$). The leader again reproposes x. By induction, every subsequent leader can only repropose x. Therefore, any decision in subsequent iterations can only be x. We now give the full proof of safety.

Theorem 5.2.2 Algorithm 10 achieves safety.

Proof. A party decides (i.e., outputs) only upon receiving a Decide message. Thus, it suffices to prove that every (Decide, k, x) message carries the same x. Suppose k is the smallest iteration in which a Decide message is sent, and that message is (Decide, k, x). By definition, there is no $(\text{Decide}, k', *)$ message with $k' < k$. (Decide, k', x') can be sent only if $L_{k'}$ has sent $(\text{Propose}, k', x')$. Thus, it remains to prove that for all $k' \geq k$, if $L_{k'}$ sends a $(\text{Propose}, k', *)$ message, it must be $(\text{Propose}, k', x)$.

We prove by induction. The base case of $k' = k$ is straightforward. The leader sends at most one Propose message and one Decide message for the same value in its iteration. (Refer to the pseudocode convention in Chapter 4.3.1.) The existence of (Decide, k, x) implies that the unique proposal in iteration k is $(\text{Propose}, k, x)$.

For the inductive step, suppose no leader from iteration k to iteration $k' - 1$ sends a Propose message for any value other than x. The existence of (Decide, k, x) implies that a set S_1 of $n - f$ parties sent (Vote, k, x). Before $L_{k'}$ sends a proposal in iteration k', it waits to receive $(\text{Status}, k', *)$ messages from a set S_2 of $n - f$ parties. S_1 and S_2 intersect. The inductive hypothesis implies no party has voted for any other value in iterations k through $k' - 1$. Therefore, the Vote message with the highest iteration number reported in S_2 must be for x (possibly from an iteration higher than k). Hence, the only value that $L_{k'}$ can propose is x. □

Validity and liveness. Recall that a correct party is one that never crashes during the entire execution.

Theorem 5.2.3 Algorithm 10 achieves validity (as in Definition 5.1.4) and liveness.

Proof. Validity. Under the simplifying assumption of synchronized clocks, all parties start the algorithm at the same time, which we define as time 0. The correct sender proposes its input x at time 0. If $\text{GST} = 0$, every correct party receives the sender's proposal and votes for x by time Δ. The sender receives enough votes and sends $(\text{Decide}, 1, x)$ by time 2Δ. All parties receive the sender's Decide message and output x by time 3Δ. All of these happen within the first iteration.

Liveness. It suffices to prove that one correct party outputs. Once a correct party outputs, it sends a Decide message, and all correct parties eventually receive this Decide message and output. Suppose toward a contradiction that no correct party ever outputs.

Because parties take turns as leaders and GST eventually comes, the algorithm eventually reaches an iteration k that starts after GST and has a correct leader. At least $n - f$ correct parties send Status to L_k upon entering iteration k. L_k receives these and sends a proposal within Δ time into iteration k. All correct parties receive the proposal and send votes to L_k within 2Δ time into iteration k. L_k receives $n - f$ votes within 3Δ time into iteration k, at which point L_k outputs (upon receiving its own Decide message). This contradicts the assumption that no correct party ever outputs. □

Efficiency analysis. After GST, Algorithm 10 terminates once it reaches an iteration with a correct leader. Among the first $f + 1$ iterations after GST, at least one must have a correct leader. Thus, the algorithm uses $O(f)$ rounds after GST. Each iteration uses $O(n)$ messages of constant size: the leader sends Propose to each party, each party sends

Vote back to the leader, and the leader sends Decide to each party. The communication complexity in both messages and bits (after GST) is $O(n^2)$.

In the good case with a correct sender and GST = 0, Algorithm 10 terminates in the first iteration. Thus, its good-case round complexity is $O(1)$. More precisely, parties output after three rounds of communication and terminate after four. Observe that each round of messages is sent as soon as the necessary messages from the previous round have been received. Recall that we use δ to denote the actual message delay. The good-case output latency is therefore 3δ, not constrained by the conservative delay bound Δ.

The algorithm's good-case message complexity is still $O(n^2)$, rather than $O(n)$, because every party forwards a Decide message to every other party to ensure termination. However, when used in a practical state machine replication system, a consensus algorithm such as Paxos is not required to terminate, since the system is intended to operate indefinitely. If termination is not required, non-leader parties do not have to forward Decide. In that case, the algorithm uses only $3n$ messages in the good case.

5.2.3 Variations

Paxos is a delicate algorithm. A good way to test one's understanding is to think about which modifications break the algorithm and which lead to correct variations. We examine several such variations below.

To start, Algorithm 10 favors conceptual simplicity and includes some unnecessary fields in certain messages. For example, the Vote message need not carry the value being voted. It suffices to include the iteration number, since the Vote message is sent to the iteration's leader, who knows the value it proposed.

The iteration number can be dropped from a Decide message, since the message takes effect (makes the recipient decide) even after the corresponding iteration ends. Why should a Decide message take effect after its iteration? Observe that the leader sending the Decide message has already decided (it received its own Decide message right away). To preserve agreement, other parties must honor the Decide message even if it is received much later.

In Algorithm 10, parties send votes only to the leader. In this design, the leader decides first, and other parties decide one round later. (Algorithm 10, as written, has the leader send Decide to itself, but that is just for ease of exposition.) An alternative design is for each party to send its Vote message to all parties. This will allow non-leader parties to decide one round earlier, reducing the good-case round complexity from 3 to 2. The trade-off is that all-to-all voting increases the number of messages per iteration from $O(n)$ to $O(n^2)$.

The crux of the safety proof is a quorum intersection argument: the deciding quorum S_1, whose votes lead to a decision, intersects the status quorum S_2, whose Status messages are collected by a new leader. A party in the intersection informs the new leader of the potentially decided value, ensuring that it is reproposed. We often target $n = 2f + 1$, which is the optimal crash fault tolerance in partial synchrony (see Chapter 6.2). In this case, we must set $|S_1| = |S_2| = n - f = f + 1$. If either set exceeds $n - f$, f crashes break liveness. If either set is smaller than $n - f$, S_1 and S_2 may become disjoint, and the algorithm may lose safety. When $n > 2f + 1$, choosing $|S_1| = |S_2| = n - f$ remains valid, but any pair of quorum sizes satisfying $|S_1| \leq n - f$, $|S_2| \leq n - f$, and $|S_1| + |S_2| > n$ also works.

5.3 PBFT

PBFT, designed by Castro and Liskov in 1999, is a deterministic consensus algorithm that tolerates up to $f < n/3$ Byzantine faults under partial synchrony. Like Paxos, PBFT achieves consensus on a sequence of values, but we again present a single-shot variant for ease of exposition.

Although PBFT and Paxos were developed independently, the two algorithms share many design principles. It is helpful to view PBFT as the Byzantine Fault-Tolerant (BFT) counterpart to Paxos. Our presentation of PBFT in this section will also closely parallel the previous section on Paxos. Although this section is self-contained, readers are encouraged to study the Paxos section first before moving to PBFT.

5.3.1 Algorithm

Algorithm 11 presents a single-shot version of PBFT. The algorithm proceeds in iterations, also called views. Each view has a designated leader. The sender serves as the leader of the first view. Thereafter, the leader role rotates among parties in a round-robin order. In each view, the leader proposes a value together with the view number. (Every other message likewise includes the view number.) Parties vote on the leader's proposal in two rounds. $n - f$ round-1 votes on a proposal form a *quorum certificate* (QC) for the proposal and trigger the second round of voting. $n - f$ round-2 votes on a proposal make parties decide (i.e., output) the proposed value.

Reproposing the value with the highest QC. The most intricate part of PBFT is how a new leader determines which value is *safe* to propose and how it *convinces* other parties that its proposed value is safe. To this end, each party keeps track of the *highest* QC (ranked by view number) for which it has cast a round-2 vote. At the beginning of each view (except the first), each party reports its highest QC to the new leader. The new leader waits to hear from $n - f$ parties and chooses the value in the *highest* QC it receives.

This idea closely parallels Paxos. But there is a crucial difference: PBFT deals with Byzantine faults, so we cannot "take a party's word for it". In particular, the new leader must convince other parties that the value it proposes indeed comes from the highest QC. To do so, the leader attaches the $n - f$ Status messages it received to the proposal, so that other parties can check for themselves that the new leader followed the protocol. Because the leader forwards Status messages, they must be signed. Likewise, both rounds of votes are forwarded (either as part of QCs or to facilitate termination) and must also be signed. Recall that $\langle m \rangle_Q$ denotes the message m signed by every party in the set Q.

View synchronization. To guarantee liveness, honest parties must eventually spend sufficient time in the same view after GST. Synchronized clocks trivially ensure all honest parties are always in the same view. But we do not make this assumption in this section.

Remark 5.3.1 Readers may find it helpful to first reason about a simplified version of PBFT that assumes perfectly synchronized clocks. With that assumption, the last two upon clauses (which handle the view transition logic) can be removed, and every party simply advances to the next view every 4Δ time.

Without synchronized clocks, we need another way to bring parties into the same view. This is known as the *view synchronization* problem. It can be solved using what we call *view-change certificates*. An honest party enters a new view upon obtaining the corresponding view-change certificate. It then sends the view-change certificate to other parties to bring them into the same view.

At the same time, we must not allow Byzantine parties to end an honest leader's view prematurely (after GST). This is why a view-change certificate comprises $f + 1$ view-change requests, which implies that at least one honest party has a legitimate reason to leave the view. An honest party requests a view change after spending sufficient time in the view. Optionally, it may also request a view change if it detects that the leader is Byzantine (e.g., sending malformed messages or proposing multiple values).

Algorithm 11: Single-shot PBFT.

upon starting the algorithm
Initialize a local variable highestQC $= \emptyset$ and enter view 1
L_1, the sender, sends $(\text{Propose}, 1, x, \perp)$ to all parties where x is its input

upon receiving $\langle \text{Status}, k, * \rangle_*$ from $n - f$ parties while in view $k > 1$
L_k, the leader of view k, sends $(\text{Propose}, k, x, \mathcal{S})$ to all parties, where $\mathcal{S}$ is the $n - f$ $\langle \text{Status}, k, * \rangle_*$ messages received and x is chosen as follows:

- If every message in $\mathcal{S}$ reports highestQC $= \emptyset$, then $x = \perp$;
- Else, x is the value in the highest quorum certificate reported among $\mathcal{S}$.

upon receiving $(\text{Propose}, k, x, *)$ from L_k while in view k
If the proposal satisfies the above rules, party i sends $\langle \text{Vote1}, k, x \rangle_i$ to all parties

upon receiving $\langle \text{Vote1}, k, x \rangle_Q$ where $|Q| = n - f$ while in view k
Set highestQC to the quorum certificate $\langle \text{Vote1}, k, x \rangle_Q$
Party i sends $\langle \text{Vote2}, k, x \rangle_i$ to all parties

upon receiving $\langle \text{Vote2}, k, x \rangle_Q$ where $|Q| = n - f$ `// even outside view` k
Output x, send $\langle \text{Vote2}, k, x \rangle_Q$ to all parties, and terminate

upon spending 4Δ time in view k or detecting misbehavior of L_k
Party i sends $\langle \text{ViewChange}, k \rangle_i$ to all parties and $\langle \text{Status}, k, \text{highestQC} \rangle_i$ to L_{k+1}

upon receiving $\langle \text{ViewChange}, k \rangle_Q$ where $|Q| = f + 1$ while in view k or a lower view
Send $\langle \text{ViewChange}, k \rangle_Q$ to all parties and enter view $k + 1$ (quit current view)

Remark 5.3.2 We use more descriptive message names than the traditional PBFT terminology. Propose, Vote1, and Vote2 messages are called Preprepare, Prepare, and Commit messages, respectively, in the literature.

5.3.2 Correctness Proofs and Efficiency Analysis

Safety. As mentioned, the crux of PBFT lies in how a new leader proposes a value that is consistent with any decisions made in previous views and how it convinces other parties of this fact.

At a high level, for a party to decide (i.e., output) x, $n - f$ parties must receive a

quorum certificate (QC) for x and send Vote2. Before making a proposal, a new leader collects the highest QC from $n - f$ parties. With $n \geq 3f + 1$, the quorum intersection argument (Chapter 4.3.1) guarantees that at least one honest party relays the QC for x to the new leader. The new leader must repropose the value with the highest QC (and prove that it did so) to ensure safety. We can then prove, by induction, that once a value has been decided, only that value can be legally proposed by subsequent leaders. We now give the full proof of safety.

Theorem 5.3.3 Algorithm 11 achieves safety.

We use $\mathcal{C}_k(x)$ as a shorthand for a quorum certificate (QC) for x from view k, i.e., $\langle \text{Vote1}, k, x \rangle_Q$ where $|Q| = n - f$. With a slight abuse of terminology, we say a party decides x in view k if it decides upon receiving $\langle \text{Vote2}, k, x \rangle_{Q_2}$ where $|Q_2| = n - f$, even though this can happen *outside* view k.

Proof. Suppose k is the lowest view in which some honest party decides, and let the decided value be x. By the definition of k, no party outputs in any view $k' < k$. For a party to decide x in a view, there must be a QC for that value from that view. Thus, it suffices to prove that no $\mathcal{C}_{k'}(y)$ where $k' \geq k$ and $y \neq x$ exists.

We prove by induction. The base case of $k' = k$ is straightforward. Each honest party sends at most one Vote1 message per view. (Recall the pseudocode convention discussed in Chapter 4.3.1.) The standard quorum intersection argument (Chapter 4.3.1) shows that there cannot be two QCs from the same view for different values.

For the inductive step, suppose all QCs from view k to view $k' - 1$ are for the value x. For an honest party to decide x in view k, a quorum Q_2 of $n - f$ parties must have seen $\mathcal{C}_k(x)$. Suppose $L_{k'}$ sends $(\text{Propose}, k', y, \mathcal{S})$. Again, by the quorum intersection argument between Q_2 and $\mathcal{S}$, at least one honest party saw $\mathcal{C}_k(x)$ and has its $(\text{Status}, k', *)$ message included in $\mathcal{S}$. By the inductive hypothesis, there exists no QC for $y \neq x$ from views k through $k' - 1$. Therefore, the highest QC in $\mathcal{S}$ is either $\mathcal{C}_k(x)$ or an even higher QC for x. $L_{k'}$ must send $(\text{Propose}, k', x, \mathcal{S})$; otherwise, no honest party will vote for its proposal in view k'. Either way, no $\mathcal{C}_{k'}(y)$ with $y \neq x$ can be formed, completing the induction. □

Validity and liveness. If we make the simplifying assumption of synchronized clocks, the proofs of validity and liveness are very similar to those of Paxos. We instead give the proofs under a more realistic model without synchronized clocks. Before doing so, we clarify a few assumptions and conventions.

We assume that after GST, every honest party can correctly measure a time interval of at least 4Δ. This is to ensure that honest parties do not abandon an honest leader's view prematurely after GST. This assumption is easy to satisfy in practice. If an honest party is concerned that its clock may run too fast, it can simply wait longer, for example, 5Δ. Using a longer timeout only affects the algorithm's efficiency, not correctness.

We also assume that if a party is in view k but receives a message from view $k + 1$, it buffers the message and processes it after entering view $k + 1$. This convention is necessary because parties may enter a view at different times. Messages sent by early entrants must be processed by parties who enter the view later. (Messages from even higher views can be either buffered or discarded.)

A similar mechanism is needed at the beginning of the algorithm to ensure that the sender's proposal in view 1 is seen by parties who start the algorithm late. As discussed

in Chapter 4.3.1, we assume that a message delivered to a party who has not started the algorithm is buffered for that party. Alternatively, we can ask all parties to wait Δ time after starting before executing the pseudocode in Algorithm 11.

Theorem 5.3.4 Algorithm 11 achieves validity (as in Definition 5.1.4) and liveness.

Proof. Validity. The honest sender proposes its input x upon starting Algorithm 11 (which we define as time 0). As GST $= 0$, every honest party receives the sender's proposal for x and sends Vote1 for x by time Δ; Every honest party receives $\mathcal{C}_1(x)$ and sends Vote2 for x by time 2Δ; Every honest party receives enough Vote2 messages and outputs x by time 3Δ. All of these happen before any honest party spends 4Δ time in view 1, because honest parties start the algorithm at most Δ earlier than the sender.

Liveness. It suffices to prove that one honest party outputs. Once an honest party outputs, it sends $n - f$ Vote2 messages. And all honest parties eventually receive them and output. Suppose toward a contradiction that no honest party ever outputs.

Because parties take turns as leaders and GST eventually comes, the algorithm eventually reaches a view k that starts after GST and has an honest leader. Let $t \geq$ GST be the first time an honest party enters view k. That honest party sends $\langle \text{ViewChange}, k-1 \rangle_Q$ where $|Q| = f + 1$ at time t. The view-change certificate makes all honest parties enter view k and send Status to L_k by time $t + \Delta$. L_k receives these and sends a valid proposal for a value x by $t + 2\Delta$. All honest parties receive the proposal and send Vote1 for x by $t + 3\Delta$. All honest parties receive $\mathcal{C}_k(x)$ and send Vote2 for x by $t + 4\Delta$. Since honest parties enter view k at time t or later, all of these happen before any honest party leaves view k. All honest parties receive enough Vote2 messages for x from view k and output x. This contradicts the assumption that no honest party ever outputs. □

Efficiency analysis After GST, Algorithm 11 terminates once it reaches a view with an honest leader. Among the first $f + 1$ views after GST, one must have an honest leader. Thus, the algorithm uses $O(f)$ rounds after GST. Each view uses $O(n^2)$ messages, since both rounds of votes and view-change requests/certificates use all-to-all communication. Thus, the message complexity (after GST) is $O(n^2 f)$.

The bit complexity of PBFT is higher. Each QC has size $O(\kappa n)$, where κ is the size of a signature. Each Status message carries a QC. Each leader's proposal (except the first one from the sender) carries $n - f$ Status messages and has size $O(\kappa n^2)$. Thus, each view consumes $O(\kappa n^3)$ bits of communication. Since $O(f)$ views may be needed after GST, the algorithm's total bit complexity is $O(\kappa n^3 f)$, or $O(\kappa n^4)$ for $f = \Theta(n)$.

In the good case with an honest sender and GST $= 0$, Algorithm 11 terminates in the first view. Thus, its good-case round complexity is $O(1)$. More precisely, parties output after three rounds of communication and terminate after four. The good-case output latency in terms of time is again 3δ where δ denotes the actual message delay, not constrained by the conservative delay bound Δ.

The good-case message complexity is $O(n^2)$ due to all-to-all voting. The sender's proposal in view 1 need not attach Status messages. The view-change logic also does not trigger in the good case. If we also exclude the communication used for termination (forwarding $n - f$ Vote2 messages), the good-case bit complexity is $O(\kappa n^2)$.

5.3.3 Variations

The Status and ViewChange messages can be merged into a single message. We keep them separate to highlight their respective roles.

Algorithm 11 can be adapted to solve the agreement problem. The key change is that the first leader L_1 can no longer propose freely. Instead, L_1 must collect inputs from other parties and use their signed inputs to justify its proposal. Among the $n - f$ collected inputs, if at least $f + 1$ of them are identical, L_1 must propose that value; otherwise, L_1 can propose an arbitrary value or $\perp$. In a subsequent view, if every message in $\mathcal{S}$ reports highestQC $= \emptyset$, the leader similarly needs to collect inputs and propose according to the above rule. It is not difficult to show that this modification ensures the strong unanimity validity of agreement. The safety and liveness proofs are unaffected.

PBFT inspired significant follow-up works. There are too many variations to cover. I may introduce Tendermint and Simplex in the next edition.

5.4 Good-case Round Complexity in Partial Synchrony

Chapter 3.2 presented synchronous broadcast algorithms with optimal good-case round complexity. We now explore the optimal good-case round complexity of broadcast in partial synchrony. Recall that the good case requires an honest sender and GST $= 0$, and the metric is the number of rounds until honest parties output (rather than terminate).

With crash faults, the Paxos algorithm with all-to-all voting guarantees that correct parties output after two rounds in the good case. This is optimal, as outputting in a single round is impossible for any partially synchronous broadcast algorithm that tolerates at least two crashes.

Theorem 5.4.1 In any partially synchronous broadcast algorithm tolerating at least two crash faults, correct parties cannot output in one round, even in the good case.

Proof. Suppose such an algorithm exists. Consider the scenario that the sender s has input x and sends its first-round message to a correct party p, but then s crashes and p experiences a long period of asynchrony. From p's perspective, s appears to be correct, so p must output x immediately by validity and one-round good-case latency. From other parties' perspectives, it appears that s and p have both crashed. They must decide a value without ever hearing from s and p. With at least constant probability, the value they decide is not x. □

With Byzantine faults, PBFT guarantees that honest parties output after three rounds in the good case. This turns out to be optimal unless one sacrifices Byzantine fault tolerance. The following negative result is independently proved by Abraham, Nayak, Ren, and Xiang and by Kuznetsov, Tonkikh, and Zhang. The proof is rather involved and is omitted. We will provide some intuition later.

Theorem 5.4.2 In any partially synchronous broadcast algorithm tolerating $f \geq \frac{n+2}{5}$ Byzantine faults, honest parties cannot output in two rounds, even in the good case.

If one is content with about one-fifth Byzantine fault tolerance, then two-round good-case output is achievable. We present a partially synchronous Byzantine broadcast al-

gorithm for $n \geq 5f + 1$. We remark that this threshold is not optimal, as $n \geq 5f - 1$ (equivalently, $f < \frac{n+2}{5}$) suffices. We present the $n \geq 5f + 1$ algorithm because it is considerably simpler and represents the approach taken by most two-round BFT algorithms.

The top-level algorithm is identical to Algorithm 6, which combines graded broadcast with Byzantine agreement. Observe that the correctness proof of Algorithm 6 makes no assumption about the timing model or the fault threshold. For the Byzantine agreement subroutine, we can use the agreement variant of PBFT. It remains to construct a partially synchronous graded broadcast algorithm with $n \geq 5f + 1$ that produces grade-1 outputs in two rounds in the good case. Algorithm 12 provides such an algorithm.

Algorithm 12: Byzantine graded broadcast with $n \geq 5f + 1$ and 2-round good-case round complexity in partial synchrony.

upon starting the algorithm
The sender s signs its input x and sends $\langle x \rangle_s$ to all parties

upon receiving $\langle x \rangle_s$ from the sender within Δ time `// triggers at most once`
Forward $\langle x \rangle_s$ to all parties

upon Δ time has elapsed and no message is received from the sender
Send $\perp$ to all parties

upon receiving messages from $n - f$ parties
If all of them are $\langle x \rangle_s$, output $(x, 1)$;
Else if at least $n - 3f$ of them are $\langle x \rangle_s$, output $(x, 0)$;
Else, output $(\perp, 0)$.

We prove that Algorithm 12 solves graded broadcast against $f < n/5$ Byzantine faults under partial synchrony. In partial synchrony, the validity property of graded broadcast is only required when GST $= 0$ and the sender is honest.

Proof. If the sender s is honest and GST $= 0$, it sends its signed input $\langle x \rangle_s$ to all parties. All $n - f$ honest parties receive $\langle x \rangle_s$ by time Δ and forward it. All honest parties receive $n - f$ forwarded $\langle x \rangle_s$ messages and output $(x, 1)$ at the end of round 2.

If an honest party p outputs $(x, 1)$, then $n - f$ parties forward $\langle x \rangle_s$. Among these, at most f are Byzantine and at least $n - 2f$ are honest. By quorum intersection, another honest party receives $\langle x \rangle_s$ from at least $(n - 2f)+(n - f)-n = n-3f$ parties. Meanwhile, it receives $\langle x' \rangle_s$ for any $x' \neq x$ from at most $2f$ parties: the f Byzantine parties and the f honest parties who did not forward $\langle x \rangle_s$. Since $2f < n - 3f$, every honest party outputs $(x, 0)$ or $(x, 1)$. □

The above proof of Algorithm 12 provides intuition for Theorem 5.4.2. If n is smaller, say $n = 5f$, it is possible that p outputs x based on $n - f$ votes for x, but another honest party q receives an equal amount of "evidence" for x and x', i.e., $2f$ votes for x' and $n - 3f = 2f$ votes for x. Intuitively, in this situation, there is no way for q to make the right decision and agree with p. (The actual critical threshold is $n \geq 5f - 1$ rather than $n \geq 5f + 1$ because, in the situation above, honest parties can detect that the sender is malicious, and this observation can slightly improve fault tolerance.)

As in the synchronous setting, outputting in a single round is impossible even in the best case (Theorem 3.2.2). Ignoring the small difference between $n \geq 5f - 1$ and $n \geq$

$5f + 1$, we have a complete characterization of the optimal good-case round complexity of partially synchronous broadcast, summarized below.

Fault tolerance	**Good-case round**	**Upper bound**	**Lower bound**
$< 50\%$ crash	2	All-to-all Paxos	Theorem 5.4.1
20%–33% Byzantine	3	PBFT	Theorem 5.4.2
$< 20\%$ Byzantine	2	Algorithms 6 + 12	Theorem 3.2.2

Table 5.1. Optimal good-case round complexity of partially synchronous broadcast. Partial synchronous broadcast is unsolvable in the presence of $f \geq n/2$ crash faults or $f \geq n/3$ Byzantine faults (see Chapter 6.2).

CHAPTER 6
Fault Tolerance Thresholds

This chapter provides a systematic review of fault tolerance thresholds for consensus, namely, the maximum number of faults that can be tolerated in various settings.

We focus on the broadcast and agreement problems, but many (though not all) results in this chapter apply to other variants of the consensus problem, such as graded broadcast, graded agreement, reliable broadcast, and long-running consensus variants that make multiple decisions.

Unless otherwise stated, the impossibility results in this chapter apply to both deterministic and randomized algorithms. Although the theorem statements and proofs we present do not explicitly account for randomization, extending them to randomized algorithms is generally straightforward. Readers can refer to the example in Chapter 4.1.

6.1 Fault Tolerance Thresholds in Synchrony

We begin by summarizing the fault tolerance thresholds for synchronous consensus established in previous chapters.

Theorem 6.1.1 The broadcast problem and the agreement problem can be solved in synchrony in the presence of f crash faults for all $f < n$.

Proof sketch. Algorithm 1 is a broadcast algorithm that tolerates any number of crash faults. Adapting it to solve agreement is straightforward and left as an exercise. □

Theorem 6.1.2 The broadcast problem can be solved in synchrony in the presence of f Byzantine faults for all $f < n$.

Proof. Follows from the Dolev-Strong algorithm (Algorithm 2). □

Theorem 6.1.3 The agreement problem can be solved in synchrony in the presence of f Byzantine faults if and only if $f < n/2$.

Proof. Follows from Algorithm 3 and Theorem 2.3.1. □

This is one of the few settings in which broadcast and agreement have different fault tolerance thresholds. The next subsection shows that this distinction is often inconsequential in practice once clients (or users) are taken into account.

6.1.1 Supporting Clients Requires Byzantine Minority

Thus far, we have considered only one type of party: one that executes the consensus algorithm and outputs the consensus result. Recall that the state machine replication paradigm introduced in Chapter 1.1 involves two types of parties: (i) servers that execute a consensus algorithm to maintain a service, and (ii) users (also called clients) that use the service. Typically, clients neither know nor care about the internal workings of the consensus algorithm, and they may not even be present when the servers execute the algorithm. But it is still desirable for a consensus algorithm to provide a mechanism that allows such clients to obtain the consensus result correctly.

With only crash faults, supporting clients is straightforward. The client can ping any server to request the consensus result. If the server does not reply (it may have crashed), the client pings the next server and repeats until some server replies. Any server's reply is the genuine consensus result.

With Byzantine faults, the above simple approach does not work because a Byzantine server can lie about the consensus result. Nevertheless, supporting clients is still easy if $f < n/2$. The client can contact $2f+1$ servers. Once it receives the same result from $f+1$ servers, it can safely accept that result, since at least one of those servers must be honest. PBFT represents another common mechanism for supporting clients. In Algorithm 11, $n - f$ signed Vote2 messages from the same view for the same value can be sent to a client as a proof for the correct consensus result.

In fact, in the case of Byzantine faults, supporting clients requires $n \geq 2f + 1$ (equivalently, $f < n/2$). This is why tolerance to $f \geq n/2$ Byzantine faults offers limited practical value for practical state machine replication systems.

Theorem 6.1.4 No algorithm solving any variant of the consensus problem can support clients in the presence of $f \geq n/2$ Byzantine servers.

Proof sketch. Suppose $n \leq 2f$. The f Byzantine servers may simply ignore the honest servers and instead simulate a fictitious execution of the algorithm, including any public verification mechanism. From the client's perspective, this simulated execution is indistinguishable from a legitimate execution in which the honest servers refuse to participate. If a client does not observe communication among the servers or was not present while the algorithm is running, it has no way to distinguish the genuine execution carried out by the honest servers from the simulated execution produced by the Byzantine servers. □

6.1.2 Byzantine Fault Tolerance Thresholds without Signatures

Pease, Shostak, and Lamport (1980) proved that, in the absence of digital signatures, Byzantine agreement or broadcast can be solved if and only if $f < n/3$. This result is historically significant because, as mentioned in Chapter 1.3.4, at the dawn of consensus research, digital signatures were costly and not widely accepted. Fischer, Lynch, and Merritt (1986) gave a beautiful and accessible proof that involves a hexagon. Interested readers are encouraged to check out their proof. Today, digital signatures are widely used, reducing the practical importance of this result.

6.2 Fault Tolerance Thresholds in Partial Synchrony

We saw in Chapter 5 that broadcast in partial synchrony (Definition 5.1.4) tolerates up to $f < n/2$ crash faults (Paxos) or up to $f < n/3$ Byzantine faults (PBFT). These turn out to be the optimal fault tolerance thresholds in partial synchrony for most consensus problems.

Theorem 6.2.1 No algorithm can solve broadcast, agreement, or graded agreement in partial synchrony if $f \geq n/2$ parties may crash.

Proof. We prove for broadcast. The proof for (graded) agreement is very similar. Suppose toward a contradiction that there is an algorithm that solves broadcast in partial synchrony for n total parties and up to $f \geq n/2$ crashes. Partition the parties into two groups P and Q such that the sender belongs to P, $|P| \leq f$, and $|Q| \leq f$ (this is possible because $2f \geq n$). Consider the following three scenarios.

Scenario 1: all parties in P crash at the beginning, and all messages are delivered instantly. Because the algorithm tolerates f crashes, parties in Q must eventually output some value. Without loss of generality, suppose they output 0 by time t_1.

Scenario 2: all parties in Q crash at the beginning, the sender has input 1, and all messages are delivered instantly. Because the algorithm tolerates f crashes and the sender is correct, parties in P must eventually output 1. Suppose they do so by time t_2.

Scenario 3: all parties are correct, all messages sent between P and Q are delayed until after GST, while all messages within each group are delivered instantly. The adversary sets GST $\geq \max(t_1, t_2)$. Parties in P cannot distinguish Scenario 3 from Scenario 2 until time t_2, so they output 1. Similarly, parties in Q cannot distinguish Scenario 3 from Scenario 1 until time t_1, so they output 0. Safety is violated. We have arrived at a contradiction. □

Theorem 6.2.2 The broadcast and agreement problems can be solved in partial synchrony in the presence of f crash faults if and only if $f < n/2$.

Proof. Algorithm 10 (Paxos) solves broadcast in partial synchrony and tolerates $f < n/2$ crash faults. It can be adapted to solve agreement with the same fault tolerance. The only modification is that, if all $n - f$ Status messages received by a leader report having never voted, the leader proposes its own input rather than $\perp$. The safety and liveness proofs remain unchanged. For validity (strong unanimity), observe that if all parties input the same value, then only that value can ever be proposed.

Combined with the negative result of Theorem 6.2.1, this completes the proof. □

Theorem 6.2.3 No algorithm can solve broadcast, reliable broadcast, graded agreement, or agreement in partial synchrony if $f \geq n/3$ parties may be Byzantine.

Proof. We prove for agreement. The proofs for the other problems are very similar. Suppose toward a contradiction that there is an algorithm that solves agreement in partial synchrony for n total parties and up to $f \geq n/3$ Byzantine faults. Partition the parties into three groups P, Q, and R such that $|P| \leq f$, $|Q| \leq f$, $|R| \leq f$ (this is possible because $3f \geq n$). Consider the following three scenarios.

Scenario 1: all parties in P crash at the beginning, all parties in $Q \cup R$ are honest and input 0, and all messages are delivered instantly. Because the algorithm tolerates f

faults, parties in $Q \cup R$ must eventually output. Suppose they do so by time t_1. Because all honest parties input 0, the output is 0 by validity (strong unanimity).

Scenario 2: all parties in Q crash at the beginning, all parties in $P \cup R$ are honest and input 1, and all messages are delivered instantly. By the same argument, parties in $P \cup R$ must eventually output 1 by some time t_2.

Scenario 3: parties in P are honest and input 1, parties in Q are honest and input 0, parties in R are Byzantine, all messages between P and Q are delayed until after GST, and all messages within each group are delivered instantly. The Byzantine parties in R behave towards parties in P as in Scenario 2, and behave towards parties in Q as in Scenario 1. The adversary sets GST $\geq \max(t_1, t_2)$. Parties in P cannot distinguish Scenario 3 from Scenario 2 until time t_2, so they output 1. Similarly, parties in Q cannot distinguish Scenario 3 from Scenario 1 until time t_1, so they output 0. Safety is violated. We have arrived at a contradiction. □

Theorem 6.2.4 The broadcast and agreement problems can be solved in partial synchrony in the presence of f Byzantine faults if and only if $f < n/3$.

Proof. Algorithm 11 (PBFT) solves broadcast in partial synchrony and tolerates $f < n/3$ Byzantine faults. It can be adapted to solve agreement with the same fault tolerance, as discussed in Chapter 5.3.3. Combined with the negative results of Theorem 6.2.3, this completes the proof. □

6.3 Fault Tolerance Thresholds in Asynchrony

We have already covered two negative results in asynchrony. Theorem 4.1.1 states that no broadcast algorithm can tolerate even a single crash fault in asynchrony. Theorem 4.2.1, better known as the FLP impossibility, states that no *deterministic* agreement algorithm can tolerate even a single crash fault in asynchrony. Accordingly, this section focuses on reliable broadcast, graded agreement, and randomized agreement in asynchrony. Note that the negative results for partial synchrony apply to asynchrony.

Theorem 6.3.1 The reliable broadcast problem can be solved in asynchrony in the presence of f crash faults for all $f < n$.

Proof. Left as an exercise. □

Theorem 6.3.2 The reliable broadcast problem can be solved in asynchrony in the presence of f Byzantine faults if and only if $f < n/3$.

Proof. Follows from Algorithm 7 and Theorem 6.2.3. □

Theorem 6.3.3 The graded agreement problem and the agreement problem can be solved in asynchrony in the presence of f crash faults if and only if $f < n/2$.

Proof. Follows from Algorithm 8, Algorithm 9, and Theorem 6.2.1. □

Theorem 6.3.4 The graded agreement problem and the agreement problem can be solved in asynchrony in the presence of f Byzantine faults if and only if $f < n/3$.

Proof sketch. There exist asynchronous graded agreement algorithms that tolerate $f < n/3$ Byzantine faults. We did not cover them because they are somewhat sophisticated. One example is given in Bracha's 1987 paper. Plugging such a graded agreement algorithm into Algorithm 9 yields a randomized asynchronous agreement algorithm that tolerates $f < n/3$ Byzantine faults. Combined with the negative result of Theorem 6.2.3, this completes the proof. □

6.4 Summary of Fault Tolerance Thresholds

The fault tolerance threshold of consensus is highly sensitive to the precise problem definition and the exact model. Throughout this and the preceding chapters, we have seen that any aspect of the model, such as fault type, network timing, availability of signatures, use of randomization, and support for clients, can affect the optimal fault tolerance threshold. As another example, agreement is solvable against any number of Byzantine faults ($f < n$) if the validity property is weakened to *weak unanimity*, which says that if all parties have the same input x *and no party is faulty*, then all parties output x.

While the fault tolerance threshold is sensitive to all these factors, it is useful to focus on (what I consider) the *mainstream* settings:

- digital signatures are available;
- randomization is allowed;
- clients are supported; and
- the problem under consideration is broadcast (when solvable), agreement with strong unanimity, state machine replication (Chapter 8), or another consensus problem of comparable difficulty.

In this mainstream setting, the optimal fault tolerance thresholds for consensus can be summarized succinctly as follows.

	Synchrony	**Partial Synchrony or Asynchrony**
Crash Faults	$f < n$	$f < n/2$
Byzantine Faults	$f < n/2$	$f < n/3$

Table 6.1. Optimal fault tolerance threshold under mainstream models.

We reiterate that this succinct summary applies only to the *mainstream* setting. It may not hold if digital signatures or randomization are unavailable, if there are no clients, or if easier variants of the consensus problem are considered.

CHAPTER 7

Communication Bounds of Byzantine Consensus

This chapter studies the communication complexity of Byzantine consensus. We begin with lower bounds, showing that deterministic Byzantine consensus algorithms must incur quadratic communication. Several algorithms from previous chapters already achieve quadratic *message* complexity, but none achieves quadratic *bit* complexity. We then introduce Bracha's Byzantine reliable broadcast, which achieves quadratic bit complexity. Next, we discuss several approaches to circumventing the quadratic lower bounds. Finally, we study the communication complexity of Byzantine consensus on long messages.

Remark The communication complexity of crash-tolerant consensus is now well understood. For broadcast, $\Omega(n)$ communication is necessary even in the best case because every non-sender party must learn the sender's input. A simple and elegant algorithm due to Lewis-Pye matches this lower bound. Both the lower bound and the algorithm extend to the agreement problem.

7.1 Communication Lower Bounds

We present two lower bounds on the communication complexity of *deterministic* Byzantine consensus algorithms. Both are due to Dolev and Reischuk. Although originally proved for broadcast, they can be easily extended to agreement and reliable broadcast. Both bounds were proven for binary inputs in lockstep synchrony, so they apply to multi-bit inputs and harsher networks.

The first lower bound allows digital signatures, but not the more powerful threshold signatures (see Chapter 1.3.4 for the distinction). A particularly striking feature of this bound is that it applies even in the best case when all parties are honest.

Theorem 7.1.1 Any deterministic algorithm solving reliable broadcast, broadcast, or agreement among n parties and tolerating f Byzantine faults must incur more than $\frac{fn}{4}$ bit complexity, even when all parties are honest.

Proof. We present the proof for reliable broadcast. The proof applies verbatim to broadcast and extends to agreement with minor modifications. Suppose toward a contradiction that there exists such a reliable broadcast algorithm whose best-case bit complexity is at most $\frac{fn}{4}$.

Let E_0 be the execution in which all parties are honest and the sender has input 0.

By validity, all parties output 0 in E_0. For each party p, define $A_0(p)$ to be the set of parties such that $q \in A_0(p)$ if at least one of the following is true:

- q directly sends a message to p;
- q directly receives a message from p;
- p receives a signature of q (possibly forwarded); or
- q receives a signature of p (possibly forwarded).

In other words, $A_0(p)$ is the set of parties that exchange *authenticated* information with p in E_0, where authenticated information means either a direct message or a signature (possibly forwarded). Let E_1 be the execution in which all parties are honest and the sender has input 1. By validity, all parties output 1 in E_1. Similarly, define $A_1(p)$ to be the set of parties that exchange authenticated information with p in E_1.

In E_0, if a direct message from p to q does not contain a signature, q is added to $A_0(p)$ and p is added to $A_0(q)$. Furthermore, if that message contains a signature from party r, then r is added to $A_0(q)$ and q is added to $A_0(r)$. Thus, every transmitted bit or signature in E_0 contributes at most 2 to $\sum_p |A_0(p)|$. Since E_0 incurs at most $\frac{fn}{4}$ bit of communication, we have $\sum_p |A_0(p)| \leq \frac{fn}{2}$. Following the same argument, $\sum_p |A_1(p)| \leq \frac{fn}{2}$. Thus, we have

$$\sum_p |A_0(p) \cup A_1(p)| \leq fn.$$

Since the above sum over n parties is fn, there must exist a party p such that $|A_0(p) \cup A_1(p)| \leq f$. Now consider a third execution E' in which all parties in $A_0(p) \cup A_1(p)$ are Byzantine. The Byzantine parties simulate execution E_0 when interacting with p and simulate execution E_1 when interacting with the other honest parties. Party p cannot distinguish E' from E_0, because p only exchanges authenticated information with $A_0(p)$, who are now Byzantine and simulate E_0. Similarly, the other honest parties cannot distinguish E' from E_1. Thus, p outputs 0 and the other honest parties output 1 in E'. This violates safety, a contradiction. □

Remark 7.1.2 If an algorithm can use threshold signatures, Theorem 7.1.1 no longer applies. This is because a threshold signature, while compact in size, carries authenticated information from many parties.

The second lower bound applies to all deterministic algorithms, including those that use threshold signatures, but it applies only to the *worst-case* message complexity. Since each message contains at least one bit, it immediately implies the same lower bound on worst-case bit complexity.

Theorem 7.1.3 In any deterministic algorithm solving reliable broadcast, broadcast, or agreement and tolerating f Byzantine faults (among n parties), honest parties must collectively send at least $f^2/4$ messages in the worst case.

The proof of Theorem 7.1.3 is somewhat involved. We omit the full proof and provide some intuition. If honest parties collectively send fewer than $f^2/4$ messages, then with an appropriately chosen set of corrupted parties, one can show that there must exist a party p who receives at most $f/2$ messages. In another carefully constructed execution,

the adversary corrupts all those $f/2$ parties (leaving the remaining corruption budget for other steps in the proof), and p receives no message at all. Without hearing from the other correct parties, p cannot reach an agreement with them.

Remark 7.1.4 Theorem 7.1.3 holds even for *omission* faults. An omission-faulty party may deviate from the protocol only by ignoring selected incoming messages and dropping selected outgoing messages.

When f is linear in n, as is typically the case, both theorems yield $\Omega(n^2)$ lower bounds on communication complexity.

7.2 Reliable Broadcast with Quadratic Communication

Recall the reliable broadcast problem introduced in Chapter 4.3.1. Bracha's reliable broadcast algorithm (Algorithm 13) tolerates up to $f < n/3$ Byzantine faults in asynchrony, which is optimal. It achieves $O(n^2)$ message complexity and $O(n^2)$ bit complexity, both of which are also optimal.

Most of the algorithm is quite natural. The sender proposes a value, and parties vote in two rounds. $n - f$ round-1 votes trigger round-2 votes, and $n - f$ round-2 votes lead to outputs. The key innovation of Bracha's algorithm is an additional rule: a party also sends round-2 votes upon receiving $f + 1$ round-2 votes. This is called the *amplification* step (of round-2 votes).

To understand the purpose of the amplification step, let us first consider what goes wrong without it. Suppose H is a subset of honest parties such that $n-2f \leq |H| < n-f$. A Byzantine sender sends its input only to H. These $|H| < n - f$ parties send round-1 votes. The f Byzantine parties then send round-1 votes only to H, allowing the parties in H to collect enough round-1 votes and send their round-2 votes. The f Byzantine parties then send round-2 votes only to H, making the parties in H output. But honest parties outside H never receive enough votes and cannot output. This violates the liveness property of reliable broadcast (Definition 4.3.1). As we will see next, the amplification step defeats this kind of attack.

Correctness proofs. Validity is straightforward. Safety follows from a standard quorum intersection argument. We focus on liveness.

Proof. Liveness. Suppose one honest party outputs x. Applying a standard quorum intersection argument on Vote1 messages, we have that no honest party sends $(\text{Vote2}, x')$ for any $x' \neq x$. The honest party who outputs x has received $(\text{Vote2}, x)$ from $n-f$ parties. At least $n - 2f \geq f + 1$ of those parties are honest and send $(\text{Vote2}, x)$ to all parties. With the amplification step, all honest parties eventually send $(\text{Vote2}, x)$. Therefore, all honest parties eventually receive $n - f$ copies of $(\text{Vote2}, x)$ and output. □

Efficiency analysis. Algorithm 13 uses three asynchronous rounds in the good case. A Byzantine sender can force the algorithm to use four asynchronous rounds. (Of course, it can also make the algorithm hang, which is permitted for reliable broadcast.)

Algorithm 13: Bracha's Byzantine reliable broadcast algorithm.

upon starting the algorithm
    The sender sends its input to all parties (including itself)

upon receiving x from the sender `// only for 1st message from sender`
    Send $(\text{Vote1}, x)$ to all parties

upon receiving $(\text{Vote1}, x)$ from $n - f$ parties
    Send $(\text{Vote2}, x)$ to all parties `// if it has not already sent` Vote2

upon receiving $(\text{Vote2}, x)$ from $f + 1$ parties `// the amplification step`
    Send $(\text{Vote2}, x)$ to all parties `// if it has not already sent` Vote2`,`
  `even if it previously sent` $(\text{Vote1}, x')$ `for some` $x' \neq x$

upon receiving $(\text{Vote2}, x)$ from $n - f$ parties
    Output x and terminate

Besides the sender's initial n messages, every party sends at most one Vote1 message and one Vote2 message to every other party. Assuming the input has constant size, both vote messages have size $O(1)$. Therefore, the message complexity and bit complexity are both $O(n^2)$, which are both optimal given Theorem 7.1.3.

7.3 Circumventing Quadratic Communication

7.3.1 Best-case Linear Communication with Threshold Signatures

Recall that Theorem 7.1.1 applies even in the best case when all parties are honest, but this lower bound can be circumvented using more powerful cryptographic primitives, such as threshold signatures.

As discussed in Chapter 1.3.4, a threshold signature has the same size as an individual signature, but certifies that a threshold number of parties signed the message. With the help of threshold signatures, leader-assisted voting can reduce communication compared to all-to-all voting.

We illustrate the idea using the basic reliable broadcast algorithm in Algorithm 7. That algorithm has two sources of quadratic (or more) communication: (i) every party sends a signed vote for the sender's proposed value to all other parties, and (ii) every party forwards a collection of signatures to help other parties output. To achieve linear communication in the best case, both sources of quadratic communication need to be avoided. For (ii), we let a party help only upon request rather than proactively. For (i), we replace all-to-all voting with leader-assisted voting using threshold signatures.

The resulting algorithm is shown in Algorithm 14. To elaborate, we use a threshold signature scheme with a threshold of $n - f$. Upon receiving the sender's input x, each party sends its signature on x only to the sender. The sender waits for $n - f$ signatures on its input, aggregates them into a threshold signature, and sends the threshold signature to all parties. Receiving the threshold signature has the same effect as receiving $n - f$ individual signatures from distinct parties.

Algorithm 14: A Byzantine reliable broadcast algorithm with linear communication cost in the best case.

upon starting the algorithm
The sender sends its input to all parties (including itself)

upon receiving x from the sender `// only for 1st message from sender`
Party i sends $\langle x \rangle_i$ to the sender

upon the sender receiving signatures on x from $n - f$ parties
The sender aggregates them into a threshold signature $\mathcal{C}(x)$
The sender sends $\mathcal{C}(x)$ to all parties

upon receiving threshold signature $\mathcal{C}(x)$
Party i outputs x (but does not terminate)
If "need help" has been received from party j, party i sends $\mathcal{C}(x)$ to party j

upon 4Δ time after starting the algorithm
If party i still has not output, it sends "need help" to all parties

Remark 7.3.1 Despite using a timeout, Algorithm 14 works in asynchrony. The timeout is only used to improve communication efficiency in the best case.

Unlike Algorithm 7, however, Algorithm 14 does not guarantee that honest parties terminate, even when all parties are honest. It guarantees only that they output.

Correctness proofs. We prove that Algorithm 14 solves reliable broadcast against up to $f < n/3$ Byzantine faults in asynchrony.

Proof. Safety follows from a standard quorum intersection argument, exactly as in the proof of Algorithm 7.

Validity. Suppose the sender is honest and has input x. It sends x to all parties. All $n - f$ honest parties send their signatures on x, so the sender can create a threshold signature $\mathcal{C}(x)$. The sender sends $\mathcal{C}(x)$ to all parties, so all honest parties output x.

Liveness. Suppose an honest party p outputs x. Every other honest party either (i) outputs before timing out, or (ii) times out, requests help from other parties, receives $\mathcal{C}(x)$ from party p, and outputs. □

Efficiency analysis. The all-to-all voting (signature exchange) round in Algorithm 7 is replaced by two rounds of linear communication in Algorithm 14. This incurs an extra round but reduces the communication cost of the voting step from quadratic to linear.

In the best case, all parties are honest, they start the algorithm within Δ time of each other, and the network is synchronous. In this case, all parties output before the 4Δ timeout, and crucially, no party requests help or forwards $\mathcal{C}(x)$. The message complexity and bit complexity in this best case are both $O(\kappa n)$, where κ is the size of a (threshold) signature.

More generally, if $f' \leq f$ parties other than the sender are Byzantine, they can request help from all honest parties. The message complexity and bit complexity in this good

case are both $O(\kappa f' n)$. Thus, the communication cost in the good case scales linearly with the number of *actual* faults, rather than the fault threshold.

Observe the critical role of the threshold signature. Without it, the sender would have to send $n - f$ individual signatures to every party, resulting in quadratic bit complexity.

Applications in PBFT-style algorithms. PBFT (Algorithm 11) and its follow-up works can adopt the leader-assisted voting approach. PBFT has two rounds of all-to-all voting. This approach increases PBFT's good-case round complexity from 3 to 5, but reduces the best-case bit complexity from $O(n^2)$ to $O(n)$.

Besides the extra rounds, this approach has another drawback: it creates a potential performance bottleneck at the leader (sender). The leader sends and receives $n - f$ messages, whereas every other party sends and receives a single message. Moreover, the leader is responsible for aggregating individual signatures into a threshold signature, a step that can be computationally expensive.

7.3.2 Sampling a Random Committee

The two quadratic communication lower bounds apply only to deterministic algorithms, so another natural way to circumvent them is to use randomization.

A simple idea is to sample a random committee to run a (typically deterministic) consensus algorithm and notify the remaining parties of the consensus result. If a consensus algorithm incurs $O(n^2)$ communication cost with n parties, then using a committee of size k reduces the communication cost to $O(k^2 + kn)$. The first term is the cost of running consensus within the committee, and the second term is the cost of notifying other parties of the result.

The above idea is mostly straightforward, so we omit the pseudocode and correctness proof. We make a few remarks on the subtleties and limitations of the committee-based approach.

- Most distributed coin-tossing algorithms require quadratic communication and thus do not help circumvent the lower bounds. One needs to rely on more advanced techniques or an external source of shared randomness.
- The committee-based approach usually sacrifices fault tolerance. For example, suppose the underlying consensus algorithm tolerates up to one-third Byzantine faults. If $n = 3f + 1$, then there is a good chance that a randomly sampled committee has more than one-third Byzantine members. Thus, f/n must be noticeably smaller than $1/3$ (e.g., $1/4$ is a common choice), and the committee size k typically needs to be at least a few hundred so that Byzantine parties do not take over the committee by luck.
- The committee-based approach does not diminish the importance of deterministic algorithms. In many practical scenarios, n is a few hundred or less, and a committee does not help. Even when n is large enough to use a committee, we still need a consensus algorithm (often deterministic) to be run by the committee.
- In its basic form, the committee-based approach must assume *static corruption*. An adversary capable of *adaptive corruption* (see Chapter 1.3.5) can simply wait until the committee is selected and then corrupt all of its members. The Algorand protocol addresses this problem using verifiable random functions.

7.4 Byzantine Consensus for Long Messages

So far in this book, we have either ignored the size of the input or implicitly assumed that it is constant. For instance, Bracha's reliable broadcast, presented earlier in this chapter, achieves the optimal $O(n^2)$ bit complexity only when the input has $O(1)$ size. If the input is ℓ-bit long, the bit complexity becomes $O(n^2\ell)$ since every message carries the input. When ℓ is large, this can be substantially higher than $O(n^2)$.

In this section, we show that it is possible to achieve $O(n\ell)$ bit complexity when the input length ℓ is sufficiently large. Such an algorithm is also known as an *extension protocol*, reflecting that it typically extends a consensus protocol for short messages to handle long messages. The earliest extension protocols were proposed by Fitzi and Hirt (2006) and Cachin and Tessaro (2005).

Before presenting Byzantine consensus extension protocols, we introduce two standard tools: cryptographic hash functions and error-correcting codes.

7.4.1 Cryptographic Hash Functions

A *cryptographic hash function* is a deterministic function that maps an input of arbitrary finite length to an output of a fixed length. The output is called a *hash value* or a *digest*. Formally,

$$H : \{0,1\}^* \to \{0,1\}^\kappa,$$

where $\{0,1\}^*$ denotes the set of all finite binary strings, and $\{0,1\}^\kappa$ denotes the set of κ-bit strings. A typical value of κ is 256.

A cryptographic hash function is believed to satisfy the following properties:

- *Collision resistance.* It is infeasible to find two distinct inputs $x_1 \neq x_2$ such that $H(x_1) = H(x_2)$.
- *Preimage resistance.* Given $y \in \{0,1\}^\kappa$, it is infeasible to find x such that $H(x) = y$.
- *Pseudorandomness.* For any previously unseen input x, $H(x)$ appears to be a random κ-bit binary string.

For our purpose in this section, we only care about the collision-resistance property. Later chapters need a pseudorandom cryptographic hash function. Preimage resistance is less relevant to consensus (but essential in many other applications).

7.4.2 Error Correcting Codes and Data Dissemination

An *error-correcting code* is a method to encode messages such that the original message can be recovered even if parts of the encoded message are corrupted by errors. Formally, an error-correcting code is a pair of functions Encode and Decode, parameterized by three parameters n, k, and e.

- $c = \mathsf{Encode}(m)$ takes as input a message m, splits it into k chunks, and outputs an encoded message c of n chunks.
- $m' = \mathsf{Decode}(c)$ takes as input an encoded message c consisting of n chunks, and reconstructs a message m'.

A message can be recovered correctly if at most e chunks of the encoded message are corrupted. More formally, given $c = \mathsf{Encode}(m)$, we have $\mathsf{Decode}(c') = m$ if c and c' differ in at most e chunks.

We have error-correcting codes that can correct up to $e = (n-k)/2$ errors, e.g., Reed-Solomon codes. In the context of Byzantine consensus, we usually let each party handle one chunk, so n is (conveniently) both the number of parties and the number of chunks in the encoded message. We need to correct up to f erroneous chunks, since there are up to f Byzantine parties. Thus, we target the setting with $n \geq 3f+1$ and set $k = n-2f$, so that $(n-k)/2 = f$ errors can be corrected.

Another building block we will use is *data dissemination*, introduced by Das, Xiang, and Ren. A data dissemination algorithm *efficiently* disseminates a long message from a sufficiently large subset of honest parties to all honest parties. More formally, a data dissemination algorithm requires the following initial condition: at least $f+1$ honest parties have the same ℓ-bit input m while all the other honest parties have the special input $\perp$. Under this condition, all honest parties output m at the end of the algorithm.

If the initial condition is not satisfied, i.e., if f or fewer honest parties have the same input or if honest parties have different inputs, a data dissemination algorithm provides no guarantees.

If we do not impose an efficiency requirement, data dissemination is trivial: all honest parties who have the input send it to all parties, and every party outputs the most frequently received value. But this simple algorithm has $O(n^2\ell)$ bit complexity. Algorithm 15 presents a synchronous data dissemination algorithm that tolerates $f < n/3$ Byzantine faults and has $O(n\ell)$ bit complexity for sufficiently large ℓ.

Algorithm 15: An efficient data dissemination algorithm tolerating $f < n/3$ Byzantine faults. Code below is for party i whose input is x_i.

Round 1: If $x_i \neq \perp$, $c_i = \mathsf{Encode}(x_i)$ and send j-th chunk $c_i[j]$ to each party j.
Round 2: Let $c[i]$ be the chunk received from at least $f+1$ parties in round 1. Send $c[i]$ to all parties.
End of round 2: Let $c'[j]$ be the chunk received from party j in round 2. Output $\mathsf{Decode}(c'[1 \ldots n])$.

We prove the correctness and efficiency of Algorithm 15.

Proof. Suppose at least $f+1$ honest parties have the same input x and all other honest parties have input $\perp$. Let $c = \mathsf{Encode}(x)$. In round 1, every honest party j receives $c[j]$ from at least $f+1$ parties, and any different value for the j-th chunk appears at most f times (sent by Byzantine parties). Thus, in round 2, every honest party i sends the correct $c[i]$, so every honest party receives at most f erroneous chunks (from Byzantine parties). With the parameters we use, Decode can correct up to f errors, so every honest party reconstructs and outputs x.

Every honest party sends at most two chunks. For sufficiently large ℓ, each chunk has size $\lceil \ell/k \rceil$. Note that $k = n-2f = \Theta(n)$. The total communication complexity in bits across all parties is $2n^2 \cdot \lceil \ell/k \rceil = O(n\ell)$. □

Remark 7.4.1 When ℓ is small, each chunk must still be at least a bit. In fact, for Reed-Solomon codes, each chunk must be at least $\Theta(\log n)$ bits. Thus, the bit complexity of Algorithm 15 for a general ℓ is $O(\max(n\ell, n^2 \log n)) = O(n\ell + n^2 \log n)$.

7.4.3 Byzantine Consensus Extension Protocols

Byzantine agreement extension. We now describe an efficient Byzantine agreement (BA) algorithm for long messages in Algorithm 16. The high-level idea is as follows. Parties first run a BA to agree on a short hash digest. Then, parties run another binary BA to determine whether enough parties have the message that matches the agreed-upon hash digest. If so, parties use an efficient data dissemination (DD) algorithm to help everyone retrieve the message; otherwise, parties output $\perp$.

Algorithm 16: A synchronous agreement algorithm for long inputs against $f < n/3$ Byzantine faults. Code below is party i whose input is x_i.

Step 1: $h \leftarrow \mathsf{BA}(H(x_i))$ `// Run Byzantine agreement on the hash digest`
Step 2: If $H(x_i) = h$, send "happy" to all parties; else, set $x_i \leftarrow \perp$.
Set $b_i \leftarrow 1$ if receiving "happy" from $n - f$ parties; else set $b_i \leftarrow 0$.
Step 3: $b \leftarrow \mathsf{BA}(b_i)$ `// Run Byzantine agreement on a single bit`
If $b = 0$, output $\perp$ and terminate.
Step 4: $y_i \leftarrow \mathsf{DD}(x_i)$ `// Invoke data dissemination if` $b = 1$
Output y_i and terminate.

Correctness proofs. We prove the correctness of Algorithm 16. Liveness is clear. We focus on validity and safety.

Proof. Validity. Suppose all honest parties have the same input x. They all input $H(x)$ to the first BA. By the validity of that BA, all honest parties obtain $h = H(x)$. Thus, every honest party sends "happy" and sets $b_i \leftarrow 1$ in step 2. The BA in step 3 outputs $b = 1$ (again by validity), so DD is invoked. All honest parties input x to DD, so they all output x from both DD and Algorithm 16.

Safety. All honest parties agree on h and b. If $b = 0$, all honest parties output $\perp$ and safety holds. If $b = 1$, DD is invoked. In this case, at least one honest party inputs 1 to the BA in step 3, so it must have received "happy" from $n - f$ parties. Thus, at least $n - 2f \geq f + 1$ honest parties are "happy". By the collision resistance of the hash function H, all the "happy" honest parties have the same input x such that $H(x) = h$, and they input x to DD. The other honest parties (if any) are "unhappy" and input $\perp$ to DD. The initial condition of DD holds. Hence, all honest parties output x from both DD and Algorithm 16. □

Efficiency analysis. We focus on the bit complexity. In the setting of synchrony and $f < n/3$, BA algorithms with the optimal $O(n^2)$ bit complexity are known (e.g., Coan and Welch; Berman, Garay, and Perry). Step 1 invokes BA on κ-bit inputs where κ is the hash digest length; this step incurs $O(n^2\kappa)$ bit complexity. The "happy" message in step 2 can be a single bit, so this step incurs $O(n^2)$ bit complexity. Step 3 invokes BA on 1-bit inputs and incurs $O(n^2)$ bit complexity. Data dissemination in step 4 incurs $O(n\ell + n^2 \log n)$ bit complexity (see Remark 7.4.1). Note that κ is typically 256, so $\log n < \kappa$. The overall bit complexity of Algorithm 16 is therefore $O(n\ell + n^2\kappa)$.

When ℓ is sufficiently large, the $n\ell$ term dominates, and the $O(n\ell)$ bit complexity is substantially better than the naïve $O(n^2\ell)$ approach of including the input in every message.

More extension protocols. While we presented a BA extension protocol in synchrony with $f < n/3$ as an example, extension protocols for other consensus problems (e.g., broadcast) and other settings (e.g., $f < n/2$ or asynchrony) have been extensively studied.

We very briefly describe an extension protocol tailored for asynchronous or partially synchronous leader-based Byzantine consensus algorithms, such as Bracha's reliable broadcast and PBFT. Instead of directly proposing a value x, the leader proposes $H(x)$, the hash digest of x. The leader encodes its proposed value using error-correcting codes (or erasure codes) and sends the i-th chunk to party i. Upon receiving the above from the leader, party i sends a vote for $H(x)$ and forwards the i-th chunk to all parties. When a party receives enough votes for $H(x)$, it also receives enough chunks to reconstruct x. Thus, if $H(x)$ is decided, enough honest parties possess x, and they can use data dissemination to help other parties retrieve x. As every party sends only one chunk to every other party, the bit complexity is $O(n\ell)$ for sufficiently large ℓ.

This method is known as *verifiable information dispersal* (VID) and was introduced by Cachin and Tessaro (2005). It is particularly relevant in practice given the popularity of partially synchronous leader-based Byzantine consensus algorithms. We refer the reader to their work for further details.

Optimal bit complexity with long messages. For broadcast-like problems where only the sender has an ℓ-bit input, $\Omega(n\ell)$ is an obvious lower bound on the bit complexity, even in the best case, because every other party needs to learn the sender's input. A more involved argument establishes the same $\Omega(n\ell)$ lower bound for agreement-like problems.

Combined with the lower bounds in Chapter 7.1, we have $\Omega(n\ell + n^2)$ as a lower bound for deterministic Byzantine consensus with ℓ-bit inputs. The bound simplifies to $\Omega(n\ell)$ when ℓ is sufficiently large, and to $\Omega(n^2)$ when ℓ is small.

CHAPTER 8
State Machine Replication

In previous chapters, we focused on *single-shot* consensus, i.e., consensus algorithms in which parties agree on and output a single value. Single-shot consensus suffices to illustrate the core ideas of consensus algorithms. Real-world systems, however, typically process a growing sequence of inputs, commands, or transactions. As discussed in Chapter 1.1, this abstraction is captured by *State Machine Replication* (SMR), the primary application of consensus. This chapter introduces the SMR problem and presents several widely used paradigms for implementing it.

8.1 Problem Definitions

8.1.1 Multi-shot Consensus

We first define *multi-shot* consensus, in which replicas repeatedly reach consensus on a growing sequence of values. Multi-shot consensus has also been referred to as *atomic broadcast, total-order broadcast, replicated logs*, and, more recently, *distributed ledgers* and *blockchains*.

We refer to each party as a *replica*. Each replica i decides a growing sequence of values, called its *log* and denoted by Log_i. The log is indexed by consecutive integers starting from 1. $\mathsf{Log}_i[s]$ is the s-th value decided by replica i. The index s is called a *slot* or a *height*.

Definition 8.1.1 (Multi-shot consensus, informal) Each replica may receive many inputs over time. Each replica i has a growing log Log_i. Initially, $\mathsf{Log}_i[s] = \emptyset$ for every replica i and every height s. A replica writes each log entry at most *once* (to a value that is not $\emptyset$). A multi-shot consensus algorithm must achieve the following properties.

- ◇ (Safety) Correct replicas agree on each decided value, i.e., if two correct replicas i and j have $\mathsf{Log}_i[s] \neq \emptyset$ and $\mathsf{Log}_j[s] \neq \emptyset$, then $\mathsf{Log}_i[s] = \mathsf{Log}_j[s]$.
- ◇ (Liveness) If a correct replica inputs x, x is eventually recorded in the log, i.e., there exists a height s such that every correct replica i eventually sets $\mathsf{Log}_i[s] = x$.

8.1.2 State Machine Replication

State machine. We first define a *state machine*. At any point in time, a state machine stores a system's current state, starting from a fixed initial state. It continuously receives inputs, also called *commands*, and processes them one at a time. Each command is applied

using a deterministic state transition function, which produces an output and updates the state. The following pseudocode gives a simple description of a state machine.

```
state := initial state
log := []
while true
    upon receiving cmd
        log.append(cmd)
        state, output := apply(cmd, state)
```

Here, we assume that the state machine adds each command to a log before executing it. This is not essential to the definition of a state machine, but it provides a convenient connection to state machine replication.

We consider the standard client-server setting. The server maintains the state machine, and clients submit commands to the server. The server executes each command and sends the resulting output to the client (who issued the command).

State machine replication (SMR). The state machine replication approach assigns multiple servers to jointly maintain the state machine, so that the state machine continues to function correctly even if a fraction of the servers are faulty.

Once we have a multi-shot consensus algorithm, we are very close to obtaining SMR. The servers (replicas) use the multi-shot consensus algorithm to agree on the growing log of commands. They then execute the same ordered sequence of commands, so they remain in the same state and produce the same output after executing each command. (The execution, i.e., the state transition function, must be deterministic. If an SMR-based system wishes to incorporate randomness, randomness must be derived from the commands.)

A remaining discrepancy that must be addressed is that SMR involves two types of parties, clients and servers, whereas multi-shot consensus is formulated with a single type of parties (replicas). To obtain SMR from multi-shot consensus, replicas play the role of servers, while inputs originate from clients. Furthermore, a client must also be able to obtain the state machine's output for its command, even if the client is not present when its command is decided in the log and executed.

For all practical purposes, SMR can be viewed as multi-shot consensus with client support. We give the following *informal* definition of SMR.

Definition 8.1.2 (State machine replication, informal) Many clients submit commands over time. Initially, $\mathsf{Log}_i[s] = \emptyset$ for every replica i and every height s. A replica writes each log entry at most *once*, to a client command. A state machine replication algorithm must achieve the following properties.

- (Safety, same as Definition 8.1.1) If two correct replicas i and j have $\mathsf{Log}_i[s] \neq \emptyset$ and $\mathsf{Log}_j[s] \neq \emptyset$, then $\mathsf{Log}_i[s] = \mathsf{Log}_j[s]$.
- (Liveness) If a correct *client* submits a command x, then:
  - x is eventually recorded in the log, i.e., there exists a height s such that every correct replica i eventually sets $\mathsf{Log}_i[s] = x$, and
  - the client eventually receives the output produced by the state machine after executing the commands in a correct replica's log up to (and including) x.

Each command decided in the log must also satisfy the syntactic and semantic requirements imposed by the application, ensuring that only well-formed and application-compliant commands are executed by the replicated state machine. This requirement is known as *external validity*.

Remark 8.1.3 We emphasize that the Definition 8.1.1 and Definition 8.1.2 are both *informal*. They do *not* constitute fully rigorous definitions of multi-shot consensus and state machine replication. In this specific case, however, readers should not be overly concerned about the lack of full rigor. The informal definitions provided here suffice for all practical purposes.

As an algorithm textbook that emphasizes formal modeling and rigorous proofs, the decision to compromise on rigor is not made lightly and warrants a detailed explanation.

In what sense is Definition 8.1.1 for multi-shot consensus not rigorous? The issue is that there exist algorithms that satisfy this definition, but are not proper solutions to the multi-shot consensus problem. But this concern is not too serious in practice, because a pathological construction that exposes this deficiency exists, but is not immediately obvious. This contrasts with, for example, the validity requirement in broadcast and agreement problems. There, dropping validity would lead to an obvious trivial solution.

More importantly, a fully rigorous definition would be cumbersome. Most attempts in the literature fall short in one way or another. Some apply only to benign faults (e.g., crash and omission), but not to Byzantine faults. Some rule out certain pathological solutions but still permit other pathological solutions. Some use language that is itself informal or imprecise.

Having said the above, we provide a fully rigorous definition of multi-shot consensus in Chapter 8.1.3. This serves two purposes. First, it assures readers that such a definition can indeed be given. Second, it illustrates that adding full rigor here is tedious and contributes little to the essence of the subject.

The aforementioned challenges apply to SMR as well. But even after they are addressed in Chapter 8.1.3, we choose to omit a formal definition of state machine replication. To provide the right abstraction, the interface and guarantees of SMR should be stated *entirely* from the clients' perspective, with little or no mention of servers or replicas. The SMR guarantees should also incorporate *linearizability*, a foundational concept in the broader field of distributed algorithms but not central to the study of consensus. For these reasons, an appropriate and rigorous definition of SMR would be even more tedious and cumbersome than that of multi-shot consensus, and contribute even less to the essence of the subject.

To the author's knowledge, SMR is usually defined informally in the literature, at a level of rigor comparable to Definition 8.1.2. As long as the definition captures consensus on a log and client support, it should not hinder the clarity or correctness of results on SMR.

8.1.3 A Rigorous Definition of Multi-shot Consensus

(Please read Remark 8.1.3 before this subsection. Readers not looking for full theoretical rigor may skip this subsection.) Our formal definition of multi-shot consensus has the

same safety and liveness properties as Definition 8.1.1, but adds two additional properties to rule out pathological solutions.

Definition 8.1.4 (Multi-shot consensus, formal) Each replica may receive many inputs over time. All inputs are drawn from a domain $\mathcal{V}$. Each replica i has a growing log Log_i. Initially, $\mathsf{Log}_i[s] = \emptyset$ for every replica i and every height s. A replica writes each log entry at most *once*, to a value in $\mathcal{V}$ ($\emptyset \notin \mathcal{V}$). A multi-shot consensus algorithm must achieve the following properties.

- (Safety, same as Definition 8.1.1) If two correct replicas i and j have $\mathsf{Log}_i[s] \neq \emptyset$ and $\mathsf{Log}_j[s] \neq \emptyset$, then $\mathsf{Log}_i[s] = \mathsf{Log}_j[s]$.
- (Liveness, same as Definition 8.1.1.) If a correct replica inputs x, then there exists a height s such that every correct replica i eventually sets $\mathsf{Log}_i[s] = x$.
- (Completeness) If a correct replica i has $\mathsf{Log}_i[s] \neq \emptyset$, then every correct replica j eventually has $\mathsf{Log}_j[s'] \neq \emptyset$ for every height $s' \leq s$.
- (Integrity) There exists a sub-domain $\mathcal{V}_H \subseteq \mathcal{V}$ that satisfies the following: (i) there is an efficient algorithm $\mathsf{Verify}(\cdot)$ that checks if a value is in $\mathcal{V}_H$; (ii) it is infeasible for faulty replicas to generate a value in $\mathcal{V}_H$; and (iii) if a correct replica i has $\mathsf{Log}_i[s] = x$ for some $x \in \mathcal{V}_H$, then some correct replica previously input x.

The completeness property rules out algorithms that advance to higher heights while leaving a lower height permanently unresolved. Such a "hole" in the log would stall the execution of the state machine if the multi-shot consensus algorithm were used to implement SMR, as observed by Blum, Katz, and Loss.

The integrity property states that some values can only originate from correct replicas (for example, transactions that spend their funds), and that any such value is decided only if it was actually input by a correct replica. This rules out degenerate solutions that simply write every value in $\mathcal{V}$ to the log in some predetermined order, without performing any meaningful input-driven computation and communication.

If we only consider benign faults (e.g., crash or omission), the integrity property can be formulated more simply: every decided value must have been input by some replica. This formulation is imprecise in the Byzantine setting. A Byzantine replica can easily induce a decision that was never input by any honest replica. At the same time, it is not meaningful to speak of "Byzantine inputs", because Byzantine replicas may behave arbitrarily, and there may not be a well-defined way to attribute any of their actions as "inputting a value". This is why Definition 8.1.4 requires additional machinery to rigorously capture the integrity property in the Byzantine setting.

Remark 8.1.5 Multi-shot consensus with benign faults sometimes includes a "no duplication" property, requiring that each input be decided at most once. This is a reasonable additional constraint in those settings.

However, it is not rigorous in the Byzantine setting, again because the notion of Byzantine inputs is not well-defined. It should also not be included in single-shot consensus, where it is redundant and already implied by the "single-shot" nature of the problem.

8.2 Leader-based SMR

8.2.1 Composition of Single-shot Broadcast

The simplest way to obtain multi-shot consensus and SMR is to compose a sequence of single-shot consensus instances. Concretely, run a sequence of broadcast (Definition 1.2.1) or partially synchronous broadcast (Definition 5.1.4) instances, with replicas taking turns as the sender. Whenever a replica becomes the sender, it broadcasts the oldest client command it has received (or, in multi-shot consensus, its oldest input). Each replica i writes the output of the s-th broadcast instance to $\mathsf{Log}_i[s]$. It is easy to prove the correctness of this solution.

Proof. Safety at each height is guaranteed by the safety of each broadcast instance. For liveness, consider the moment a correct replica i receives a value x (either a client command or a replica input). At this time, replica i may have a finite number of values older than x waiting to be sent through broadcast. Every time replica i becomes the sender (either in synchrony or during a synchronous period under partial synchrony), it successfully broadcasts one value. After finitely many turns, every value older than x as well as x itself is sent through a broadcast instance, output by that broadcast instance, and written to the log. □

We mention a simple optimization. If a replica is expected to accumulate many values before becoming the sender again, we can have each sender broadcast a batch of up to B values, where B is a fixed parameter. The batch occupies up to B consecutive heights in the log. This technique is known as *batching*, and such a batch is often called a *block*.

We emphasize again that, although synchronous broadcast and the resulting multi-shot consensus can tolerate any $f < n$ Byzantine faults, state machine replication based on them tolerates at most $f < n/2$ Byzantine faults due to the need to support clients (see Chapter 6.1.1). The optimal fault tolerance thresholds for SMR in the mainstream settings have been summarized in Chapter 6.4.

8.2.2 The View-based Approach

The sequential composition solution is conceptually simple but is not very efficient. Practical SMR algorithms instead compose single-shot consensus instances in a non-black-box manner.

Many practical SMR algorithms follow the view-based approach pioneered by Paxos and PBFT. It is helpful to view such an algorithm as consisting of two components: *normal operation* and *view change*.

- During normal operation, a stable leader is in charge. For each height, the leader sends a proposal, replicas vote on it, a consensus decision is reached, and this process repeats for the next height.
- If progress stalls, replicas initiate a view change to switch to the next leader.

The view change procedure can be complex, especially in the presence of Byzantine faults. The new leader needs to collect status reports from replicas for many heights. Based on the collected information, the new leader potentially needs to (i) learn from other replicas about decisions it missed; (ii) inform other replicas about decisions they

missed; (iii) repropose values at certain heights based on delicate rules; and (iv) justify its choice of reproposed heights and reproposed values. Although the high-level ideas are the same as in the single-shot versions presented in Chapter 5, extending them to multi-shot consensus introduces substantial bookkeeping, and the precise details are tedious and error-prone.

8.2.3 The Block Chaining Approach

Ethereum proposes a conceptually simpler approach, inspired by Bitcoin's block-chaining structure (Chapter 9.2.2). Instead of reaching consensus on each height independently, the idea is to link the heights using a collision-resistant hash function. This way, a proposal at height s implicitly specifies the entire sequence of values from heights 1 through s.

More precisely, let H be a collision-resistant hash function (Chapter 7.4.1). A block B at height s has the form

$$B = (s, x, h')$$

where s is the height, x is a batch of client commands, and $h' = H(B')$ is a hash of a block B' at the preceding height $s - 1$. In this case, we say B *succeeds* B'. A special block $(0, \perp, \perp)$ is considered to occupy height 0.

Remark 8.2.1 (Batching) Note that we have incorporated the *batching* technique: x is not a single client command but a batch of client commands. Strictly speaking, there needs to be a translation between the block height here and the log height in Definitions 8.1.1 and 8.1.2, but we omit these details.

By including the previous block's hash, a block at height s recursively determines a unique sequence of ancestor blocks from heights 1 through s (ignoring the special block at height 0). Consequently, when a block is added to the log, all of its ancestor blocks are also added to the log (if they have not already been recorded).

Algorithm 17 presents a PBFT-based SMR algorithm using the block-chaining approach. The reader may find it helpful to first review the single-shot variant of PBFT in Chapter 5.3. It is crucial that quorum certificates (QCs) are ranked by the views in which they are generated, *not* by the heights of their certified blocks.

Theorem 8.2.2 Algorithm 17 achieves SMR safety and liveness.

The proof largely follows that of Chapter 5.3, and we do not repeat all the details. As before, let $\mathcal{C}_k(B)$ denote a quorum certificate (QC) generated in view k for block B, i.e., $\langle \text{Vote1}, k, B \rangle_Q$ where $|Q| = n - f$. With a slight abuse of notation, if a block Y extends (not necessarily immediately) block B, we also say $\mathcal{C}_k(Y)$ extends B.

Proof sketch. Safety. Suppose a block B at height s is decided by some honest party. Let k be the lowest view in which some honest party decides B. It suffices to prove that every QC $\mathcal{C}_{k'}(Y)$ with $k' \geq k$ extends B.

We prove by induction. The base case of $k' = k$ follows from a standard quorum intersection argument (Chapter 4.3.1). For the inductive step, suppose all QCs from view k to view $k' - 1$ extend B. Again by quorum intersection, the $n - f$ $(\text{Status}, k', *)$ messages obtained by $L_{k'}$ include at least one honest party's QC of $\mathcal{C}_k(B)$ (or a higher QC that extends B). Together with the inductive hypothesis, the highest QC obtained

Algorithm 17: PBFT with block chaining.

upon starting the algorithm
Initialize a local variable highestQC $= \emptyset$ and enter view 1

upon entering view k
Party i sends $\langle \text{Status}, k, \text{highestQC} \rangle_i$ to L_k, the leader of view k

upon receiving $\langle \text{Status}, k, * \rangle_*$ from $n - f$ parties while in view k
L_k sends $(\text{Propose}, k, B, \mathcal{S})$ to all parties, where $\mathcal{S}$ is the $n - f$ $\langle \text{Status}, k, * \rangle_*$ messages received, and the proposed block B succeeds a block with the *highest* QC (ranked by views) reported among $\mathcal{S}$

upon receiving $(\text{Propose}, k, B, \mathcal{S})$ from L_k while in view k
If the proposal satisfies the above rules, party i sends $\langle \text{Vote1}, k, B \rangle_i$ to all parties

upon receiving $\langle \text{Vote1}, k, B \rangle_Q$ where $|Q| = n - f$ while in view k
Party i sends $\langle \text{Vote2}, k, B \rangle_i$ to all parties and sets highestQC $= \langle \text{Vote1}, k, B \rangle_Q$

upon receiving $\langle \text{Vote2}, k, B \rangle_Q$ where $|Q| = n - f$ `// even outside view` k
Decide block B (and all its ancestor blocks)
Send $\langle \text{Vote2}, k, B \rangle_Q$ to all parties and all clients with newly decided commands

upon spending 4Δ time in view k or detecting misbehavior of L_k
Party i sends $\langle \text{ViewChange}, k \rangle_i$ to all parties and $\langle \text{Status}, k, \text{highestQC} \rangle_i$ to L_{k+1}

upon receiving $\langle \text{ViewChange}, k \rangle_Q$ where $|Q| = f + 1$ while in view k or a lower view
Send $\langle \text{ViewChange}, k \rangle_Q$ to all parties and enter view $k + 1$ (quit current view)

by $L_{k'}$ extends B. Thus, $L_{k'}$ must propose a block extending B to get honest votes, and any QC formed in view k' must extend B. This completes the inductive step.

Liveness. In a synchronous period, every view with an honest leader decides a block of new client commands, as there is sufficient time (4Δ) for the honest leader to collect status messages and send a valid proposal, and for the parties to complete two rounds of voting. Thus, every command from every honest client is eventually included in a decided block. The client can obtain the correct output either by receiving the same reply from $f + 1$ replicas or by receiving a reply signed by $f + 1$ replicas. □

How often to replace the leader? Algorithm 17 replaces the leader after every decision. This choice is orthogonal to the block-chaining approach. There are other natural designs. For example, the stable-leader paradigm allows a leader to propose blocks at increasing heights until it stops making progress. A middle-ground option allows a leader to propose for a bounded number of heights (say, 100) unless it stops making progress prematurely.

In practice, PBFT is not suitable for frequent leader changes because its view-change procedure is expensive. If frequent leader changes are desired, Tendermint is a good choice, due to its asymptotically more efficient view-change protocol.

Efficiency analysis. The good-case and worst-case efficiency analyses remain unchanged from Chapter 5.3. For multi-shot consensus and SMR, it is also natural to consider *amortized* efficiency. During a sufficiently long synchronous period, at least $n - f$ blocks

are decided every n views, because each honest leader's view decides a new block. Since each view has $O(1)$ rounds and $O(n^2)$ messages, the amortized round complexity is $O(1)$, and the amortized message complexity is $O(n^2)$ per block.

If we instead adopt the stable-leader paradigm, an honest leader can decide arbitrarily many blocks without a view change. In this case, the amortized efficiency approaches the good-case efficiency.

8.3 SMR via Asynchronous Common Subset

The leader-based SMR paradigms in the previous section work under synchrony and partial synchrony, but not under asynchrony. This subsection presents an asynchronous SMR algorithm based on single-shot reliable broadcast and (randomized) binary agreement.

Asynchronous common subset. We first introduce the Asynchronous Common Subset (ACS) problem.

Definition 8.3.1 (Asynchronous Common Subset) There are n parties. Party i has input x_i and eventually outputs a set Y_i. An algorithm solves *asynchronous common subset* if it guarantees the following:

- (Liveness) Every party eventually outputs.
- (Safety) Any two correct parties i and j output $Y_i = Y_j$.
- (Validity) For every correct party i, its output Y_i includes the inputs of at least $n - 2f$ correct parties.

Algorithm 18 presents the ACS algorithm of Ben-Or, Kelmer, and Rabin (BKR). The algorithm uses n parallel instances of reliable broadcast (RBC) and n parallel instances of (randomized) binary agreement (BA). It tolerates $f < n/2$ crash faults or $f < n/3$ Byzantine faults in asynchrony, both of which are optimal (Chapter 6.3). At a high level, the j-th BA instance determines whether the j-th RBC instance finishes correctly and timely. The final output consists precisely of the outputs of those RBC instances whose corresponding BA instances output 1.

The algorithm is mostly straightforward except for one key trick. An RBC instance may hang if its sender is faulty, so a party cannot wait for all RBC instances to finish. After $n - f$ RBC instances output, a party must give up on the remaining f RBC instances and input 0 to their corresponding BA instances. However, if every party does so immediately after observing $n - f$ RBC instances terminate, every party may see a different set of $n - f$ completed RBC instances. If that happens, every BA instance could have a correct party inputting 0. In that case, the strong unanimity validity allows that all BA instances output 0. This violates the ACS validity because the output set is not large enough.

The key trick is that a party delays inputting 0 until $n - f$ BA instances have already output 1. Once $n - f$ BA instances output 1, ACS validity is secured.

Lemma 8.3.2 $n - f$ BA instances output 1.

Proof. Suppose toward a contradiction that fewer than $n - f$ BA instances ever output

Algorithm 18: The BKR algorithm for asynchronous common subset (ACS).

upon algorithm starts
Start n instances of reliable broadcast (RBC), where party i is the sender of the i-th RBC and inputs x_i (its ACS input).
Start n instances of binary agreement (BA).

upon j-th RBC instance outputs
Input 1 to the j-th BA (unless it has already input 0).

upon $n - f$ BA instances output 1
Input 0 to each remaining BA (unless it has already input 1).

upon all BA instances output
Initialize $Y = \emptyset$.
For all j, if the j-th BA outputs 1, wait for the j-th RBC to output, and add its output y_j to Y.
Output Y and terminate.

1. In this case, no correct party ever inputs 0 to any BA instance. By BA validity, a BA instance cannot output 0 if no correct party inputs 0. Hence, the only possible scenario is that fewer than $n - f$ BA instances output 1 while all remaining BA instances never terminate.

However, the $n - f$ RBC instances with correct senders eventually terminate at all correct parties. All correct parties eventually input 1 to the corresponding $n - f$ BA instances. By BA liveness and validity, these $n - f$ BA instances eventually output 1, leading to a contradiction. □

Theorem 8.3.3 Algorithm 18 solves ACS.

Proof. **Safety.** All correct parties agree on all BA outputs. Thus, every correct party adds the same set of RBC outputs to its output set. The safety of ACS then follows from the safety of RBC.

Validity. By Lemma 8.3.2, every correct party's output set includes at least $n - f$ RBC outputs. At least $n - 2f$ of those are inputs of correct parties.

Liveness. By Lemma 8.3.2, $n - f$ BA instances output 1. Thus, every correct party eventually provides an input (either 1 or 0) to every BA instance. Thus, all BA instances eventually output. For every BA instance that outputs 1, at least one correct party inputs 1 to that BA, which means the corresponding RBC instance finished at that correct party. By RBC liveness, that RBC instance eventually finishes at all correct parties. The ACS liveness follows. □

Asynchronous multi-shot consensus and SMR. ACS provides a natural route to asynchronous multi-shot consensus and SMR. We use SMR as an example. Replicas execute a sequence of ACS instances. Every replica continuously collects commands from clients. In each ACS instance, every replica participates with its oldest pending command (or a batch of oldest pending commands).

ACS guarantees that all correct replicas eventually output the same set of values. Replicas then apply a deterministic ordering rule (e.g., sorting by replica ID) to this set,

and append the resulting ordered sequence of commands to the log. Note that different replicas may provide the same client command to an ACS instance, so a command may appear multiple times in the set. Deduplication can be performed before appending the sequence to the log.

Repeating this process across successive ACS instances yields a growing log of commands. Safety of SMR follows directly from the safety of ACS. For liveness, observe that every ACS instance's output includes inputs from correct replicas, and that correct replicas prioritize their oldest pending commands. Thus, all commands from correct clients eventually appear in the log.

8.4 Reconfiguration

So far, we have defined consensus problems assuming a fixed set of participants throughout the execution. This assumption is natural for single-shot consensus problems, but need not hold for multi-shot consensus or SMR. The process of changing the set of replicas is called *reconfiguration*, and it has been studied extensively.

The simplest approach to reconfiguration is to rely on the replicated log itself. A command in the log can be a special reconfiguration command that adds, removes, or replaces one or more replicas. Whether such a command is valid is an application-specific matter. For example, a system may require that a reconfiguration command include a signed request from the departing replica, an endorsement for its replacement, approval by a majority of current replicas, or some combination of these requirements.

Because all correct replicas agree on the contents of the replicated log, they agree on the set of reconfiguration commands and, hence, the set of participating replicas at all times. Most SMR algorithms can support reconfigurations this way with only minor modifications to their base algorithms.

CHAPTER 9

Nakamoto Consensus

Bitcoin is a decentralized digital currency and payment system designed in late 2008 and deployed in early 2009 by its mysterious and pseudonymous inventor, Satoshi Nakamoto. Since its introduction, Bitcoin has revived and fundamentally reshaped the study of Byzantine consensus. Its significance stems from three revolutionary contributions:

- a radically new *permissionless* model for Byzantine fault-tolerant (BFT) consensus,
- an elegant and unconventional solution to BFT state machine replication (SMR),
- an ingenious application of BFT SMR to decentralized digital currency.

As a textbook on algorithms, our primary focus will be on the first two contributions of Bitcoin: its new model and novel consensus algorithm. However, it is difficult to overstate Bitcoin's transformative insight and impact on the application of BFT consensus. For this reason, we begin our study of Bitcoin from the application perspective. We then present Bitcoin's consensus algorithm, which we refer to as *Nakamoto consensus*.

9.1 Bitcoin and Decentralized Applications

9.1.1 The Missing Application of BFT

Before Bitcoin's rise to fame, research on Byzantine fault-tolerant (BFT) consensus had largely remained a topic of theoretical interest with unclear practical value. As discussed in Chapter 1.1, the primary application of consensus was State Machine Replication (SMR): an organization or company runs a web service, and deploys multiple servers replicating the service in order to improve reliability and availability. In such settings, *crash* faults are the dominant concern, and crash-tolerant SMR algorithms, such as Paxos, have been widely adopted in practice.

By contrast, it was difficult to imagine a scenario in which several servers within the organization would fall under complete adversarial control. If such a catastrophic compromise were to occur, the organization would likely face problems far more severe than service consistency.

One might argue that servers may still exhibit erroneous behaviors due to hardware faults, bit flips during transmission or storage, software bugs, misconfigurations, or computation errors. However, these issues can often be mitigated through lighter-weight mechanisms such as redundancy, checksums, and error-correcting codes. They do not require the full machinery of BFT. Consequently, although the theory of BFT consensus was elegant and algorithms such as PBFT demonstrated promising performance, there appeared to be no compelling real-world application.

Bitcoin fundamentally changed this perspective by introducing a new class of applications: decentralized systems maintained collectively by mutually distrusting parties without any central authority. No single organization controls the service or the underlying infrastructure. Instead, the system's properties, such as consistency and availability, must emerge from the guarantees of the underlying distributed algorithm. In this environment, Byzantine behavior is not merely possible but expected. In this sense, Bitcoin transformed BFT consensus from a largely theoretical discipline into the foundation problem underpinning practical decentralized systems.

9.1.2 Bitcoin: Decentralized Currency from BFT SMR

Bitcoin itself provides the canonical example of such a decentralized application: a peer-to-peer digital currency and payment system. The system maintains a global ledger recording the minting, ownership, and transfer of coins. Yet this ledger is not operated by any bank, company, or government authority. Instead, a distributed network of mutually distrusting participants collectively maintains and agrees on the ledger by executing a BFT SMR algorithm over the Internet.

Transactions in Bitcoin's ledger. As a textbook on algorithms, we rarely delve into application-specific details. In the preceding chapters, we have presented algorithms for reaching consensus on a generic value x or an abstract log of values, without discussing the concrete format or meaning of these values. In this section, however, we describe the "values" in Bitcoin's ledger in greater detail, partly due to Bitcoin's prominence and foundational role in decentralized systems, and partly because its ledger's design embodies elegant ideas worthy of study.

At a high level, the "values" in the Bitcoin ledger are payment transactions. A transaction can be viewed as a statement of the following form:

"Alice pays Bob 3.1415 bitcoins."

Remark 9.1.1 The lowercase "bitcoin" is the unit of currency, while "Bitcoin" (with a capital B) refers to the entire payment system, including the algorithm, protocol, network, and implementation.

In a payment system, a transaction must be authorized by the payer. That is, only Alice should be able to generate the transaction above. The appropriate tool to achieve this is a digital signature scheme (see Chapter 1.3.4).

All identities are now represented by public keys of the digital signature scheme. Concretely, Alice and Bob are identified by their respective public keys, denoted pk_A and pk_B. The corresponding secret key sk_A is known only to Alice. The above Bitcoin transaction is therefore of the form:

"pk_A pays pk_B 3.1415 bitcoins,"
together with a signature on the above statement generated using sk_A.

Or more concisely, using our notation for signatures,

$$\langle \mathsf{pk}_A \text{ pays } \mathsf{pk}_B \text{ 3.1415 bitcoins} \rangle_A.$$

The signature serves as cryptographic proof that the payer authorized the transaction. The authenticity of the transaction can be verified by anyone with pk_A, which is included as part of the transaction. In particular, replicas executing the BFT SMR must verify the signature of each transaction. Due to its use of cryptography, Bitcoin is also called a cryptocurrency.

It should be clear by now that the secret key controls access to a user's funds. If Alice loses the only hard drive storing her secret key, her funds become permanently unspendable. If Mallory steals Bob's secret key, Mallory can transfer Bob's funds to herself.

It is also worth noting that Bitcoin provides a privacy property called *pseudonymity*. Users of Bitcoin are not identified by their real-world names such as Alice or Bob, but rather by cryptographic public keys. Mapping these cryptographic keys to real-world identities is possible but challenging. This property is a double-edged sword: it provides a degree of privacy for users but also facilitates illicit and criminal transactions by reducing real-world accountability.

Consensus prevents double spending. It is now time to connect Bitcoin's transaction format back to BFT SMR. The Bitcoin ledger is simply an ordered list of transactions. The role of SMR is to ensure that all honest participants agree on the same sequence of transactions and, hence, on the same evolution of the system state, i.e., the balance associated with every public key.

When processing a transaction, replicas perform two fundamental checks. First, they verify that the transaction is properly authorized by verifying the signature. Second, they verify that the payer has sufficient funds according to the transaction history agreed upon so far. In other words, the validity of a transaction depends on both cryptographic authorization and the current system state.

Without consensus, different users may observe different transaction histories and hence different account balances. This would allow the *double-spending* attack: the same coin may appear to be paid to Bob from the perspective of some users, while appearing to be paid to Charlie from the perspective of other users.

Preventing double-spending is the central challenge in implementing a decentralized digital currency. The difficulty stems from the fact that digital information can be copied easily and perfectly. Unlike physical cash, a digital "coin" is merely data, and nothing prevents a malicious user from generating conflicting (but otherwise valid) payment messages involving the same coin and sending them to multiple recipients. Lacking a decentralized solution to double-spending, digital currency proposals prior to Bitcoin ultimately fell back on some form of trusted authority to resolve conflicting payments, which is why they failed to generate serious interest.

Summary of Bitcoin's application-level design and insight. To summarize the high-level design of Bitcoin, it combines digital signatures and consensus to implement a decentralized cryptocurrency and payment system. Specifically, (i) public keys serve as pseudonymous account identifiers, (ii) signatures generated using secret keys authorize spending, and (iii) consensus on the transaction history prevents double spending.

We reiterate that the last connection between consensus and currency is one of Bitcoin's key insights: consensus on transaction history suffices to implement a decentralized

currency. If this connection appears obvious today, it is likely a benefit of hindsight. Although this connection can be viewed as a special case of the decade-old observation that SMR implements an arbitrary service, in the specific context of currency, it was not readily apparent, partly due to the prevailing intuition that (fiat) money derives its value from the faith and credit of governments and financial institutions.

9.1.3 Decentralized Applications beyond Currency

Bitcoin's success naturally triggered an explosion of interest in decentralized applications beyond payment systems. The central idea that an agreed-upon ledger (i.e., SMR) can replace a trusted authority is, in principle, applicable to any application. Projects such as Ethereum generalize the approach to what they call *smart contracts*. Behind the nondescript jargon, a smart contract is just a replicated state machine whose commands are generic instructions (analogous to CPU instructions), thereby allowing any decentralized application to be programmed.

As of 2026, nearly every conceivable application domain has likely inspired attempts at decentralization. Examples include decentralized finance (DeFi), identity systems, storage networks, gaming and betting platforms, digital collectibles (NFTs), ticket resale platforms, insurance markets, supply-chain tracking, anti-counterfeiting systems, diploma verification, and many others. The area continues to attract enormous interest and investment. At the same time, it is also characterized by substantial hype, overzealous entrepreneurship, and outright fraud, perhaps to a greater extent than most other tech industries. It is beyond the scope of this textbook to evaluate the merits of each proposed application. Nevertheless, several general observations are worth keeping in mind.

First, the primary motivation for decentralization is to avoid a trusted central authority. However, many applications inevitably rely on trusted entities, especially when physical objects and real-world events are involved. For example, a decentralized supply-chain application may reliably record that "banana #2065642 is certified organic", but it has no way to verify whether banana #2065642 is actually organic or whether a given banana is actually banana #2065642. Establishing these facts typically requires trusting the farmers and the truck drivers. For another example, as universities are already trusted to issue diplomas correctly, one may question the need for a decentralized diploma verification platform. Why not simply trust the same university to operate its own diploma verification service? In these and many other applications, the value of decentralization requires careful justification, especially when only part of the application is migrated to a decentralized design.

Second, decentralized systems inherently incur substantial overheads in communication, computation, storage, and latency in order to eliminate trusted entities and single points of failure. A central question for any proposed decentralized application is whether the benefits of decentralization outweigh these costs. (Claims that decentralization improves performance while reducing trust relative to centralized alternatives should particularly be viewed with skepticism.)

9.2 The Nakamoto Consensus Algorithm

9.2.1 The Permissionless Setting

Prior to Bitcoin, Byzantine consensus was studied in the traditional setting we have used in previous chapters: all participants know each other, and a large fraction of them (typically one-half or one-third) faithfully follow the algorithm at all times.

Bitcoin considers a radically different setting, often referred to as the *permissionless* setting. Anyone on the Internet may join as a participant or leave the system at any time, without notice or authorization. A participant does not know most other participants (it only knows a few neighbors), nor does it know how many participants there are in total.

The permissionless setting represents a significant departure from the classical setting of consensus. It introduces a host of new challenges, as many assumptions taken for granted in classical BFT no longer hold.

A first challenge concerns communication. In the classical setting, it is customary to assume a direct communication channel between every pair of participants. This assumption is not appropriate in the permissionless setting. Not only would maintaining connections with all participants be prohibitively expensive, but it is also impossible because a participant does not even know the complete set of participants in the network.

Bitcoin instead adopts a peer-to-peer communication model. Each participant maintains network connections to only a small number of neighboring participants. When a participant wishes to disseminate a message to the entire network, it sends the message to its neighbors, who then forward the message to their own neighbors. Through this gossip-style propagation process, a message gradually reaches the entire network.

A much harder and more fundamental challenge is that we can no longer rely on an upper bound on the *number* of malicious participants (i.e., f in previous chapters). Such an upper bound is meaningless in the permissionless setting because identities are cheap to create. A malicious participant can easily create thousands of seemingly distinct identities and participate under all of them simultaneously. This problem is known as the *Sybil attack*. Overcoming the Sybil attack is arguably the central challenge of the permissionless setting. We next discuss how the Bitcoin consensus algorithm addresses this challenge.

9.2.2 Proof-of-Work

Many classical BFT algorithms, including those from previous chapters, are based on voting (possibly in multiple rounds). Participants exchange votes, and a value is committed (i.e., decided or output) once it receives support from a majority or supermajority of participants. In a permissionless setting, however, this basic form of voting does not make sense. Due to the Sybil attack, a malicious participant can easily create a large number of identities and cast votes under all of them. Bitcoin hence requires a voting mechanism that is resistant to Sybil attacks.

Proof-of-Work. Bitcoin's solution is the notion of *proof-of-work* (PoW), a concept first proposed by Dwork and Naor (in a different context from consensus). The high-level idea is to assign voting power based on computational work rather than identities. To cast a vote, a participant must first solve a moderately hard computational puzzle. Creating

additional identities provides no advantage in solving the puzzle and hence does not increase one's voting power.

The puzzle itself is based on a *pseudorandom* cryptographic hash function H, which is introduced in Chapter 7.4.1. Being pseudorandom means that, for a previously unseen input x, $H(x)$ appears to be sampled uniformly at random from the output space.

Given a puzzle z, a solution to the puzzle is a value, called a *nonce*, such that

$$H(z \mid nonce) < T,$$

where $\mid$ denotes bit concatenation and T is a publicly known threshold.

Because the output of H appears random, there is no known shortcut for finding such a nonce. The only viable strategy is trial and error: repeatedly try different nonce values, evaluate the hash function, and check whether the output falls below the threshold T. Let M denote the size of the output space of H, i.e., hash outputs are integers in $\{0, 1, \ldots, M-1\}$. Then, in expectation, M/T hash evaluations are required to solve the puzzle. For this reason, a nonce that satisfies the condition is called a *proof-of-work* (PoW).

Importantly, while finding a PoW requires a huge number of hash evaluations, verifying one requires only a single hash evaluation. This asymmetry between expensive generation and cheap verification is an important property of PoW.

The puzzle and block chaining. Now that we understand PoW, the next question is: what exactly should the PoW puzzle be?

Conceptually, a solution to a puzzle z can be considered a *vote* for z. Thus, ideally, we would like to make the puzzle the entire ledger so that participants vote on the ledger. However, this is not a good idea because verifying a puzzle solution would require computing a hash over the entire ledger, which grows over time and would make verification increasingly expensive.

Instead, we would like the puzzle to be a short string that succinctly represents the ledger. In particular, we need a mechanism for compressing an arbitrarily long sequence of transactions into a fixed-size digest. A cryptographic hash function comes to the rescue once again, as compression is one of its primary applications.

More specifically, let H be a collision-resistant hash function (Chapter 7.4.1). A block B contains the following fields:

$$B = (x, h', nonce)$$

where x is a batch of transactions, h' is a hash of a predecessor block B', and *nonce* is the PoW. For simplicity, we omit several additional fields, such as the PoW target threshold T, the timestamp, and the software version information.

The above block B is a valid successor of block B' only if (i) $h' = H(B')$, and (ii) *nonce* is a solution to the puzzle defined by (the hashes of) B' and the new batch of transactions x, i.e.,

$$H(h' \mid H(x) \mid nonce) < T.$$

The predecessor relation among blocks induces a *chain* of blocks. The chain structure, together with the collision-resistance property of the cryptographic hash function, ensures that a block uniquely determines its entire chain of ancestor blocks. Any modification to a block changes its hash digest and invalidates all of its descendant blocks.

Any chain of blocks must start with a special block $(x_0, \perp, \perp)$, called the *Genesis block*, where x_0 is a hard-coded unpredictable string. A block's position in the chain is called its *height*. The Genesis block has height 0. If B is a valid successor of B', then the height of B is one greater than the height of B'.

Remark 9.2.1 Bitcoin was launched on January 3rd, 2009. The unpredictable string x_0 in the Genesis block is famously taken from a newspaper headline on that day: "The Times 03/Jan/2009 Chancellor on brink of second bailout for banks".

9.2.3 Longest Proof-of-Work Chain Wins

The Nakamoto consensus algorithm can be succinctly summarized by the *longest-chain rule*. Every honest node regards the longest PoW chain known to it as the candidate consensus outcome and attempts to extend that chain. We next describe the algorithm in more detail.

Mine on the longest chain. The process of searching for a nonce that produces a valid block is called *mining*. At any point in time, every node attempts to mine a new block that (i) extends a longest chain of blocks known to that node, and (ii) contains a batch of new valid transactions given the history of transactions in preceding blocks in the chain (see Chapter 9.1.2).

When a node successfully mines a block, it disseminates the newly mined block through the peer-to-peer gossip network. Upon receiving a new block, a node verifies its validity and updates its local view accordingly. If the newly received block becomes the new longest chain, a node disseminates the block (and any predecessor block that has not been disseminated) through the peer-to-peer gossip network, and starts mining on top of the new longest chain.

Because block propagation incurs network delays and multiple blocks may be mined concurrently, different nodes may temporarily disagree on the longest chain. Such disagreements are typically short-lived. Some node will mine a new block B and disseminate it to the network. Then, all nodes have the same longest chain again.

However, while B is propagating, it is possible that another node concurrently mines a competing block B' at the same height. In this case, the longest chains at different nodes may diverge in their last *two* blocks. Additional concurrent mining events may further extend both branches. Nevertheless, under appropriate assumptions about the PoW difficulty, network delays, and fraction of adversarial mining power, such divergent branches cannot grow indefinitely. Eventually, one branch becomes longer than all competing branches and is adopted by all honest nodes. As a result, nodes reconverge to a common longest chain.

The k-deep commit rule. The above discussion illustrates that a node cannot commit a block merely because it belongs to its current longest chain. A chain that is longest in a node's local view may later be overtaken by another chain, causing the node to abandon a few blocks on the former chain.

Fortunately, blocks become increasingly stable as they are buried deeper beneath subsequently mined blocks. For this reason, the Nakamoto consensus algorithm considers a block committed only if it belongs to a longest chain and a chain of at least k blocks

has been mined on top of it. In other words, the most recent k blocks in a longest chain are *not* committed, whereas all earlier blocks are committed. Here, k is a parameter that introduces a latency-security tradeoff. A larger k increases the delay between a block's creation and commit, but decreases the probability that a committed block is later reverted.

Remark 9.2.2 Another common way to state Bitcoin's commit rule is in terms of *confirmations*. A block itself counts as the first confirmation for the transactions within it. Each successor block in a chain is an additional confirmation. A transaction is considered committed when it gets a prescribed number of confirmations. The widely used heuristic of six confirmations hence corresponds to the 5-deep commit rule.

Algorithm 19: The Nakamoto consensus algorithm.

upon algorithm starts
    Initialize $\mathcal{C}$, the set of chains, to $\{B_0\}$ where B_0 is the Genesis block
    Initialize the longest chain $C^* = B_0$

continuously do
    Attempt to mine a block that extends C^*

upon mining or receiving a new valid block B (that extends some chain in $\mathcal{C}$)
    Add the chain ending at B to $\mathcal{C}$
    If the chain ending at B is longer than C^*, **then**
        Set C^* to be the chain ending at B
        Send to all neighbors any blocks in C^* that have not been sent
        Commit every block in C^* except the last k blocks

Algorithm 19 presents the high-level pseudocode of Nakamoto consensus. For clarity, it omits two important details, which we discuss next.

Difficulty adjustment. The Bitcoin protocol dynamically adjusts the difficulty of the PoW puzzle (i.e., the threshold T) so that, across the entire network, a new block is mined approximately once every ten minutes.

This adjustment is important because the total hash power devoted to Bitcoin can vary substantially over time. Indeed, Bitcoin's total hash rate has experienced dramatic growth over the years, as well as occasional declines due to factors such as government regulations and price fluctuations. If the difficulty is too high, block production becomes too slow, hurting both latency and throughput. Conversely, if the difficulty is too low, blocks are often mined concurrently, increasing the likelihood of competing chains and thereby degrading the algorithm's fault tolerance (see Chapter 10).

To maintain the target block interval, the Bitcoin protocol adjusts the difficulty periodically. At each adjustment, the protocol calculates the average block interval during the previous period and adjusts the difficulty based on whether, and by how much, the observed average block interval falls below or exceeds 10 minutes. The precise adjustment rule, while important, is somewhat ad hoc, so we omit the details.

Incentives. The Nakamoto consensus algorithm relies on a large population of miners repeatedly performing PoW computations. These computations require substantial capital investment in specialized hardware and consume significant amounts of energy. One may naturally ask why miners are willing to participate.

Bitcoin incentivizes miner participation through monetary rewards. When a miner successfully mines a block and that block becomes sufficiently deep in the longest chain, the miner receives a reward. The reward consists of two components: newly minted bitcoins and transaction fees. Together, these rewards provide miners with a direct economic incentive to contribute computational resources to the Bitcoin protocol.

Transaction fees serve an additional purpose beyond rewarding miners. They provide a mechanism for allocating a scarce resource: block space.

To limit network bandwidth consumption and storage requirements, the Bitcoin protocol imposes an upper bound on the size of each block. At any given time, users may collectively submit more transactions than can fit into a single block. When that happens, not all transactions can be included in the very next block, and a miner needs to decide which transactions to prioritize.

Bitcoin resolves this allocation problem through transaction fees. When creating (signing) a transaction, the payer specifies a fee that will be paid to the miner who includes the transaction in a block. Miners are naturally incentivized to prioritize transactions that offer higher fees. This mechanism creates a market for block space. Users effectively compete for inclusion by bidding for the limited block space. When demand is low, users can often enjoy prompt inclusion with low fees. When demand exceeds the available block space, fees rise as users compete for priority, with the resulting fee level determined by market forces.

9.2.4 Why the Algorithm Works

Intuitively, the Nakamoto consensus algorithm ensures safety and liveness because, under the assumption that honest nodes control a majority of the total hash power, a unique longest PoW chain should grow faster than any competing chains. The longest PoW chain represents the greatest cumulative amount of work (i.e., hash computation). As honest miners collectively contribute their hash power to extending the longest chain, it accumulates work faster than any competing chain that an adversary may attempt to construct. Blocks buried deep in the longest chain become increasingly stable and less likely to be dislodged by competing chains, thereby ensuring safety. The continued growth of the longest chain ensures liveness.

While the above intuition may be compelling, formalizing it rigorously is surprisingly challenging. The original Bitcoin paper provided valuable insight, but did not attempt a rigorous proof. Garay, Kiayias, and Leonardos presented the first rigorous proof for Nakamoto consensus under the idealized model of zero network delay. Most proofs that account for network delay are complex, and many rely on informal or heuristic arguments at key steps. We present a rigorous and relatively simple proof in the next chapter.

CHAPTER 10

Analysis of Nakamoto Consensus

This chapter proves that Nakamoto consensus solves SMR and analyzes its efficiency. Furthermore, we contrast Nakamoto consensus with classical Byzantine consensus algorithms to highlight its unique models, assumptions, techniques, guarantees, and trade-offs.

10.1 Correctness Proofs

This section proves that the Nakamoto consensus algorithm solves State Machine Replication (SMR, Definition 8.1.2). Among the three properties of SMR in Definition 8.1.2, liveness and client correctness are relatively easy to establish. Most of this section is devoted to the safety proof. We follow the simple proof by Guo and Ren.

10.1.1 Model and Overview

Modeling mining as Poisson processes. We follow the Bitcoin paper and model proof-of-work mining as a homogeneous Poisson point process in continuous time. A Poisson point process is a standard mathematical model for random events occurring over time. The process is characterized by a rate parameter $\lambda > 0$, which specifies the average number of events occurring per unit time. The property most relevant to our proof is that the time between consecutive events is independently distributed according to the exponential distribution with mean $1/\lambda$. That is, if T denotes the time until the next event, then

$$\Pr[T > t] = e^{-\lambda t}.$$

In the context of Nakamoto consensus, λ denotes the total mining rate of the network (honest and adversarial nodes combined). Concretely, $\lambda = 1/600$, meaning that the network produces one block every 600 seconds (ten minutes) on average.

We assume a single adversary controls all adversarial nodes. Let $\rho > \frac{1}{2}$ be the fraction of the total mining rate controlled by honest nodes, and let $1-\rho$ be the fraction controlled by the adversary. An honest node always mines to extend a longest chain to its knowledge. Ties are broken arbitrarily. The adversary may mine on any chain.

We say a block is *honest* if it is mined by an honest node and *adversarial* if it is mined by the adversary. Honest block arrivals and adversarial block arrivals are two independent Poisson point processes with rates $\rho\lambda$ and $(1-\rho)\lambda$, respectively. The combined block arrivals form a Poisson point process with rate λ, in which each block is honest with probability ρ and adversarial with probability $1-\rho$.

Remark 10.1.1 The model ignores fluctuations in the total mining rate and the effects of difficulty adjustment. The model also ignores incentives. Ideally, one would like to assume that the vast majority of nodes are *rational* rather than honest, and prove that everyone faithfully following the protocol is an equilibrium. However, attacks such as selfish mining seem to suggest that is not the case. Although mining power fluctuations, difficulty adjustment, and incentives are critical aspects of Nakamoto consensus, a rigorous treatment of these phenomena remains challenging.

The adversary's goal. Following the Bitcoin paper again, we assume the adversary aims to attack the safety of a particular target transaction tx. The safety of tx is violated if a block containing tx is committed and a different block (which may or may not contain tx) is also committed at the same height.

We assume all honest nodes use the k-deep commit rule with the same k. We assume tx appears in every node's view at time τ. We assume that tx offers a sufficiently high transaction fee that honest miners prioritize its inclusion in the next block they mine.

Non-lockstep synchrony. We use the standard non-lockstep synchrony model: every message is delivered within a known delay upper bound Δ. We abstract away the topology and internal operation of the gossip network: Δ bounds the end-to-end propagation delay, regardless of how many hops a message traverses. Specifically, in the Nakamoto consensus algorithm, if an honest node mines or receives a new block b that becomes the new longest chain at time t, it immediately disseminates b through the gossip network, and all honest nodes receive b by time $t + \Delta$.

Remark 10.1.2 The Bitcoin paper never specified a timing model. A reader may therefore wonder whether non-lockstep synchrony is the right choice.

We first observe that neither partial synchrony nor asynchrony is an option, because the Nakamoto consensus algorithm would fail in those models. Recall that partial synchrony permits unbounded network delays until an *unknown* time called GST, meaning that the adversary can choose GST after all the other parameters have been fixed. Then, the adversary can violate safety even with zero mining power: simply choose a very large GST to keep the network partitioned long enough so that honest nodes mine two chains of length k and commit conflicting blocks.

Some works in the literature incorrectly claim that Nakamoto consensus operates under partial synchrony or asynchrony, because they conflate those models with non-lockstep synchrony. In fact, it is not difficult to show that consensus is impossible in the permissionless setting under partial synchrony or asynchrony (Theorem 11.2.2).

This leaves synchrony as the only remaining mainstream timing model. We remark that this is the first time we use synchrony *without* the idealized abstraction of lockstep rounds. It is particularly appropriate here since nodes in Nakamoto consensus send messages at arbitrary times rather than in synchronized rounds.

This section establishes that synchrony is *sufficient* for the safety and liveness of Nakamoto consensus. Whether synchrony is necessary remains unknown. It is plausible that Nakamoto consensus operates correctly in a model weaker than synchrony yet stronger than partial synchrony. Identifying such an intermediate model remains an open problem.

Overview of the safety proof. The objective of the safety proof is to upper-bound the probability that the adversary successfully violates the safety of tx.

At a high level, a fully rigorous proof faces two challenges. The first one is network delay. Because blocks take time to propagate through the network, honest nodes may produce competing chains, which can help the adversary in subtle ways that are difficult to analyze. The second challenge is that the proof must hold against all possible adversarial strategies, including those we may never have thought of. The Bitcoin paper, for example, falls short in this regard, as it analyzes only one simple attack strategy, known as private mining.

We address these challenges in two steps. We begin by considering an idealized model with zero network delay. In this case, all honest nodes have identical views at all times, so honest mining never creates competing chains. This allows us to focus on the second challenge. We show that an augmented variant of the private-mining attack is optimal in the zero-delay model. Then, instead of reasoning about all possible adversarial strategies, it suffices to analyze this one simple attack.

We then return to the first challenge of network delays. We use a reduction technique that "gifts" certain honest blocks to the adversary so that private mining remains optimal even with network delays. The result for the zero-delay model now carries over, with some loss in tightness, to the actual model with network delays.

This two-step approach cleanly separates the two sources of difficulty. The zero-delay model obviates the need to reason about arbitrary adversarial strategies, while the reduction deals with the effects of network delays.

10.1.2 The Private Mining Attack

We describe below the private-mining attack against a target transaction tx.

Definition 10.1.3 (The private-mining attack) Starting from time 0, every adversarial block extends a longest chain that does not contain the target transaction tx. The adversary delays every honest block maximally (i.e., by Δ). All adversarial blocks are kept private until the adversary can violate the safety of tx by publishing all blocks.

Perhaps not immediately apparent from the above description, the private-mining attack consists of two stages. In the first stage, from time 0 to τ (before tx appears), the adversary tries to build a "lead" over the honest chain, formally defined as follows:

Definition 10.1.4 (lead) The lead (of the adversary) at time t is the height of the highest block mined by time t minus the height of the highest honest block mined by t.

By definition, the lead is never negative. In this stage of the private-mining attack, if the highest (private) adversarial block is higher than any honest block, the lead is positive, and the adversary mines on this highest adversarial block to try to increase the lead; otherwise, the lead is zero, and the adversary mines on a highest honest block to try to obtain a positive lead.

The second stage of the private-mining attack starts at time τ. Honest nodes will include the target transaction tx in the next honest block. The adversary tries to build a private chain that does not contain tx. If there ever comes a time, after an honest node commits tx, that the adversary's private chain is as long as the public honest chain, then

the adversary publishes its private chain and the attack succeeds in violating the safety of tx. If such an instance never occurs, then the private-mining attack on tx fails.

The attack considered by the Bitcoin paper is essentially the private mining attack with only the second stage.

Optimality of private mining under zero delay. We temporarily consider an idealized model in which the network delay is zero. Guo and Ren proved that, in this model, private mining is an optimal attack. That is, if any other attack can successfully violate the safety of the target transaction, so can the private-mining attack.

Theorem 10.1.5 When $\Delta = 0$, for every execution, if any attack violates the safety of the target transaction tx, then the private-mining attack also violates the safety of tx.

The full proof of Theorem 10.1.5 is rather involved and is omitted. The main intuition is that, when $\Delta = 0$, honest blocks are always "aware of" one another and never compete with one another. Formally, they satisfy the following property.

Property 10.1.6 Every honest block has a higher height than all previously mined honest blocks.

Proof. With zero delay, an honest miner sees all previously mined honest blocks and always mines at a higher height. □

Success probability of private mining under zero delay. We now calculate the private-mining attack's success probability when the network has zero delay.

Let L denote the adversary's lead at time τ. At any time, the next mined block is honest with probability $p = \rho$ and is adversarial with probability $q = 1 - p$, independent of any other block. Thus, the lead L evolves as a one-dimensional random walk with a reflecting barrier at 0: at each step, L increments with probability q and decrements (or stays at 0) with probability p. There are two cases based on the value of L.

If $L \geq k + 1$, the private-mining attack succeeds. The adversary can withhold its private chain until the honest chain picks up the target transaction tx and grows by k blocks to commit tx. At that point, the adversary releases its longer private chain, which overtakes the honest chain and violates the safety of tx.

If $L \leq k$, consider the first $2k + 2 - L$ blocks mined since time τ and let B denote the number of adversarial blocks among them. Immediately after these $2k+2-L$ blocks are mined, the adversary's private chain is deeper than the target transaction tx by $D_a = L+B-1$, while the public honest chain is deeper than tx by $D_h = 2k+2-L-B-1$. There are again two cases.

- If $L + B > k$, then $D_a \geq k \geq D_h$. The private-mining attack succeeds in this case. Once again, the adversary simply needs to wait (if necessary) for the honest chain to grow long enough to commit tx, and then release its private chain.
- If $L + B \leq k$, then $D_a < k < D_h$. By this point, tx has been committed in the honest chain and the adversary's private chain trails behind the honest chain by

$$D_h - D_a = 2\,(k + 1 - L - B)\,.$$

The adversary needs to catch up from this deficit for the attack to succeed. The remainder of the attack can be modeled by the same one-dimensional random walk

that describes the lead. The walk starts at 0, and at each step, increments with probability q (upon an adversarial block) and decrements with probability p (upon an honest block). Let M denote the maximum value ever reached by this random walk.

Let F denote the event that the private-mining attack succeeds. We have

$$\begin{aligned}\Pr(F) &= \sum_{i=0}^{\infty} \Pr(L=i) \cdot \Pr(F \mid L=i) \\ &= \Pr(L>k) + \sum_{i=0}^{k} \Pr(L=i) \cdot \Pr(F \mid L=i),\end{aligned}$$

where $\Pr(F \mid L=i) = \sum_{j=0}^{2k+2-i} \Pr(B=j \mid L=i) \cdot \Pr(M > 2k+1-2i-2j)$.

All the random variables and conditional random variables introduced above follow common distributions. The rest of the calculation is straightforward.

It is fair to assume that τ is not small, so that the distribution of L has converged to the stationary distribution of the random walk. The stationary distribution is a variant of the geometric distribution with parameter $p = 1-q > \frac{1}{2}$. (For the stationary distribution to exist, $p > 1/2$ is required; otherwise, the lead L will keep increasing.) For $i = 0, 1, \ldots$, its probability mass function (pmf) is

$$\Pr(L=i) = P_1(i;p) = \left(\frac{q}{p}\right)^i \left(1-\frac{q}{p}\right), \qquad [\mathbf{10.1}]$$

and its complementary cumulative distribution function (ccdf) is

$$\Pr(L>i) = \overline{F}_1(i;p) = \left(\frac{q}{p}\right)^{i+1}. \qquad [\mathbf{10.2}]$$

Conditioned on $L=i$, B is a binomial random variable with parameters $(2k+2-i, q)$. Hence, for every $i = 0, 1, \ldots$,

$$\begin{aligned}\Pr(B=j \mid L=i) &= P_2(j; 2k+2-i, q), \\ \Pr(B>j \mid L=i) &= \overline{F}_2(j; 2k+2-i, q). \qquad [\mathbf{10.3}]\end{aligned}$$

where P_2 and $\overline{F}_2$ denote the pmf and ccdf of the binomial distribution:

$$P_2(j;n,q) = \binom{n}{j} q^j (1-q)^{n-j}, \quad \overline{F}_2(j;n,q) = \sum_{l=j+1}^{n} P_2(l;n,q).$$

Plugging in [10.1], [10.2],and [10.3], we have

$$\begin{aligned}\Pr(F) = \overline{F}_1(k;p) + \sum_{i=0}^{k} P_1(i;p) \cdot \Bigg(&\overline{F}_2(k-i; 2k+2-i, 1-p) \\ &+ \sum_{j=0}^{k-i} P_2(j; 2k+2-i, 1-p) \cdot \overline{F}_1(2k-2i-2j+1; p) \Bigg)\end{aligned}$$

Although the expression is not in closed form, the pmf and ccdf involved are straightforward to compute, so the exact value of $\Pr(F)$ can be determined numerically.

Given the optimality of private mining when $\Delta = 0$, $\Pr(F)$ provides an upper bound on the probability of a safety violation under any arbitrary attack strategy when $\Delta = 0$.

10.1.3 Safety and Liveness under Network Delays

We now return to the actual model with $\Delta > 0$ network delays. As outlined in Chapter 10.1.1, we will reduce the safety in the Δ-delay model to the zero-delay model. To this end, we differentiate between *blocks mined by honest (resp. adversarial) nodes* and *honest (resp. adversarial) blocks*. We introduce a *rigged* model against honest nodes: We "gift" selected blocks mined by honest nodes to the adversary, and grant the adversary complete control over these blocks. The rigged model makes the adversary strictly more powerful because the adversary has the option of behaving honestly with these gifted blocks.

The gifted blocks are selected carefully so that Property 10.1.6 continues to hold and the calculation of $\Pr(F)$ remains tractable. The optimality of the private-mining attack relies solely on Property 10.1.6 and thus holds in the rigged model. It follows that the success probability of the private-mining attack upper-bounds the success probability of any attack in the rigged model. Since the rigged model only strengthens the adversary, this bound also applies to any attack in the actual model.

To specify which blocks are gifted to the adversary, we introduce the following notions.

Definition 10.1.7 (Tailgaters and laggards) Let block b be mined at time t_b. If no other block is mined during $(t_b - \Delta, t_b]$, then block b is a *laggard*; otherwise, block b is a *tailgater*.

Each block now has two attributes: whether it is mined by an honest or an adversarial node, and whether it is a tailgater or a laggard. In the rigged model, we convert all the tailgaters mined by honest nodes into adversarial blocks. A converted block is treated exactly like a regular adversarial block: the adversary can control what predecessor block it extends, and when it is revealed to honest nodes.

In other words, the only honest blocks in the rigged model are laggards mined by honest nodes; all other blocks are adversarial blocks. A key consequence is that now every honest block is received by all honest nodes before the next honest block is mined, i.e., Property 10.1.6 holds in the rigged model.

Remark 10.1.8 A tailgater is mined without knowledge of one or more previous blocks, so it may be at the same height as another honest block and may aid the adversary in creating two conflicting chains. But tailgaters do not always help the adversary. Thus, our reduction technique does not yield a perfectly tight bound.

Each block is mined by an honest node with probability ρ and by the adversary with probability $1 - \rho$. Recall from Chapter 10.1.1 that the inter-arrival times in a Poisson point process are independent and identically distributed exponential random variables with the same rate parameter. Therefore, each block is a laggard with probability $e^{-\lambda\Delta}$ and a tailgater with probability $1 - e^{-\lambda\Delta}$. Furthermore, whether a block is a laggard or tailgater is independent of whether it is mined by an honest or adversarial node as well as all other blocks' attributes. It follows that, in the rigged model, each block is an honest block (i.e., a laggard mined by an honest node) with probability

$$p = \rho e^{-\lambda\Delta}$$

and an adversarial block with probability $1 - p$, independent of all other blocks.

The calculation of $\Pr(F)$, the success probability of the private-mining attack, from Chapter 10.1.2 then remains essentially unchanged, once we plug in $p = \rho e^{-\lambda\Delta}$ (instead of $p = \rho$) to account for the tailgaters gifted to the adversary. The only other difference is that the random walk in the last stage may only need to reach $D_h - D_a - 1$, because the last honest block may not have been received by all honest nodes yet. Therefore, we have the following theorem establishing the safety of the Nakamoto consensus algorithm.

Theorem 10.1.9 (Nakamoto safety) Given the k-deep commit rule, the delay bound Δ, the total mining rate λ, and the fraction of honest mining rate ρ, as long as $p = \rho e^{-\lambda\Delta} > \frac{1}{2}$, a target transaction's safety can be violated with probability no greater than

$$\Pr(F) = \overline{F}_1(k;p) + \sum_{i=0}^{k} P_1(i;p) \cdot \left(\overline{F}_2(k-i; 2k+2-i, 1-p) + \sum_{j=0}^{k-i} P_2(j; 2k+2-i, 1-p) \cdot \overline{F}_1(2k-2i-2j;p) \right)$$

in the Nakamoto consensus algorithm, regardless of the adversary's attack strategy.

We next turn to liveness.

Theorem 10.1.10 (Nakamoto liveness) Given the delay bound Δ, the total mining rate λ, and the fraction of honest mining rate ρ, as long as $p = \rho e^{-\lambda\Delta} > \frac{1}{2}$, every transaction issued by an honest client is eventually committed.

Proof. By time t, the expected number of honest and adversarial blocks mined is $p\lambda t$ and $(1-p)\lambda t$, respectively. By Property 10.1.6, no two honest blocks share the same height. Hence, the length of every honest node's longest chain is at least the number of honest blocks minus one (as the last honest block may not have been received by all honest nodes yet). The last k blocks in a longest chain are not yet committed under the k-deep commit rule. Therefore, the expected number of honest blocks committed by an honest node at time t is at least

$$p\lambda t - 1 - (1-p)\lambda t - k = (2p-1)\lambda t - k - 1.$$

When $p > \frac{1}{2}$, this quantity grows linearly with t. Hence, honest nodes commit an unbounded number of honest blocks over time. Since each honest block contains new transactions, every transaction issued by an honest client is eventually committed. □

The way Nakamoto consensus supports clients differs somewhat from traditional BFT SMR algorithms such as PBFT. In those algorithms, committed decisions are accompanied by *transferable* cryptographic proofs, typically quorum certificates consisting of signatures from sufficiently many replicas. A client can obtain such a proof from any source–even a Byzantine replica–and verify it independently.

In contrast, the Nakamoto consensus algorithm does not provide transferable proofs for committed values. Instead, a client must communicate with sufficiently many miners (akin to replicas) so that at least one of them is honest. An honest miner will provide the client with the genuine longest chain, upon which the client can locally apply the k-deep commit rule to determine which blocks and transactions are committed. However, if the client communicates only with adversarial miners, it may be presented with a

fabricated longest chain and thereby be misled into accepting transactions that were never committed. This vulnerability is known as an eclipse attack, in which the adversary isolates a subset of clients (or miners) from the rest of the network.

Therefore, the Nakamoto consensus algorithm requires a stronger assumption to support clients: a client must be able to receive the current longest chain from at least one honest source. Under this stronger assumption, the Nakamoto consensus algorithm supports clients and implements State Machine Replication (SMR), as established by Theorems 10.1.9 and 10.1.10.

10.2 Efficiency Analysis and Parameter Choices

Latency and throughput. A significant drawback of the Nakamoto consensus algorithm is its high latency. Our safety proof shows that the probability of a safety violation decreases exponentially with k. The widely used heuristic of six confirmations only ensures a safety violation probability of roughly 0.1% against an adversary controlling 10% of the total mining power. A lower safety violation probability against a more powerful adversary may put k in the dozens or hundreds.

Bitcoin sets the average block interval to be 10 minutes. Under this parameterization, six confirmations correspond to a latency of roughly one hour, whereas values of k in the hundreds imply a latency on the order of days.

Another limitation of the Nakamoto consensus algorithm is its low throughput. Bitcoin imposes an upper limit on the size of each block. Originally, the limit was 1 MB. Subsequent upgrades switched to a more complex accounting rule that permits blocks of roughly 1.5–2 MB. Combined with Bitcoin's 10-minute average block interval, the throughput is about 3–7 transactions per second. This is several orders of magnitude lower than that of modern centralized payment systems, which can process thousands of transactions per second.

Parameter choices. The preceding analysis of latency and throughput raises a natural question: can we improve the latency and throughput of Nakamoto consensus by adjusting the protocol parameters? For example, one might consider reducing the block interval or increasing the block size. Many projects that mimic or fork Bitcoin have pursued this approach, but in many cases, their parameter choices appear misguided. As we show next, the safety proof in Chapter 10 provides principled guidance for parameter selection and sheds light on the trade-offs involved.

Recall that tailgater blocks may not contribute to safety. The fraction of tailgaters is $1 - e^{-\lambda\Delta}$, where $1/\lambda$ is the average block interval and Δ is an upper bound on block propagation delay. A shorter block interval corresponds to a larger λ. Larger blocks take longer to propagate and call for a larger Δ. Both effects increase the fraction of tailgaters, which, to a first approximation, implies that a larger fraction of mining power is "wasted" because of block propagation delays. This intuition is also reflected in Theorem 10.1.9, which requires $\rho e^{-\lambda\Delta} > \frac{1}{2}$. As $\lambda\Delta$ increases, a larger fraction ρ of honest mining power is required to satisfy this condition, and correspondingly a smaller fraction of adversary mining power can be tolerated. Thus, the choices of block interval and block size present a trade-off between efficiency and security.

Having understood the efficiency-security trade-off, one could argue that Bitcoin's parameter choices are overly conservative in favor of security. Empirical measurements suggest that a block reaches 90% of the network within 2–4 seconds. If we take $\Delta = 10$ seconds as an upper bound on block propagation delay, then the fraction of tailgater blocks, or "wasted" mining power, is around 1%. If one assumes that the adversary's mining power is well below 50% (say, 25%), then a protocol can afford a larger fraction of wasted mining power in exchange for improved efficiency.

Suppose one decides to trade some security margin for better efficiency. The next question is whether this should be achieved by shortening the block interval or by increasing the block size. Is one approach preferable over the other, or are they essentially equivalent? From a throughput perspective, they are equivalent. Doubling the block size or doubling the block generation rate has the same effect on throughput. The two approaches are also roughly equivalent from a security perspective, since security and fault tolerance are governed primarily by their product $\lambda\Delta$ (more precisely, by $e^{-\lambda\Delta}$). But the two approaches differ significantly in their effects on latency. A shorter block interval directly and proportionally reduces latency, whereas a larger block size provides no latency benefit.

Therefore, taking Bitcoin's parameters as a baseline, shortening the block interval is superior to increasing the block size. In fact, additional improvements can be obtained by *decreasing* the block size and further shortening the block interval. This reduces latency without sacrificing throughput or security. Of course, this strategy cannot be pushed to the extreme, since reducing the block size beyond a certain point no longer yields a proportional reduction in propagation delay.

Unfortunately, almost all projects and proposals either kept the block size the same as Bitcoin or went in the opposite direction by increasing it, partly due to a lack of principled methodology. The only exception is PoW-based Ethereum (2015–2022), which set the average block interval to about 13 seconds and had block sizes around 100–200 KB. Compared with Bitcoin, the parameters used by PoW Ethereum increase throughput by a factor of 5–10 and reduce latency by approximately a factor of 40, at the cost of lowering the fault tolerance from almost 50% to around 40–45%.

Communication complexity. The Nakamoto consensus algorithm has low communication overhead, thanks to the synergy between the algorithm and the sparse gossip network (Chapter 10.3 discusses this further). Each honest node sends (or forwards) each block only to its neighbors. If there are n miners in total, each miner has d neighbors, and each block contains ℓ bits, then the amortized communication complexity is $O(nd\ell)$ per block.

Computation complexity and energy consumption. A major drawback of the Nakamoto consensus algorithm is its high energy consumption. Miners continuously perform PoW computations in a race to mine the next block, resulting in substantial electricity consumption. Recent estimates (2025) place Bitcoin's annual electricity consumption at over 100 TWh. If Bitcoin were a country, it would rank among the top 30 in terms of electricity consumption, comparable to countries such as Sweden or Pakistan.

10.3 Identifying the Innovations of Nakamoto Consensus

Nakamoto consensus is a novel solution to the Byzantine Fault Tolerant State Machine Replication (BFT SMR) problem. To accurately understand its key innovations, we must contrast it with classical BFT SMR algorithms to identify the novel and desirable features that had eluded decades of prior research on Byzantine consensus. We now examine the major features commonly associated with Nakamoto consensus and evaluate their novelty and merit.

The most distinctive feature of Nakamoto consensus is its *permissionless* nature. Although this feature is widely recognized and discussed, its precise meaning is often left vague. The common description is that "nodes can join and leave at will". This statement, in fact, captures two distinct properties. We now separate them and describe each more precisely.

Permissionless participation. Anyone can participate in the Nakamoto consensus algorithm without obtaining permission from any entity. Strictly speaking, there are still practical barriers to entry. At a minimum, one needs a computer with Internet access and hardware capable of computing hashes (in practice, at a sufficiently high rate and with reasonable energy efficiency). However, these barriers are *external* to the consensus algorithm. In other words, the Nakamoto consensus algorithm imposes no restrictions on who may participate.

When participation is unrestricted, there is no notion of participant set (i.e., the set of parties eligible to participate). Indeed, operating without knowledge of the participant set appears to be an essential property of any truly permissionless consensus algorithm.

In sharp contrast, classical BFT SMR algorithms such as PBFT have an explicitly defined set of eligible participants. At every point in time, the participant set is agreed upon and known to all. Only members of this participant set are allowed to execute the consensus algorithm. The participant set may change over time through reconfiguration (see Chapter 8.4), but any such change must be agreed upon through the consensus algorithm and is typically subject to approval by current participants. In other words, new participants must obtain permission from current participants in order to join the consensus algorithm.

Sporadic participation. There is a second interesting property captured by the statement "nodes can join and leave at will," one that is orthogonal to permissions for participation. To isolate this property, imagine a setting in which the set of parties eligible to perform PoW is fixed and known, so that permissions and barriers to entry are no longer relevant. The setting further allows any number of miners to become inactive, for example, by shutting down their mining hardware. Likewise, an inactive miner may resume mining at any time. Nakamoto consensus continues to function correctly as long as the majority of the mining power among *active* miners remains honest. Notably, the active mining power may constitute only a small fraction (say 1%) of the total eligible mining power.

In contrast, classical BFT SMR algorithms always require that a majority or supermajority of the eligible participants be honest *and active*. A participant that temporarily becomes inactive is considered faulty, and the algorithm stops making progress if too many participants become inactive.

Pass and Shi first isolated and formalized this new property, and aptly termed it *sporadic participation*. This property has also been referred to as *dynamic availability*, *dynamic participation*, or *fluctuating participation*. We remark that, although the distinction is not critical for Nakamoto consensus, it is helpful to distinguish between permissionless participation and sporadic participation. The former concerns who *is eligible* to participate, whereas the latter concerns which eligible participants *happen to be active* at any given time.

Reduced communication in a sparse communication network. Bitcoin is well known for using a sparse gossip-style communication network. There is indeed some novelty in this aspect, but exactly where the novelty lies is somewhat subtle.

While Nakamoto consensus may have been the first consensus algorithm to explicitly employ a sparse gossip network, this alone is not the innovation. The communication network has traditionally been regarded as an implementation detail rather than part of the consensus algorithm. In principle, any earlier consensus algorithm could have been deployed over a sparse network. This was rarely done because a sparse network requires a stronger assumption, namely, that the adversary cannot strategically corrupt nodes to disconnect the communication graph. Classical BFT algorithms generally do not make this assumption. This also explains why the $O(nd)$ message complexity of Nakamoto consensus (Chapter 10.2) does not contradict the quadratic lower bound in Chapter 7.1, since that lower bound is proved in a model that allows such attacks.

The innovation lies not in the use of a sparse network itself, but in the way Nakamoto consensus is designed to exploit it. To see this, imagine that miners were connected by pairwise direct links, setting aside the impracticality of such a network in a permissionless setting. The node that successfully mines a block sends it to all nodes. But the other nodes cannot trust the successful miner to send the block to everyone, so each of them must forward the block to all nodes. This results in $O(n^2)$ messages per block. The sparse gossip network reduces the number of messages to $O(nd)$ for this send-and-forward step.

This improvement does not necessarily extend to other communication patterns. For example, if one participant needs to send a message to all other participants (but no one needs to forward the message), direct communication costs $O(n)$ messages, whereas propagation over a sparse network costs $O(nd)$ messages. Thus, for most algorithms, replacing a fully connected network with a sparse one does not reduce communication. Nakamoto consensus is an exception because of its send-and-forward communication pattern.

Undesirable features of Nakamoto consensus. Not all features of the Nakamoto consensus algorithm (novel or otherwise) are desirable. For example, the substantial energy consumption of PoW, while novel for consensus algorithms, is widely recognized as a major drawback. But some other drawbacks of Nakamoto consensus are still sometimes mistaken for desirable features. One example is probabilistic and gradual safety. As shown in Chapter 10, there is always a nonzero probability that a block will be reverted, and the probability decreases exponentially as the block gets buried deeper in the longest chain. Viewed in isolation, this is an undesirable property and directly contributes to the long latency of Nakamoto consensus. All else being equal, one would much prefer the conventional deterministic safety, under which a decided block can never be reverted within the assumed model. Probabilistic and gradual safety are also not novel. One of the earliest

Feature	Novel?	Desirable?
Participant set can change	×	✓
Permissionless participation	✓	✓
Sporadic participation	✓	✓
Use of a sparse communication network	×	
Reduced communication in a sparse network	✓	✓
Substantial energy consumption from PoW	✓	×
Probabilistic safety	×	×
Safety increases over time	×	×

Table 10.1. Distinguishing the true innovations of Nakamoto consensus (features that are novel and desirable) from other commonly discussed features.

randomized Byzantine agreement algorithms, due to Rabin (1983), exhibits probabilistic and gradual safety. They are uncommon in the Byzantine consensus literature precisely because they are considered undesirable and avoided whenever possible. Indeed, Rabin's paper presents a variant that achieves deterministic safety.

Misattributed features. Finally, some properties are often mistakenly attributed to Nakamoto consensus. One common misconception is that Nakamoto consensus is an asynchronous or partially synchronous consensus algorithm. As discussed in Remark 10.1.2, this is not the case. Another common misconception is that Nakamoto consensus weakens the consistency guarantee (for example, to eventual consistency). This is incorrect. Nakamoto consensus, like all SMR algorithms, provides the strongest type of consistency.

What about blockchain? "Blockchain technology" has often been touted as the defining innovation of Bitcoin. This characterization hinders an accurate understanding of Bitcoin's true innovations, so I have deliberately avoided the term.

The confusion stems in part from the evolution of the word *blockchain*. In the original context of Bitcoin, the term literally meant "a chain of blocks." Over time, however, its meaning has broadened to refer to any system that achieves consensus on a chain of blocks. In this sense, *blockchain* is, for the most part, a new name for *State Machine Replication* (SMR). Although SMR is itself somewhat opaque jargon, replacing it with the newly coined word "blockchain" creates at least two sources of confusion.

The first problem is that the term *blockchain* does not accurately capture the essence of the Bitcoin algorithm. While Bitcoin indeed builds a chain of blocks, each Bitcoin block contains several distinct components: (i) transactions (which are application data), (ii) a hash digest of the previous block, and (iii) a PoW solution. When the word "blockchain" is used to describe the algorithm, it is unclear which of these chains plays the central role.

For readers of this book, the answer may seem obvious: the central object in Nakamoto consensus is the PoW chain. The hash digest chain is merely a way to succinctly represent the entire ledger, an elegant trick, but neither a novelty nor the essence of the algorithm.

In the context of Bitcoin, using "block chain" in place of "PoW chain" may be harmless. However, confusion can arise when it comes to later systems that move away from PoW. Simply linking blocks together with hash pointers creates a "blockchain" structure,

but it does not constitute a consensus algorithm. Beginners may fail to realize that merely hash-chaining blocks together, without a proper consensus algorithm, does not yield a secure virtual currency.

Remark 10.3.1 Satoshi Nakamoto's own writings reflect this understanding. In the Bitcoin white paper and mailing list discussions, Satoshi repeatedly emphasized the "longest proof-of-work chain" as the key algorithmic idea behind Bitcoin, perhaps best summarized by the following quote.

> The proof-of-work chain is a solution to the Byzantine Generals' Problem.
>
> — Satoshi Nakamoto

Satoshi frequently wrote "longest chain" or simply "chain" as shorthand for the longest proof-of-work chain. By contrast, the phrases "chain of blocks" or "block chain" appear only once in the white paper and once in a mailing-list discussion (the latter occurred only in response to a question that had used the phrase).

The converse misunderstanding is also problematic. Many SMR algorithms involve no chains of any kind (for example, the composition approach, the view-based approach, and the ACS approach in Chapter 8). Labeling post-Bitcoin advances as "blockchain technology" encourages beginners to overlook the classic consensus literature and creates the impression that Bitcoin introduced an entirely new class of technology. This obscures the true innovations of Bitcoin and Nakamoto consensus. To appreciate what is genuinely novel about Nakamoto consensus, we must first recognize that it solves the same fundamental problem as classical BFT SMR algorithms. Only then can we meaningfully compare Nakamoto consensus with its predecessors to identify its true innovations (as done in Chapter 10.3).

CHAPTER 11

Consensus under Sporadic Participation

In Chapter 10.3, we have identified paradigm-shifting innovations of Nakamoto consensus: (i) permissionless participation and (ii) sporadic participation. (Reduced communication in a sparse network pertains to efficiency rather than a fundamental paradigm shift.)

Although proof-of-work (PoW) has proven to be a remarkably effective mechanism for achieving consensus with those two paradigm-shifting properties, Bitcoin's substantial energy expenditure has raised significant concerns and prompted a shift away from PoW. This chapter explores alternative designs that aim to achieve permissionless and/or sporadic participation without PoW.

11.1 Overview of Proof-of-Work Alternatives

Roles of Proof-of-Work Before considering replacements for PoW, it is helpful to first understand what roles PoW plays in Nakamoto consensus and how essential those roles are. This will help us evaluate whether alternative techniques can fulfill those same roles.

PoW serves two primary roles in Nakamoto consensus.

- *Sybil resistance.* This is perhaps the most fundamental role of PoW. In a system with permissionless participation, identities are essentially free to create. PoW ties eligibility to participate to a scarce physical resource (i.e., computation power), so creating fake identities does not increase a participant's influence. In this sense, PoW is the key mechanism that enables permissionless participation.
- *Leader election.* Many consensus algorithms rely on a leader to propose the next value or block. When the participant set is known and agreed upon, leader election can be as simple as a round-robin rotation. With permissionless participation, however, leader election is more challenging, since no one knows the complete set of participants. PoW provides a simple and mostly *fair* leader-election mechanism that requires no coordination among nodes.

Remark 11.1.1 PoW serves at least two other roles in Nakamoto consensus. These two roles are less critical because they can be fulfilled by other well-established mechanisms such as digital signatures and hardware clocks. We mention them to present a more complete picture of PoW's roles in Nakamoto consensus.

- *Tamper resistance.* Nakamoto consensus requires that a majority of the mining

power is controlled by honest nodes. This assumption can be viewed as the familiar honest-majority assumption, $f < n/2$, adapted to the PoW setting. Note that the Nakamoto consensus algorithm does not use digital signatures. (The virtual currency application uses digital signatures to authorize transactions, but the underlying Nakamoto consensus algorithm does not use signatures in any way.) Yet recall that Byzantine fault tolerance beyond one-third is impossible without signatures (Chapter 6.1.2). PoW circumvents this impossibility by providing a guarantee similar to that of signatures. With signatures, if an honest party casts a signed vote for a value, the adversary cannot alter that vote. Likewise, an honest miner "votes" for a chain by contributing work to it, and the adversary cannot undo or tamper with that work.

- ◇ *Intrinsic clock.* Classical synchronous and partially synchronous consensus algorithms (after GST) require parties to have loosely synchronized clocks to enforce lockstep rounds or fixed-duration views. The core steps of Nakamoto consensus, excluding the difficulty adjustment part, do not involve clocks or real time. Nodes take actions only upon finding or receiving new PoW solutions. In this sense, PoW serves as an intrinsic clock for Nakamoto consensus.

A myriad of alternatives to PoW have been proposed, almost always under the name "proof-of-X". Some preserve the philosophy of PoW to varying degrees, while others merely imitate the nomenclature. Below, we discuss proposals that have coherent design rationales.

Proof-of-useful-work. Proof-of-useful-work is an appealing alternative that stays within the PoW framework. The idea is to replace hash computations with useful computational tasks such as protein folding and machine learning training. If successful, the energy expended by Nakamoto consensus would be put to productive use.

Unfortunately, no suitable candidate has been found, as it is difficult for a useful computational task to simultaneously satisfy all the requirements of a PoW puzzle (easy to generate, difficult to solve, easy to verify, and adjustable in difficulty). A deeper objection contends that the "uselessness" of hash computation may be essential to the security of PoW-based consensus. Useful computations carry external value and effectively subsidize an adversary who mines a competing chain to attack consensus.

Proof-of-space. Proof-of-space aims to emulate PoW by using storage, rather than computation, as the scarce physical resource. A proof-of-space is a succinct proof that a participant has dedicated a certain amount of storage capacity. Variants in this category include proof-of-space-time and proof-of-replication.

Because it is also based on physical resources, proof-of-space can provide Sybil resistance (and tamper resistance), much like PoW. But unlike PoW, proof-of-space does not by itself provide leader election (or an intrinsic clock). Proving possession of storage also turns out to be more challenging than proving expenditure of computation. Therefore, a carefully designed consensus algorithm based on proof-of-space can achieve both permissionless and sporadic participation, but the algorithm and its analysis would be more complex and rely on stronger assumptions than their PoW-based counterparts. We omit a detailed discussion of proof of space.

Proof-of-stake. By far the most studied and adopted alternative is *proof-of-stake*. Although it borrows the naming convention of PoW, proof-of-stake has little in common with PoW and instead marks a return to the classical model of *permissioned* consensus.

Proof-of-stake refers to the idea that eligibility to participate is tied to ownership of the cryptocurrency, i.e., stake in the system. Participants usually have different weights, proportional to the amount of cryptocurrency they hold, but that is a minor detail for most algorithms. Without loss of generality, we assume each participant has one unit of cryptocurrency. Since ownership is publicly recorded and agreed upon in a cryptocurrency system, abstracting away the cryptocurrency context, the *proof-of-stake* model simply assumes that all eligible participants are explicitly listed, agreed upon, and publicly known, just as in the classical model.

To be more concrete, eligible participants are identified by their public keys (that hold stake). The set of eligible public keys is publicly known and agreed upon. A *proof-of-stake* is simply a digital signature that authenticates an eligible participant. As such, proof-of-stake by itself does not provide leader election (or an intrinsic clock), and Sybil resistance is no longer relevant with permissioned participation. (It does provide tamper resistance, since that is the primary function of a digital signature.)

The set of eligible participants may evolve over time as stakes are bought and sold, but anyone wishing to become a participant must acquire stake from a current participant, and the corresponding stake transfer must become a consensus decision. Both steps amount to obtaining permission from existing participants. This process is no different from classical SMR reconfiguration (see Chapter 8.4).

Even after abandoning PoW and giving up permissionless participation, there are opportunities to draw inspiration from the rest of Nakamoto consensus and accommodate sporadic participation. The remainder of this chapter explores this direction.

11.2 The Sporadic Participation Model

Consider Byzantine Fault Tolerant State Machine Replication (BFT SMR) in a permissioned setting with *reconfiguration*. Each party is identified by a public key. Initially, a publicly known set of N parties constitutes the eligible participant set, denoted $\mathcal{P}$. $\mathcal{P}$ may evolve over time as *reconfiguration commands* are decided in the SMR log. As usual, some parties may be Byzantine and controlled by a single adversary. So far, the model is identical to the classical model in Chapter 8.

The defining feature of the *sporadic participation model*, first formalized by Pass and Shi as the *sleepy model*, is that honest parties may transition between being *active* and *inactive* without notice. An inactive party neither sends nor receives messages, and does not execute the algorithm. When a party becomes active, either for the first time or after a period of inactivity, it retrieves relevant past messages from other parties and then resumes execution.

To make the model more robust, we allow the adversary to control when each party becomes active or inactive. But such transitions may occur only at round boundaries. We remark that these parties are honest and have no intention to attack the system. Going inactive abruptly in the middle of a round, e.g., after sending some but not all messages in that round, is considered Byzantine behavior.

Fault tolerance is measured among the set of active parties. Let f denote the number of Byzantine parties, and let h_t denote the number of honest eligible participants who are active in round t. We say an algorithm tolerates a ρ fraction of Byzantine faults under sporadic participation if it achieves its desired properties whenever

$$f < \rho\,(h_t + f)$$

holds for every round t. As an example, the honest majority condition becomes $f < h_t$ for all t.

It is worth emphasizing that the number of active parties can be a very small fraction of the total number of eligible participants; that is, $n_t \ll N$ for some or all t. Moreover, honest parties do not know which participants are active, inactive, or Byzantine, nor do they know how many are active, inactive, or Byzantine. This has a fundamental consequence. Most of the classical algorithms studied in Chapters 3–8 cannot handle sporadic participation. They assume all honest parties are always active, and wait for messages from at least $N/2$ or $2N/3$ distinct parties at certain steps in the algorithm. Under sporadic participation, there may not be enough active parties to ever reach these thresholds, causing the algorithm to get stuck. The only exception is the Dolev-Strong broadcast algorithm in Chapter 2. It works if one sets $f = N$, essentially treating all inactive parties as faulty. But this results in a round complexity of N, making the algorithm impractical if $n_t \ll N$.

Remark 11.2.1 The model in this chapter assumes only honest parties can be inactive, while all Byzantine parties remain active throughout the execution. A more realistic model would allow Byzantine parties to be inactive as well. Another modeling question concerns parties who behave maliciously *after leaving* the eligible participant set, for example, after selling their stake. Since they are no longer eligible participants, one may argue that they should not count towards the adversary's corruption budget f.

Observe that Nakamoto consensus naturally accommodates these weaker models. The adversarial mining rate is allowed to fluctuate, provided it stays below the honest mining rate. A node that is not participating, i.e., not mining, does not count toward the adversary's corruption budget. Without PoW, however, these weaker models introduce substantial additional challenges on top of an already difficult problem. They remain active research topics and are beyond the scope of this book.

Without PoW serving as an intrinsic clock, we assume lockstep rounds, which require loosely synchronized hardware clocks among parties. We assume synchrony among honest and active parties. By the end of round t, every honest party active in round t receives all messages sent to it during round t.

It is worth emphasizing that the sporadic participation model cannot be combined with partial synchrony (let alone asynchrony) because such a combination renders consensus impossible.

Theorem 11.2.2 No consensus algorithm supports sporadic participation under partial synchrony.

Proof. We prove the theorem for the broadcast problem as an example. The proof is similar for other consensus problems. Divide the eligible participants into two groups. Let

GST be sufficiently large and delay all messages between the two groups until after GST. All messages sent within a group are delivered on time. From each group's perspective, the execution is indistinguishable from one in which the other group is entirely inactive. If an algorithm supports sporadic participation, each group eventually decides a value. Suppose the sender is in the first group and has input 1, which does not equal the default value $\perp$. By validity, the first group decides 1 and the second group decides $\perp$, respectively, violating safety. □

Note that the proof holds even when no party is faulty and no reconfiguration occurs.

11.3 Longest-Chain Consensus without Proof-of-Work

A natural idea is to adopt the longest-chain-wins design and the k-deep commit rule from Nakamoto consensus but remove PoW. This idea was discussed in online forums as early as 2011 and in numerous technical articles ever since. These algorithms are collectively known as *longest-chain proof-of-stake*. It is worth emphasizing that the core algorithmic idea is the longest-chain-wins design inherited from Nakamoto consensus. The use of digital signatures to authenticate eligible participants (proof-of-stake) is standard and unremarkable. We therefore refer to these algorithms simply as *longest-chain* algorithms.

While the high-level idea is deceptively simple and appears similar to Nakamoto consensus, designing a full algorithm and proving safety are considerably more complex and subtle than those of Nakamoto consensus. We describe a simplified version of the algorithm presented by Pass and Shi in two papers titled *Sleepy Consensus* and *Snow White*.

Algorithm 20: High-level pseudocode of a longest-chain consensus algorithm.

upon algorithm starts
  Initialize $\mathcal{C}$, the set of chains, to $\{B_0\}$ where B_0 is the Genesis block
  Initialize the longest chain $C^* = B_0$

for each round $t = 1, 2, 3, \ldots$
  If the party is a leader of round t
    The party creates and signs a new block extending C^*

upon creating or receiving a new valid block B (that extends some chain in $\mathcal{C}$)
  Add the chain ending at B to $\mathcal{C}$
  If the chain ending at B is longer than C^*, **then**
    Set C^* to be the chain ending at B
    Send to all neighbors any blocks in C^* that have not been sent
    Commit every block in C^* except the last k blocks

Algorithm 20 gives the high-level pseudocode for a longest-chain consensus algorithm. It makes minimal changes to Nakamoto consensus. The only major change is that PoW mining is replaced by a leader-election scheme (which we specify later) and lockstep rounds. Accordingly, a block has the form

$$B = (x, h', t)\,,$$

where x is a batch of transactions, h' is a hash of a predecessor block, and t is the round in which the block is created. A block created in round t must be signed by a party who is simultaneously (i) an eligible participant according to the chain it extends, and (ii) a leader of round t. Blocks in a chain must have strictly increasing round numbers and must not come from future rounds.

Leader election. The challenge and subtlety of longest-chain consensus without PoW lie almost entirely in the leader-election step. Having returned to the permissioned setting, one could consider electing leaders simply by a round-robin rotation or another predetermined schedule. Such schemes, however, frequently elect *inactive* parties, wasting those rounds and stalling progress. Ideally, only *active* parties should be elected as leaders.

Pass and Shi propose the following leader-election scheme. Party i with public key pk_i is a leader of round r if and only if

$$H\left(\mathsf{pk}_i, r\right) < T,$$

where H is a cryptographic hash function and T is a publicly known threshold.

Note that this scheme does not guarantee a unique leader in every round. A round may have zero, one, or multiple leaders. This behavior resembles PoW-based leader election in Nakamoto consensus. With PoW, it is possible that no miner finds a puzzle solution for an unusually long period, and it is also possible that multiple miners find puzzle solutions around the same time. The former is analogous to a round with no leader, and the latter is analogous to a round with multiple leaders. As in PoW, the threshold T controls the expected number of leaders and may be periodically adjusted. When T is set/adjusted appropriately, each round has a constant probability of having a unique leader.

A key insight of Pass and Shi is that leader election should depend only on the round number and a party's identity. It should *not* depend on the chain being extended. Many early proposals involve the chain in leader election, e.g, $H\left(\mathsf{pk}_i, r, h'\right) < T$ where h' is the hash of the block the party attempts to extend. Such a design would yield a broken or substantially weakened consensus algorithm. A malicious leader can try many candidate blocks containing different transactions until it finds one that elects itself again in the next round. A less problematic variant includes block headers, but not transactions, in the leader-election computation. Although the previous attack does not apply, the adversary can still manipulate the block header to increase its probability of being elected well beyond its share of stake.

Predictability of leader identities. One drawback of the above leader-election scheme is that all future leaders are publicly predictable. This raises a concern that an adversary capable of adaptive corruption (Chapter 1.3.5) could trivially break the algorithm by corrupting all future leaders before they produce blocks. Two approaches have been proposed to mitigate this problem.

The first one is to incorporate slowly evolving randomness into the leader-election hash function. This way, the adversary can only predict leaders in the near future. Assuming corruption takes time, the algorithm can now withstand adaptive corruption to some extent. The challenge is that generating common randomness is itself a difficult problem, especially with sporadic participation. Existing proposals typically allow the adversary to bias the randomness to some extent, and the impact of such bias is not fully understood.

The second approach, introduced by Micali in the Algorand paper, employs *Verifiable Random Functions* (VRF). A VRF can be viewed as a digital signature whose output is pseudorandom yet publicly verifiable. Each party can privately determine whether it is a leader for any given round. The identities of leaders remain hidden until they reveal their VRF proofs in the corresponding round, after which anyone can verify their election. Again assuming corruption takes time, we can expect a leader to have completed its job before being adaptively corrupted.

Analysis sketch. The intuition behind the algorithm is essentially the same as that of Nakamoto consensus. Honest active parties work together to extend the longest chain and they outnumber Byzantine parties, so the longest chain tends to grow faster than any competing chain that the adversary may attempt to construct. The full correctness proofs are highly involved and thus omitted.

Compared to Nakamoto's PoW-based longest-chain algorithm, Algorithm 20 eliminates the energy consumption of PoW at the cost of giving up permissionless participation. It inherits most of the other advantages and disadvantages from Nakamoto consensus:

- Support for sporadic participation,
- Low communication over sparse networks, and
- Long latency (in fact, longer than Nakamoto consensus; see the remark below).

Remark 11.3.1 The adversary is more powerful here than in Nakamoto consensus. Whenever a malicious party is a round leader, it can create multiple blocks extending different chains in that round. As a result, Algorithm 20 requires a larger k in the k-deep commit rule to achieve the same safety-violation probability as Nakamoto consensus.

Remark 11.3.2 It is possible to solve consensus under sporadic participation with a different set of efficiency trade-offs. Momose and Ren adapt the quorum-based approach to the sporadic participation model and achieve constant good-case latency but incur higher communication costs.